

Adaptive Design of Embodied Robotic Systems for Real-Time Self-Modeling Toward Swarm Coordination

Master's Thesis
Master's Program in Data Science
HSE University

2026

Abstract

Embodied robotic systems depend on internal self-models that map motor commands, morphology, and sensory observations into actionable predictions for control. In practical deployment these models degrade continuously. Calibration slip, joint backlash, link bending, structural wear, sensor bias, and post-damage deformation create embodiment drift: the persistent divergence between the robot assumed by the controller and the robot that physically exists. Recent visual self-modeling methods based on Neural Radiance Fields and 3D Gaussian Splatting improve morphological expressiveness, but their rendering pipelines impose a critical latency bottleneck. A robot operating near a 1 kHz closed-loop regime has about one millisecond for perception, diagnosis, and actuation update; reproduced reconstruction-based baselines remain far outside this limit.

This thesis investigates adaptive design for real-time robotic self-modeling by replacing volumetric reconstruction with direct sensorimotor decoding. Baseline experiments first quantify the latency gap. The Free-Form Kinematic Self-Model reaches 17.35 dB PSNR at 5.22 FPS, while the kinematic 3D Gaussian Splatting implementation reaches 17.01 dB PSNR at 37 FPS and exhibits isotropic primitive artifacts. The proposed NeuroKin architecture bypasses explicit 3D scene reconstruction and directly maps joint-state inputs to visual and spatial self-model outputs. This design reaches 21.88 dB PSNR at 7,400 FPS, corresponding to 0.135 ms inference latency, crossing the sub-millisecond threshold required for high-frequency control integration.

The thesis further evaluates the collective implication of this individual latency breakthrough through a 100-agent sequential swarm. No experiment assumes concurrent spatial deployment. Coordination is instead modeled as temporal stigmergy: each agent reads and updates a shared `factory_state` ledger tracking `consecutive_failures`. When failures accumulate, subsequent agents enter localized recovery mode by shifting targets and expanding tolerances. Under progressive fault injection, the baseline inverse-kinematics chain collapses after damage, while the NeuroKin sequence preserves adaptive operation. These results establish the computational prerequisite for future physical swarm coordination: fast local self-diagnosis coupled to minimal state sharing across agents.

Contents

Abstract	i
1 Introduction	1
1.1 Background	1
1.2 The Computational Bottleneck	2
1.3 Problem Statement and Scope	3
1.4 Contributions	4
1.5 Research Questions	5
1.6 Thesis Organization	6
Chapter 1 References	7
2 Literature Synthesis	8
2.1 Evolution of Self-Modeling	8
2.2 The Latency Paradox	9
2.3 Theoretical Foundations of Swarm Coordination	11
2.4 Visual Self-Modeling and Functional Evaluation	12
2.5 Digital Twins, Sim-to-Real Transfer, and the Role of Simulation	13
2.6 Continual Learning and Damage Adaptation	13
2.7 Collective Endurance	14
Chapter 2 References	14
3 Baseline Implementations and the Latency Bottleneck	16
3.1 Synthetic Data Generation	16
3.2 Free-Form Kinematic Self-Model	19
3.3 Kinematic 3D Gaussian Splatting	20
3.4 Comparative Baseline Analysis	22
3.5 Metric Definitions and Evaluation Protocol	23
3.6 FFKSM Implementation Diagnostics	23

3.7	K-3DGS Optimization Lessons	24
3.8	Why the Baselines Remain Essential	24
	Chapter 3 References	25
4	The NeuroKin Architecture	26
4.1	Direct Sensorimotor Decoding	26
4.2	Architecture Design	27
4.3	Stereoscopic 3D Recovery	29
4.4	Performance Evaluation	33
4.5	Supervision Efficiency	35
4.6	Ablation Interpretation	36
4.7	ResNeuroKin-D and Geometric Structure	37
4.8	Deployment Envelope	37
	Chapter 4 References	38
5	Visual Closed-Loop Control and Damage Adaptation	39
5.1	Controller Integration	39
5.2	Baseline Closed-Loop Stabilization	40
5.3	Single-Agent Recovery Results	42
5.4	Implications for Self-Model Maintenance	43
5.5	Damage as State-Estimation Failure	44
5.6	Control Stability and Latency Headroom	44
5.7	From Recovery to Safe Operation	45
	Chapter 5 References	45
6	Sequential Swarm Coordination under Compounding Faults	46
6.1	The Sequential Swarm Pipeline	46
6.2	The Inter-Agent Communication Ledger	48
6.3	State-Dependent Collective Adaptation	49
6.4	Why This Counts as “Toward” Swarm Coordination	50
6.5	Formalizing Temporal Stigmergy	51
6.6	Failure Cascade and Recovery Threshold	52
6.7	Sequential Swarm as an Experimental Abstraction	52
6.8	Interpreting Success and Failure Rates	53
	Chapter 6 References	53

7	Discussion	54
7.1	Trade-offs of Direct Decoding	54
7.2	Validating the “Toward Swarm Coordination” Hypothesis	56
7.3	Methodological Lessons	57
7.4	Threats to Validity	57
7.5	Design Principles Derived from the Study	58
7.6	Ethical and Safety Considerations	59
7.7	What Would Falsify the Thesis Hypothesis	59
	Chapter 7 References	60
8	Conclusion and Future Work	61
8.1	Summary	61
8.2	Future Roadmap	62
8.3	Milestone Plan for Continuation	63
8.4	Path to Spatial Swarms	63
8.5	Final Statement	64
	Chapter 8 References	64
A	Figure Inventory and Evidence Mapping	65
B	Core Sequential Stigmergy Rule	66
C	Reproducibility and Audit Protocol	67
C.1	Source Boundary	67
C.2	Metric Audit	68
C.3	Figure Audit	68
C.4	Terminology Audit	69
D	Expanded Figure-Based Evidence Narrative	70
D.1	Data and Baseline Evidence	70
D.2	NeuroKin Evidence	71
D.3	Control Evidence	71
D.4	Sequential Swarm Evidence	72
E	Expanded Methodological Notes	73
E.1	Why Page Length Should Come from Content	73
E.2	Recommended Compilation Practice	73

E.3	Recommended Examination Framing	74
F	Claim Boundaries and Defense Notes	75
F.1	Claims Supported Directly	75
F.2	Claims Not Supported	76
F.3	Likely Examiner Questions	76
F.4	Core Defense Summary	77
F.5	Final Consistency Checklist	77
G	Integrated Literature Coverage	79
G.1	Purpose of This Integration	79
G.2	Visual and Neural Self-Modeling Context	79
G.3	Embodiment Scaling and Cross-Embodiment Learning	80
G.4	Damage Recovery, Safety, and Continual Adaptation	82
	Bibliography	84

List of Figures

3.1	PyBullet simulation setup with robotic arm, target marker, and virtual camera geometry.	17
3.2	End-to-end data generation and preprocessing pipeline.	17
3.3	Lorenz System Trajectory generating smooth, chaotic workspace exploration.	18
3.4	Representative simulated robot poses used for visual self-model training and validation.	19
3.5	FFKSM target versus render showing visual discrepancies.	20
3.6	K-3DGS iteration metrics highlighting FPS limitations.	21
3.7	K-3DGS render artifacts and spatial difference map.	21
4.1	Proposed convolutional neural network architecture for direct sensorimotor decoding.	27
4.2	Supplementary CNN architecture schematic emphasizing the convolutional feature extraction and prediction pathway used for direct sensorimotor decoding.	28
4.3	Synchronized dual-camera observations mapping to end-effector coordinates.	30
4.4	Target and current end-effector position estimated in 3D from synchronized observations.	31
4.5	Correlation of true versus predicted spatial coordinates.	32
4.6	End-effector error, target-prediction trajectory agreement, and training-validation loss for spatial recovery.	33
4.7	MSE loss converging near zero during NeuroKin training.	33
4.8	High-fidelity NeuroKin renders versus Target Images.	34
4.9	Validation true and predicted positions during NeuroKin spatial training.	35
5.1	Standard closed-loop control flow integrating the self-model.	39
5.2	Baseline error convergence over time steps.	40
5.3	End-effector trajectory during closed-loop reaching.	41
5.4	Joint error convergence during closed-loop reaching.	42

5.5	Dynamic error recovery following structural damage injected at Step 5. . .	42
6.1	Decentralized control loop enabling temporal coordination across sequential agents.	47
6.2	Algorithm efficiency in steps for the no-fault 100-stage comparison. . . .	48
6.3	Algorithm Efficiency in Steps under injected faults. The IK baseline fails structurally (capped at 201 steps), while the NeuroKin swarm reads shared states and adapts behavior, maintaining efficiency between 20-40 steps. .	50
7.1	Pareto frontier illustrating the optimal trade-off between model accuracy and computational parameter count.	55

List of Tables

3.1	Quantitative comparison of self-modeling architectures.	22
-----	---	----

Chapter 1

Introduction

1.1 Background

Autonomous robots rarely remain identical to the formal models used to design and control them. A manipulator may begin operation with calibrated joint limits, a known kinematic tree, measured link lengths, and clean actuator response curves. After extended operation, the same platform may experience backlash, encoder drift, small structural deformation, thermal expansion, surface wear, or unmodeled compliance. In more severe cases a link may bend, a joint may stick, a camera may lose calibration, or the reachable workspace may shift after impact. These changes create an accumulating mismatch between the robot’s internal representation and its embodied physical state. This thesis refers to that mismatch as *embodiment drift*.

Embodiment drift is not a cosmetic modeling error. It directly affects planning, feedback control, collision checking, and damage recovery. A controller that believes the end effector occupies one location while the physical robot occupies another can issue corrections in the wrong direction. A planner that trusts a damaged kinematic chain may attempt motions through unreachable or unsafe states. A visual self-model that updates too slowly may diagnose the change only after the control action has already caused instability. In high-degree-of-freedom systems, the effects of drift compound because many small errors can accumulate through the chain. In contact-rich manipulation, small morphological errors can produce large force and position errors. In remote or hazardous environments, manual recalibration may not be available at all.

Robot self-modeling addresses this problem by allowing an embodied system to infer and update an internal model of its own morphology from sensorimotor experience. The literature reviewed for this thesis organizes the field into three broad eras. The first era used symbolic morphology inference: the robot generated hypotheses about its body, executed exploratory actions, and selected models that best explained observed sensor data. The second era emphasized behavioral adaptation: rather than reconstructing

the damaged body, the robot searched a precomputed behavioral repertoire for actions that still worked. The third era uses visual self-modeling, including occupancy networks, Neural Radiance Fields, and 3D Gaussian Splatting, to represent geometry from images and joint configurations. Each stage improves some aspect of adaptive autonomy, but each also exposes a different bottleneck. Symbolic methods are interpretable but limited in representational flexibility. Behavioral methods recover fast in some locomotion settings but avoid explicit morphology diagnosis. Neural rendering methods are expressive but computationally expensive.

This thesis is motivated by a deployment-oriented criterion: a self-model is valuable when it improves functional robotic behavior. Rendering quality matters only insofar as it supports task execution, collision avoidance, drift correction, or post-damage recovery. The project therefore evaluates visual fidelity and latency together, then tests whether a learned self-model can be inserted into closed-loop control. The central claim is that self-modeling for control should not begin with the question “How can the robot reconstruct a complete 3D model of itself?” It should begin with the more operational question “What representation is sufficient for the controller to detect and compensate for embodiment drift fast enough to matter?”

The answer developed here is direct sensorimotor decoding. Instead of learning a volumetric field and rendering it through costly ray marching, the proposed architecture learns a direct map from motor state to visual self-observation and spatial estimates. The resulting model, NeuroKin, is not a general-purpose neural renderer. It is a control-oriented self-model designed to satisfy a strict latency budget. Its value lies precisely in this narrower design objective: it gives up full novel-view synthesis in exchange for sub-millisecond inference, dense supervision, and practical integration into a feedback loop.

1.2 The Computational Bottleneck

The most serious deployment obstacle for neural visual self-models is inference latency. Robotic control loops are temporally unforgiving. A system intended to operate near 1 kHz has a cycle time of approximately 1 ms. Within that interval, the robot must sense, estimate state, compute a corrective action, respect joint limits, and issue the next command. If self-model inference alone consumes tens or hundreds of milliseconds, the model is no longer part of the high-frequency feedback loop. It becomes an offline diagnostic or a low-rate supervisory module.

NeRF-style representations are especially vulnerable to this mismatch. A Neural Radiance Field represents geometry and appearance as a continuous function queried along camera rays. To render a 100×100 image with 64 samples per ray, the renderer performs

approximately 640,000 field evaluations per frame. The reproduced Free-Form Kinematic Self-Model baseline reaches 17.35 dB PSNR at 5.22 FPS, corresponding to about 191.6 ms latency. This is nearly three orders of magnitude slower than the sub-millisecond operating point required for a 1 kHz loop.

3D Gaussian Splatting improves rendering speed by replacing implicit ray marching with explicit primitives and rasterization. In this thesis, the K-3DGS implementation reaches 37 FPS at 17.01 dB PSNR. That is a meaningful improvement over FFKSM, but it remains far below 1,000 FPS. Moreover, the tractable implementation relies on isotropic Gaussians, which are poorly matched to long, thin robotic links. The result is a characteristic blobby artifact: thin articulated structures are approximated by overlapping spherical primitives rather than smooth cylinders or link surfaces.

The bottleneck is therefore architectural rather than merely a matter of engineering optimization. FFKSM pays for dense volumetric sampling. K-3DGS pays for maintaining and rendering explicit primitives. Both approaches maintain a 3D intermediate representation even when the downstream control task may require only silhouette-level self-awareness, end-effector localization, or anomaly detection. The project thus asks whether the representational burden can be reduced without sacrificing the signals needed for control.

The NeuroKin result demonstrates that it can. Direct decoding produces an image-like self-model in a single forward pass. It provides dense supervision over all 10,000 output pixels simultaneously. It does not traverse rays, sort large primitive sets, or optimize a scene representation during inference. The model reaches 21.88 dB PSNR at 7,400 FPS, corresponding to 0.135 ms latency. This performance is not a minor speedup; it changes the category of the model. FFKSM and K-3DGS are too slow for high-frequency closed-loop deployment, while NeuroKin is fast enough to be considered as an in-loop diagnostic and corrective component.

1.3 Problem Statement and Scope

The problem addressed by this thesis is the following: how can an embodied robot maintain a useful self-model under drift and damage while satisfying the latency requirements of real-time closed-loop control, and how can such self-diagnostic capability be extended toward collective coordination without claiming unsupported spatial swarm deployment?

The technical scope is simulation-based. The empirical pipeline uses PyBullet to generate robotic-arm observations, joint configurations, camera views, depth information, and closed-loop control traces. Synthetic trajectories are produced using smooth chaotic exploration, including Lorenz-attractor sampling, to cover a diverse set of reachable configurations without relying on abrupt random jumps. All reported quantitative results are

taken from the project sources and the metrics specified in the thesis task. The main experimental comparison includes FFKSM, K-3DGS, NeuroKin, and the depth-augmented ResNeuroKin-D variant. The control experiments integrate visual self-model estimates with an inverse-kinematics controller. The collective experiment is a 100-stage sequential pipeline under progressive fault injection.

The phrase “Toward Swarm Coordination” is deliberately interpreted with scientific restraint. The experiments do not deploy multiple physical robots simultaneously in a shared spatial environment. They do not test collision avoidance among concurrent agents, inter-robot localization, radio communication, formation control, or distributed sensing in physical space. Instead, the collective result is framed as a *sequential swarm* using *temporal stigmergy*. Each stage of the 100-stage chain is treated as an agent operating after its predecessor. The shared environment is not a spatial arena but a persistent state ledger, implemented as the `factory_state` dictionary. Its most important variable, `consecutive_failures`, records the recent failure history of the sequence. When the count reaches the threshold of two, the next stage enters `recovery` mode, selects a safer localized target, and doubles the tolerance.

This framing makes the collective claim defensible. The thesis does not argue that temporal stigmergy is equivalent to full physical swarm coordination. It argues that temporal stigmergy validates a computational prerequisite for such coordination: an individual agent must diagnose its own state fast enough for that diagnostic information to influence the next agent’s behavior. In other words, the sequential swarm demonstrates history-aware, state-dependent coordination across agents in time. Future work must still transition from temporal sequential environments to spatially concurrent multi-robot systems.

1.4 Contributions

This thesis makes four primary contributions.

First, it benchmarks representative self-modeling baselines along the latency-fidelity frontier. The reproduced FFKSM baseline reaches 17.35 dB PSNR at 5.22 FPS and exposes the control incompatibility of volumetric rendering. The K-3DGS implementation reaches 37 FPS at 17.01 dB PSNR, showing that explicit splatting is faster but still too slow for 1 kHz control and visually limited by isotropic primitives.

Second, it develops NeuroKin as a direct sensorimotor decoder for real-time self-modeling. NeuroKin maps joint configurations directly to visual self-model outputs using a lightweight neural architecture. The primary metric is 21.88 dB PSNR at 7,400 FPS with 0.135 ms latency. This result demonstrates that the apparent speed-quality trade-off is not fixed; by removing unnecessary 3D reconstruction, the architecture improves both

quality and speed relative to the reproduced baselines.

Third, it integrates the self-model into visual closed-loop control and damage adaptation. The control loop combines target error computation, Jacobian-based inverse-kinematics updates, joint limit enforcement, and self-model-based state estimation. Under damage injection, the learned visual estimate can help stabilize motion by reducing reliance on corrupted proprioceptive assumptions. The single-agent experiments therefore connect model performance to control behavior rather than treating PSNR as the final objective.

Fourth, it evaluates sequential swarm coordination under compounding faults. The 100-stage chain uses a shared `factory_state` ledger to transmit failure history across time. When `consecutive_failures` reaches two, agents switch to recovery behavior. Under progressive fault injection, the baseline inverse-kinematics process fails structurally, while the NeuroKin sequence maintains substantially higher success and lower step counts for most trials. This result supports the thesis title only in the carefully bounded sense of being *toward* swarm coordination: it demonstrates a state-sharing mechanism and a fast local diagnostic primitive, not full concurrent multi-robot deployment.

1.5 Research Questions

The thesis is organized around four research questions that connect the empirical work to the broader literature. The first question asks which self-modeling architecture class best balances latency and reconstruction quality under the computational constraints of robotic control. This question is answered by comparing FFKSM, K-3DGS, NeuroKin, and ResNeuroKin-D under the same dataset regime. The outcome is not merely a ranking of models. It identifies a structural difference between reconstruction-based and direct-decoding approaches. Reconstruction-based approaches spend computation to preserve general geometry, while direct decoding spends computation only on the control-relevant output.

The second question asks which metrics determine deployment readiness. The project measures PSNR, FPS, latency, training time, closed-loop error, and fault-stage success. This mix is important because visual fidelity alone can be misleading. A model with acceptable PSNR but 191.6ms latency cannot support a 1 kHz control loop. A model with slightly imperfect silhouettes but sub-millisecond inference can become a useful component of stabilization and anomaly detection. The thesis therefore treats PSNR as a diagnostic metric and latency as a deployment gate. Control success and recovery behavior are the functional metrics that ultimately matter.

The third question asks how robust self-modeling becomes under embodiment drift and post-damage adaptation pressure. The thesis does not claim complete hardware-

level damage recovery. Instead, it evaluates a simulated damage setting and asks whether visual self-model estimates can help the controller continue reducing error after disturbance. The damage experiment at step 5 and the 100-stage fault-injection chain provide two levels of evidence. The single-agent experiment tests in-loop recovery. The sequential experiment tests whether repeated failures can shape later behavior through a shared state ledger.

The fourth question asks how individual self-model confidence and failure signals might support collective coordination. This is where the thesis uses the phrase “toward swarm coordination.” The answer is intentionally bounded: the experiments show temporal coordination, not spatial concurrency. Nevertheless, the sequential swarm result is meaningful because it demonstrates the causal chain required for future multi-robot systems. Individual self-modeling must be fast enough to diagnose local state; local outcome must be compressed into a communication variable; and later agents must adapt based on that variable. The `factory_state` ledger implements the minimal version of this chain.

1.6 Thesis Organization

Chapter 2 synthesizes the literature on robot self-modeling. It traces the progression from symbolic morphology inference to behavioral adaptation, learned predictive models, visual self-modeling, NeRF, and 3DGS. It also introduces the latency paradox and defines temporal stigmergy as the theoretical frame for the sequential swarm.

Chapter 3 presents the baseline implementations. It describes the PyBullet data-generation pipeline, Lorenz-attractor exploration, FFKSM reproduction, and K-3DGS implementation. The chapter establishes the empirical latency bottleneck: FFKSM is too slow because of ray marching, and K-3DGS remains too slow because explicit primitive rendering still does not meet the sub-millisecond control budget.

Chapter 4 introduces NeuroKin. It explains the direct sensorimotor decoding principle, architecture, training objective, stereoscopic extension, and performance results. The central result is 21.88 dB PSNR at 7,400 FPS and 0.135 ms latency. The chapter also discusses ResNeuroKin-D as a depth-augmented variant that trades some speed for geometric structure.

Chapter 5 integrates the self-model into closed-loop control. It presents the Jacobian-based inverse-kinematics loop, end-effector error convergence, joint error convergence, spatial trajectory behavior, and damage recovery after an injected disturbance. The purpose is to move the evidence from rendering metrics to functional control behavior.

Chapter 6 presents the sequential swarm. It explains the 100-stage chain, the `factory_state` ledger, the `consecutive_failures` threshold, and the transition into recovery mode. It explicitly states what the experiment does and does not prove. The

experiment proves temporal stigmergic adaptation under progressive faults. It does not prove concurrent spatial swarm coordination.

Chapter 7 discusses trade-offs, limitations, and validation of the thesis hypothesis. It weighs the loss of novel-view synthesis against the acquisition of control-compliant latency, analyzes threats to validity, and defends the sequential swarm interpretation. Chapter 8 concludes and proposes a roadmap from simulation to hardware, from sequential temporal coordination to spatially concurrent swarms, and from simple failure counters to richer shared diagnostic state.

Chapter 1 References

This chapter draws on the project sources describing robot self-modeling motivation, FFKSM reproduction, K-3DGS, NeuroKin, and the sequential swarm task definition [1, 2, 6, 12, 13, 24, 28, 18].

Chapter 2

Literature Synthesis

2.1 Evolution of Self-Modeling

Robot self-modeling begins from a biological intuition: adaptive agents maintain some internal sense of the relation between body, action, and sensation. In animals this relation is often described as a body schema, and it changes after injury or tool use. In robotics, the analogous problem is to maintain an internal model of morphology, kinematics, and dynamics that remains useful as the body changes. The literature surveyed in the source materials traces a progression from explicit symbolic inference to behavioral adaptation and then to visual self-modeling.

The symbolic era is represented by the continuous self-modeling work of Bongard, Zykov, and Lipson [1]. Their robot generated candidate body models, executed actions that were informative under competing hypotheses, compared predicted and observed sensor outcomes, and refined its internal model. The core contribution was not merely a particular representation, but the closed loop connecting exploration, model selection, and behavior synthesis. The robot could recover after damage because it used physical interaction to update the model on which control depended. However, the symbolic representation imposed a strict ceiling. It could reason about body structures that were expressible in the chosen primitives, but not arbitrary deformations, soft-body effects, visual surface changes, or complex unmodeled damage. The space of possible bodies was constrained by the simulator’s vocabulary.

The behavioral era shifted the emphasis from explaining damage to finding behavior that remained functional. Cully and colleagues used MAP-Elites to precompute a large repertoire of candidate behaviors, then used online search to identify a compensatory behavior after damage [2, 5]. This approach is powerful because it does not require the robot to know exactly what changed. It asks instead what actions still produce acceptable task outcomes. The literature identifies this as a pragmatic form of resilience: when a leg is damaged, the system may not need a precise damaged morphology if it can rapidly find

a gait that works. Yet this avoidance strategy is less satisfying for tasks requiring precise geometry. A manipulator reaching through a narrow passage cannot simply choose any behavior that moves; it must know where its body and end effector are. For manipulation, inspection, and contact-sensitive control, the internal model itself remains important.

Learned self-models introduced a more flexible alternative. Kwiatkowski and Lipson proposed task-agnostic self-modeling in which robots learn motor-to-sensory predictive models from experience [6]. Such models can support closed-loop control after morphology change because they learn the consequences of action rather than relying only on manually specified kinematics. Proprioceptive approaches further reduce dependence on external cameras and remain robust under poor lighting or occlusion. However, proprioception alone may not capture external geometry, link deformation, or visual occupancy. The survey sources therefore treat proprioception as reliable but less expressive than multimodal self-modeling.

The visual era expands representational capacity by allowing robots to model themselves in image or geometry space. Chen and colleagues used full-body visual self-modeling with occupancy and distance-field style representations to support planning and damage adaptation [12]. Hu, Lin, and Lipson introduced the Free-Form Kinematic Self-Model, a NeRF-like occupancy model conditioned on joint angles [13]. This work is especially relevant because it reduces hand-designed morphology assumptions and learns a visual self-model from observations. Articulated 3D Gaussian Splatting further extends the visual paradigm by using explicit renderable primitives to represent articulated bodies with improved rendering speed [18].

Across these stages, one pattern is consistent: each new representation solves a limitation of the previous one while introducing a new deployment constraint. Symbolic systems are interpretable but restricted. Behavioral repertoires are fast at recovery but not diagnostic. Neural rendering is expressive but slow. The thesis contribution emerges from this historical tension. Rather than adding still more expressive rendering machinery, it asks whether an intentionally narrower representation can meet the requirements of real-time control.

2.2 The Latency Paradox

The latency paradox is the central literature-level problem motivating this thesis. Visual self-modeling methods are attractive because they can represent morphology more richly than symbolic kinematic descriptions. Yet the very mechanisms that make them expressive often make them too slow for the control regimes where adaptation matters most.

NeRF defines a continuous radiance or density field queried at spatial locations and

viewing directions [24]. For static scene reconstruction and novel-view synthesis, this formulation is elegant. It can represent smooth surfaces, complex occlusions, and view-dependent appearance. Robotics researchers adapt this family of methods because a robot’s body can be treated as a queryable spatial field conditioned on joint state. In a self-modeling context, the question becomes: given joint angles and a spatial query, is this point occupied by the robot? Rendering then converts many such queries into an image or occupancy map.

The computational problem is sampling. A single rendered image requires many rays, and each ray requires many samples. In the reproduced FFKSM setup, a 100×100 image with 64 samples per ray requires about 640,000 network evaluations. The result is 5.22 FPS on the project hardware. Even if the method could be optimized further, this order of magnitude is incompatible with sub-millisecond control. NeRF-style self-models therefore create a contradiction. They are designed to improve adaptive autonomy, but their inference latency prevents them from entering the high-frequency loops that autonomy relies on.

3D Gaussian Splatting was introduced in graphics as a faster explicit alternative to implicit neural rendering [28]. It represents scenes as sets of Gaussians that are projected and alpha-blended into the image plane. For robotics, articulated Gaussian models are attractive because they provide explicit geometric elements and faster rasterization. The survey materials note the growing role of 3DGS in robotic digital twins, manipulation, SLAM, and sim-to-real pipelines. The thesis implementation confirms the speed advantage but also its limit. K-3DGS reaches 37 FPS, far faster than FFKSM but still far below 1,000 FPS. Furthermore, the tractable isotropic implementation produces artifacts on thin links, while the attempted anisotropic version is difficult to optimize under the project dataset conditions.

This produces a representation-control mismatch. Visual fidelity and geometric expressiveness are not the same as deployability. A model can produce a plausible render while failing to support a fast enough feedback loop. Conversely, a lower-dimensional direct decoder may be less expressive in a graphics sense but more useful in a control sense because it satisfies the timing requirement. The thesis therefore treats PSNR and visual difference maps as diagnostic metrics, not final deployment criteria. The decisive metrics are latency, closed-loop stability, success under damage, and the ability to produce state information quickly enough for adaptation.

The latency paradox also reframes the role of 3D reconstruction. Full 3D reconstruction is necessary when the task requires arbitrary novel views, manipulation of unknown objects, or precise geometry under changing camera configurations. It is not necessarily necessary for detecting self-occlusion patterns, estimating end-effector location from fixed cameras, or monitoring whether a damaged body deviates from expected appear-

ance. For those tasks, the extra geometric generality may be wasteful. NeuroKin exploits this observation by making the self-model task-specific rather than renderer-general.

2.3 Theoretical Foundations of Swarm Coordination

Swarm coordination normally evokes many agents acting simultaneously in a shared environment. Such systems may coordinate through local sensing, distributed communication, pheromone-like traces, role allocation, or emergent spatial patterns. This thesis does not experimentally evaluate that setting. The available empirical data instead uses a 100-stage sequential pipeline. A scientifically defensible account must therefore avoid claiming concurrent multi-robot deployment. The theoretical bridge is temporal stigmergy.

In this thesis, *temporal stigmergy* means coordination across agents through persistent state left by previous agents in a shared process environment. The environment is not a physical field marked by simultaneous robots. It is a minimal state ledger. The relevant ledger is the `factory_state` dictionary, and the key diagnostic value is `consecutive_failures`. Each stage reads the ledger before acting. If recent predecessors have failed twice in sequence, the next stage changes behavior by entering `recovery` mode. In recovery mode the target is localized to a safer position and tolerance is expanded. The agent then writes its own success or failure back into the same ledger, influencing later agents.

This mechanism contains the essential logic of stigmergic coordination: agents do not require direct negotiation or centralized planning to adapt collectively. They respond to a shared trace of prior activity. The trace is temporal rather than spatial. It records the recent history of operational breakdown. The adaptive response is local: no agent computes a global swarm policy, but the sequence as a whole changes behavior after repeated failures. This is why the thesis describes the experiment as a sequential swarm.

The connection to real-time self-modeling is direct. A swarm can coordinate around diagnostic state only if individual agents can produce that state quickly enough. If self-diagnosis is slow, communication of diagnostic confidence becomes stale. If inference latency exceeds the control timescale, the agent cannot reliably determine whether its own body is drifting or whether a failure is caused by environment, controller, or morphology. Sub-millisecond self-modeling is therefore a computational prerequisite for future swarm endurance. It enables each robot to contribute timely confidence, fault, and recovery signals to collective decision-making.

The sequential experiment should be understood as a minimal proof of this prerequisite. It demonstrates that a history-aware state variable can trigger local behavioral adaptation across a chain of agents, and that the NeuroKin agents retain enough efficiency under progressive faults to benefit from that adaptation. It does not demonstrate

spatial formation control, simultaneous interaction, or physical robot communication. Those remain future work. The value of the present contribution is that it grounds the swarm claim in observable state-dependent behavior without overstating the experiment.

2.4 Visual Self-Modeling and Functional Evaluation

The survey sources repeatedly emphasize that a self-model should be evaluated by what it enables the robot to do. Visual reconstruction metrics such as PSNR and SSIM are useful because they reveal whether a model has learned the structure of the robot’s body. They are not sufficient because a visually plausible self-model can still be too slow, too brittle, or too poorly calibrated for control. This distinction is particularly important in robotics because the cost of error is not an ugly render; it is failed motion, unsafe contact, or inability to recover after damage.

In the visual self-modeling literature, occupancy and distance-field methods provide explicit planning value because they can be queried for collision and free-space structure. Chen and colleagues demonstrate the importance of full-body geometry for collision-aware planning and recovery [12]. FFKSM extends the idea by reducing dependence on manually defined models and learning self-appearance from visual feedback [13]. Articulated 3DGS then moves the field toward faster explicit rendering and higher-fidelity articulated representations [18]. These works establish the promise of visual self-modeling, but they also reveal deployment constraints: camera placement, calibration, occlusion, sensor noise, and inference latency.

The functional evaluation criterion can be stated as a hierarchy. At the lowest level, the self-model must reconstruct or predict observations well enough to represent the robot. At the next level, the model must operate quickly enough that its predictions are available to the controller. At the next level, the controller must use the model to improve behavior. At the highest level, the robot must remain robust under drift, damage, and task variation. Many neural rendering papers satisfy the first level but do not fully address the next three. This thesis attempts to move through the hierarchy by first measuring PSNR and FPS, then integrating NeuroKin into control, and finally testing sequential fault adaptation.

The hierarchy also explains why the thesis includes both qualitative figures and numerical tables. Difference maps reveal where models fail spatially. Training curves reveal whether learning stabilizes. FPS and latency reveal whether the model can run in the relevant control regime. Closed-loop error plots reveal whether the self-model supports behavior. The sequential swarm plot reveals whether local adaptation can propagate across agents in time. No single metric captures the complete picture.

2.5 Digital Twins, Sim-to-Real Transfer, and the Role of Simulation

The literature synthesis in the source materials treats digital twins as an important unifying concept. A robot self-model can be interpreted as a learned or reconstructed digital twin of the robot’s body. In robotics, however, a useful digital twin is not merely a visual replica. It must be compatible with planning, control, and safety constraints. A visually accurate twin that cannot be queried quickly enough is incomplete for high-frequency control. A fast but poorly calibrated twin is also incomplete. The ideal twin balances accuracy, latency, updateability, and functional integration.

Simulation plays a double role in this thesis. On one hand, PyBullet simulation is a strength because it isolates the architectural question. The project can precisely control joint states, camera views, fault injection, and ground-truth end-effector positions. This allows a clean comparison between FFKSM, K-3DGS, and NeuroKin. On the other hand, simulation is a limitation because it avoids real sensing noise, real mechanical wear, real lighting variation, and real contact dynamics. The thesis therefore treats the results as evidence for computational feasibility, not hardware readiness.

Sim-to-real transfer would require several additional layers. The visual pipeline would need to handle segmentation under non-blue backgrounds or use robust preprocessing to isolate the robot body. The camera system would require calibration and synchronization. Joint encoders would introduce noise and bias. Physical links would have compliance and manufacturing variation. Actuator commands would not execute exactly as in PyBullet. A real robot might also encounter occlusion from objects, humans, or its own body. These conditions could degrade both image prediction and coordinate recovery.

Nevertheless, solving the latency problem in simulation is not trivial. If a model cannot meet the timing budget in a clean simulator, it is unlikely to meet it on hardware under added uncertainty. The thesis therefore frames simulation as a necessary but insufficient stage. NeuroKin establishes that the architectural latency barrier can be broken. Future work must then test whether the same architecture remains accurate and robust after domain shift.

2.6 Continual Learning and Damage Adaptation

Damage adaptation raises a stability-plasticity dilemma. A robot must be plastic enough to update its self-model after damage, yet stable enough not to forget healthy components. The source materials discuss catastrophic forgetting in neural systems and mention mitigation families such as Elastic Weight Consolidation, sparse subnetworks, and mixture-of-experts routing [165, 166]. These methods are relevant because a self-model

updated only on damaged observations may distort its predictions for undamaged regions of the body.

The problem is especially difficult for monolithic representations. If all joints and links share the same dense hidden layers, a gradient update caused by a damaged joint can alter features used for healthy joints. In a robot arm, this could mean that learning about a bent link degrades predictions for the base or another segment. Modular representations, physically structured latent spaces, and auxiliary geometric supervision are promising because they can localize updates.

NeuroKin does not solve continual learning by itself. Its contribution is to make continual learning operationally more plausible. Because inference is sub-millisecond and training is fast relative to the baselines, the robot can collect discrepancy signals and targeted recovery data quickly. It can also run frequent comparisons between expected and observed self-appearance. This supports safety-gated adaptation: detect discrepancy first, enter conservative control, then decide whether to update a local module. The thesis therefore treats NeuroKin as an enabling architecture for future continual self-model maintenance rather than a complete continual-learning solution.

2.7 Collective Endurance

The literature on robot adaptation often focuses on individual resilience: can one robot continue functioning after damage? Swarm coordination shifts the question to collective endurance: can a group maintain task performance when some members degrade? The sequential swarm experiment sits between these two levels. It uses individual self-modeling to produce local success or failure, then uses a shared ledger to influence later agents.

This bridge is important because swarm systems amplify both strengths and weaknesses of individual robots. If each robot is brittle, a swarm may fail through cascading errors. If each robot can diagnose and communicate its state, the group can adapt through role reassignment, conservative mode switching, or task redistribution. The present thesis does not implement those full mechanisms. It implements the minimal trace: repeated failure causes later agents to change behavior. This minimal trace is enough to show that self-modeling is not only an individual capability. It can become a collective signal.

Chapter 2 References

This chapter synthesizes the self-modeling progression from symbolic inference, behavioral adaptation, learned self-modeling, visual occupancy, NeRF, and 3DGS systems [1, 2, 5, 6, 12, 13, 24, 28, 18]. The temporal stigmergy definition is the thesis-specific interpretation

of the 100-stage pipeline described in the project task and notebooks.

Chapter 3

Baseline Implementations and the Latency Bottleneck

3.1 Synthetic Data Generation

The empirical pipeline begins with controlled simulation. PyBullet provides the physics environment, the robot arm, joint-state access, camera rendering, and end-effector ground truth. The robot is represented as a four-actuated-joint arm with rendered views at 100×100 resolution. The training data pairs joint configurations with corresponding visual silhouettes and, for the stereoscopic and depth variants, additional camera-derived spatial information. The emphasis is not photorealism; it is controlled variation under known kinematic and visual conditions.

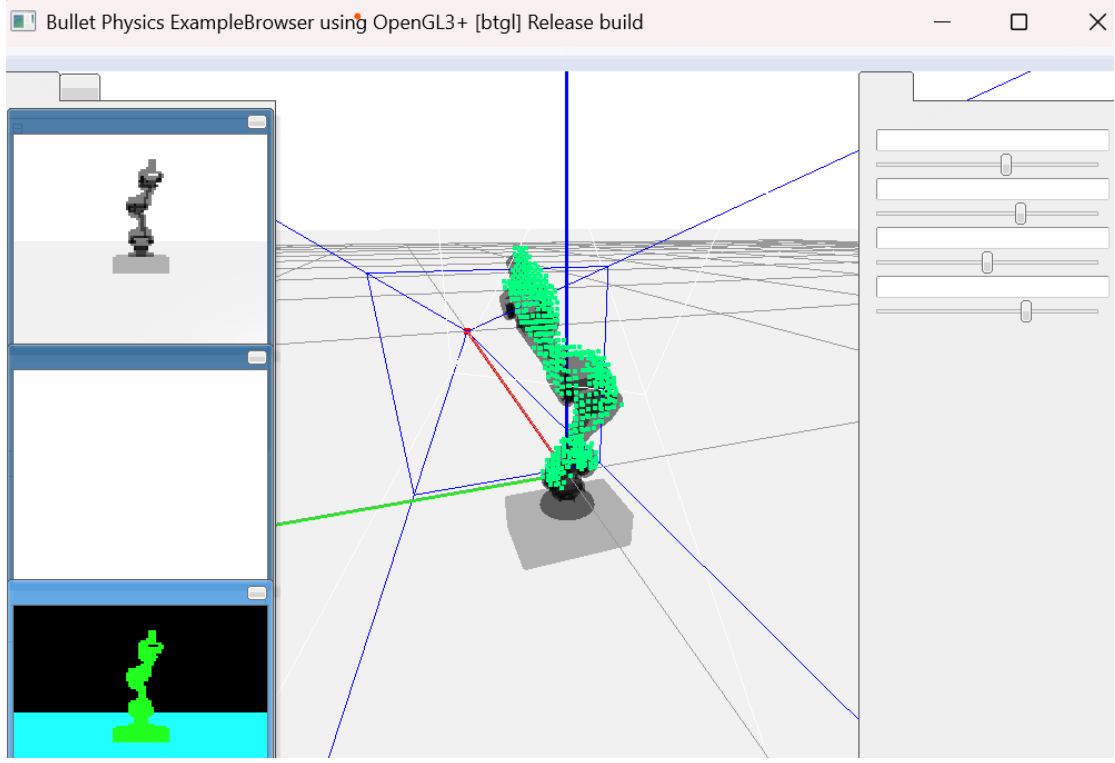


Figure 3.1: PyBullet simulation setup with robotic arm, target marker, and virtual camera geometry.

The PyBullet setup enables systematic observation of the robot under many joint configurations. The simulator records joint angles, camera images, depth buffers, and end-effector positions. This makes it possible to train both image decoders and spatial regressors without requiring unsafe real-world exploration. Since the project investigates latency and architecture rather than sim-to-real transfer, simulation is an appropriate controlled environment. However, the thesis repeatedly treats this as a limitation: hardware validation remains necessary before claiming field deployment.

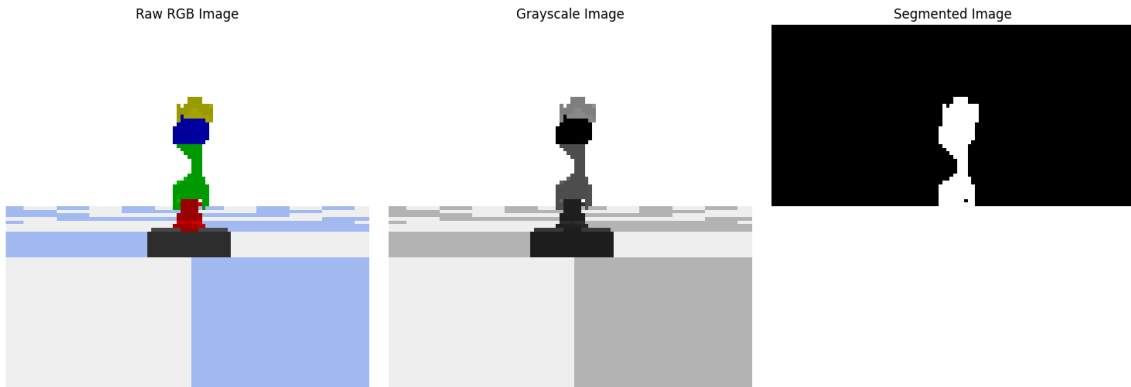


Figure 3.2: End-to-end data generation and preprocessing pipeline.

The pipeline proceeds in five stages. First, the robot is initialized in simulation with known joint limits and visual assets. Second, a trajectory generator samples configura-

tions in a smooth way rather than selecting independent random poses. Third, PyBullet renders the robot from predefined camera viewpoints and records the associated state. Fourth, images are thresholded and normalized into silhouette-like training targets. Fifth, the dataset is saved as tensors used by the baseline and NeuroKin training procedures.

Lorenz-attractor sampling is used to generate smooth, chaotic workspace exploration. This choice matters because robot data collection should not consist only of independent random configurations. A real robot moves continuously through state space, and smooth trajectories expose the model to correlated but diverse transitions. The Lorenz trajectory produces a dual-lobed path that avoids trivial repetition while preserving continuity. In the project dataset, this produces 2,000 diverse robot configurations. The preprint reports an 80/20 split, yielding 1,600 training samples and 400 testing samples for the NeuroKin training notebook.

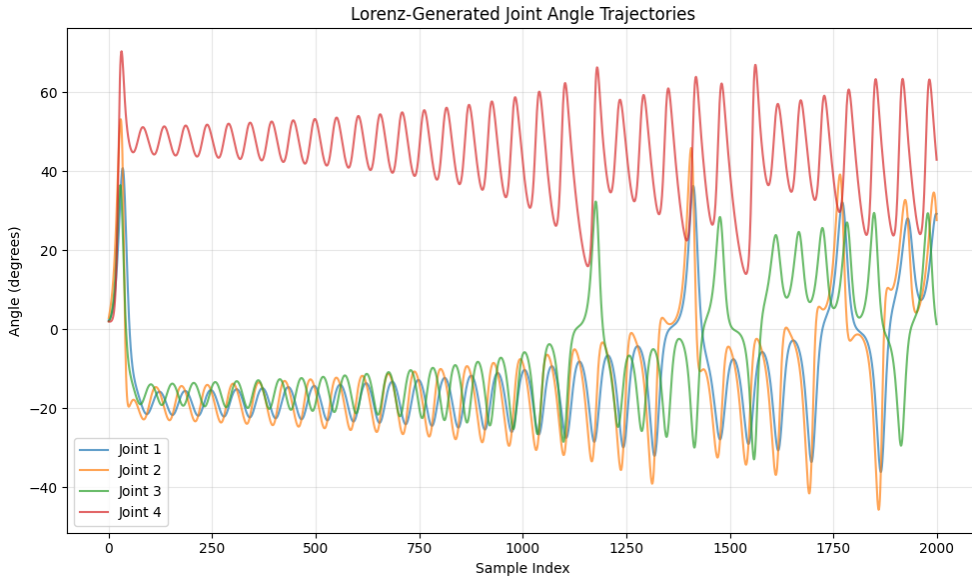


Figure 3.3: Lorenz System Trajectory generating smooth, chaotic workspace exploration.

The dataset is intentionally modest. This makes the baseline comparison stricter because dense neural rendering often benefits from larger datasets. The source materials note that the original FFKSM work used a larger scale, while the reproduced dataset contains 2,000 samples. NeuroKin’s ability to reach 21.88 dB with this data volume is therefore not simply a function of more data. It reflects a supervision advantage: direct decoding receives gradients from every output pixel during each forward pass, while ray-marched models distribute learning across sparse samples along rays.

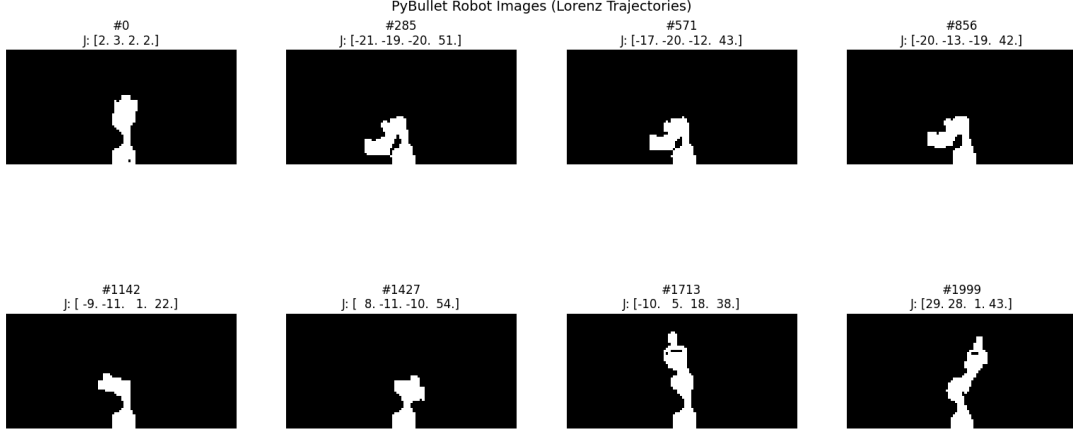


Figure 3.4: Representative simulated robot poses used for visual self-model training and validation.

The visual task is deliberately constrained. The robot appears against a simple background, and the target output is primarily a silhouette or grayscale representation. This is not a weakness for the thesis question because the aim is not general scene understanding. The aim is to test whether a robot can learn a fast, control-relevant map from motor state to expected self-observation. The background simplicity removes unrelated texture variation and isolates the morphology-latency question.

3.2 Free-Form Kinematic Self-Model

The Free-Form Kinematic Self-Model is the first reproduced baseline. It represents the robot using a NeRF-like occupancy query model conditioned on joint angles. Given a joint configuration and spatial query point, the network predicts occupancy or density. Rendering then samples along rays to create a view of the robot. This approach is flexible because it does not require a fixed mesh or manually specified link geometry. It is also computationally expensive because rendering requires many network evaluations.

The thesis reproduction reports 17.35 dB PSNR at 5.22 FPS. At a high level, the model works: it learns approximate visual structure and produces self-model renders that resemble the target images. However, the rendered images show blurring, edge uncertainty, and difference-map errors concentrated around joint boundaries and link extremities. These errors are consistent with limited sampling density and sparse supervision. More importantly, the latency is incompatible with real-time control. At 5.22 FPS, each frame takes about 191.6 ms, far slower than a 1 ms control cycle.

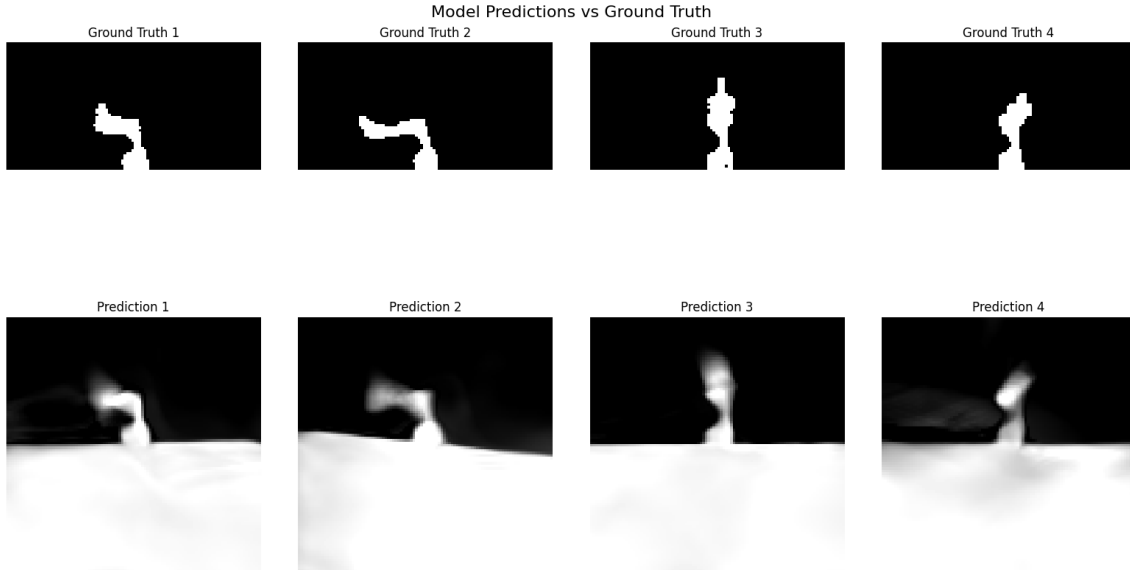


Figure 3.5: FFKSM target versus render showing visual discrepancies.

The computational reason is clear. For a 100×100 image with 64 samples per ray, the renderer evaluates the network around 640,000 times per frame. Each evaluation contributes only a small part of the final image. Even with batching, GPU acceleration, and efficient implementation, the ray-marching structure creates a latency floor. Since the model must integrate information along rays, it cannot reduce the task to one simple forward pass.

The source materials also document practical implementation challenges. FFKSM can suffer from black-screen convergence and kinematic blindness if the training signal does not adequately connect joint configurations to visual differences. Weighted losses and curriculum choices are used to address class imbalance because most pixels are background. These engineering details matter because they show that the problem is not only slow inference. Volumetric self-modeling also requires careful training design to avoid degenerate solutions under limited data.

From the standpoint of this thesis, FFKSM establishes the low-speed, moderate-quality point on the Pareto frontier. It validates the appeal of visual self-modeling but exposes the fatal mismatch with 1 kHz control. It is therefore a useful baseline precisely because it demonstrates why a more direct architecture is needed.

3.3 Kinematic 3D Gaussian Splatting

The second baseline is Kinematic 3D Gaussian Splatting. It replaces volumetric ray marching with explicit Gaussian primitives attached to a differentiable kinematic chain. In principle, this should improve speed because the renderer projects primitives into the image plane rather than sampling empty space along rays. In the thesis implementation,

K-3DGS reaches 37 FPS at 17.01 dB PSNR. This is a $7.1\times$ speedup over FFKSM but still far from the 1,000 FPS threshold required for 1 kHz control.

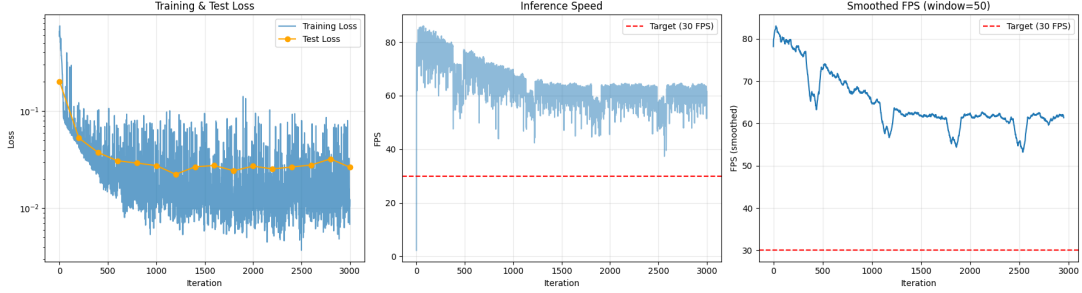


Figure 3.6: K-3DGS iteration metrics highlighting FPS limitations.

The K-3DGS training curves show rapid loss reduction and fluctuating training speed. The result confirms that splatting is a more efficient rendering family than NeRF for this task. However, the frame rate remains in the tens rather than thousands. This matters because real-time robotics is categorical: 37 FPS may appear fast in graphics or visualization contexts, but it is still about 27 ms per frame. A 1 kHz loop cannot wait 27 cycles for a self-model estimate.

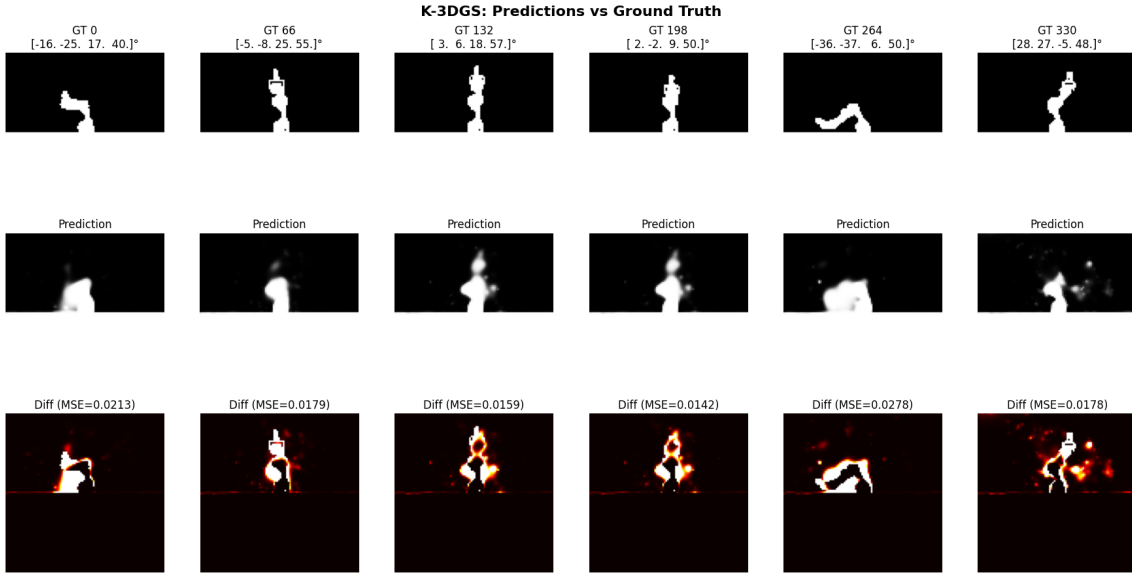


Figure 3.7: K-3DGS render artifacts and spatial difference map.

The project attempted anisotropic Gaussian primitives because real robot links are not spheres. Long cylindrical links require elongated covariance structures. In practice, the anisotropic formulation was difficult to optimize under the limited dataset and chosen parameterization. The source materials describe mode collapse and unstable optimization on the non-convex rotation manifold. The tractable implementation therefore retreated to isotropic Gaussians, each with scalar scale and opacity. This made training reliable

but reduced expressiveness. The render artifacts confirm the trade-off: thin links become chains of overlapping blobs.

K-3DGS is thus informative in two ways. It shows that explicit representations can reduce latency relative to implicit fields, but it also shows that retaining a 3D renderable primitive set is still too expensive for the strict control setting. It also demonstrates that theoretical representational expressiveness is not the same as trainability. An anisotropic representation may be geometrically appropriate, but if it collapses during optimization, it cannot serve as a reliable self-model.

3.4 Comparative Baseline Analysis

Table 3.1 summarizes the main baseline and NeuroKin comparison. The important observation is that NeuroKin does not merely trade quality for speed. It improves both relative to FFKSM and K-3DGS.

Table 3.1: Quantitative comparison of self-modeling architectures.

Method	PSNR (dB)	FPS	Latency (ms)	Training Time
FFKSM	17.35	5.22	191.6	50.0 min
K-3DGS	17.01	37	27.0	50.5 sec
NeuroKin	21.88	7,400	0.135	33.5 sec
ResNeuroKin-D	21.23	2,500	0.400	365.5 sec

The table reveals the fundamental argument of the thesis. Neural rendering families preserve more general geometric structure, but their inference pathways do not match the time budget of control. K-3DGS reduces the cost but does not eliminate the representational overhead. NeuroKin discards the intermediate scene representation and therefore changes the computational structure of the problem.

The baseline chapter also clarifies why PSNR alone cannot determine deployment readiness. FFKSM and K-3DGS produce visually meaningful outputs, but they cannot operate within the relevant latency window. NeuroKin achieves higher PSNR, but its real contribution is not the additional decibels. The real contribution is that the model can produce a self-estimate in 0.135 ms. That timing makes visual self-modeling compatible with in-loop correction, drift monitoring, and sequential coordination.

3.5 Metric Definitions and Evaluation Protocol

The project uses several metrics, each serving a different role. PSNR measures visual reconstruction similarity between predicted and target images. It is computed from mean squared error and expressed in decibels. Higher PSNR indicates better image-level agreement, but not necessarily better control. FPS measures throughput in frames or samples per second. Latency is the reciprocal timing quantity, reported in milliseconds per prediction. Training time measures how long the model takes to reach its trained state under the reported setup. In the control experiments, Euclidean end-effector distance measures functional reaching error. In the sequential swarm experiments, success rate, failures, and steps to convergence measure operational robustness.

The evaluation protocol is comparative rather than absolute. FFKSM, K-3DGS, and NeuroKin are all tested on the same general robot self-modeling problem and the same simulation regime. This matters because neural rendering papers often report metrics under different datasets, resolutions, hardware, and training procedures. The thesis does not attempt to claim universal superiority over every possible NeRF or 3DGS implementation. It claims that within the project reproduction and dataset, direct decoding produces a more deployable operating point.

Latency is the most important gate. A model below 1,000 FPS cannot satisfy a 1 kHz inference budget, even if it has acceptable visual quality. FFKSM and K-3DGS both fall below this gate. NeuroKin and ResNeuroKin-D exceed it. NeuroKin’s 7,400 FPS is especially important because it leaves time in the control cycle for other computations. A self-model that consumes the entire millisecond budget would still be difficult to deploy. A 0.135 ms model leaves headroom for image preprocessing, error computation, inverse kinematics, safety checks, and actuation commands.

3.6 FFKSM Implementation Diagnostics

The FFKSM reproduction is valuable because it exposes practical issues that are often hidden by high-level method descriptions. Volumetric self-modeling must learn where the robot is in image space while most pixels are background. If the loss is dominated by background pixels, the network can converge to a trivial or nearly empty render. The source materials describe black-screen convergence as an implementation challenge. Weighted loss and curriculum learning are used to prevent the model from ignoring foreground structure.

Another issue is kinematic blindness. A model can learn an average-looking robot silhouette without properly conditioning on joint angles. This produces visually plausible but control-useless outputs: the robot appears in roughly the right region but does not

move correctly with the commanded configuration. Kinematic blindness is especially dangerous for self-modeling because a controller needs the conditional relationship between joint state and body geometry, not a static mean image. The FFKSM implementation therefore has to ensure that joint information influences occupancy prediction.

These diagnostics strengthen the case for direct decoding. NeuroKin does not remove the need for careful training, but its dense image-level objective makes the conditional mapping more direct. Each training example asks the network to transform a joint vector into a complete image. The loss applies to all pixels of that image at once. This encourages the model to use joint information to explain global silhouette variation, rather than learning a sparse field through many local queries.

3.7 K-3DGS Optimization Lessons

K-3DGS was motivated by the speed of Gaussian splatting but constrained by trainability. Anisotropic Gaussians would better match robot links because links are elongated and oriented. However, learning anisotropic covariance and rotation parameters introduces a difficult optimization problem. The source materials report attempts using multiple learning rates, rotation parameterizations, initialization strategies, and regularization terms. The anisotropic version remained unstable and tended toward mode collapse.

The isotropic version is therefore a pragmatic compromise. It converges reliably and improves speed, but sacrifices shape fidelity. This result carries a broader lesson for adaptive robotics: the most expressive representation is not always the best representation. A damaged robot must train, update, and infer under resource constraints. A representation that is theoretically ideal but brittle during optimization may be less useful than a simpler representation that reliably converges.

The K-3DGS result also clarifies why explicit 3D structure is not automatically control-ready. Even if a representation is rasterized rather than ray-marched, it still requires maintaining primitives, projecting them, sorting or blending them, and dealing with primitive-level artifacts. For high-frequency self-modeling, those costs may remain too high. The direct decoder removes the primitive layer entirely.

3.8 Why the Baselines Remain Essential

Although NeuroKin is the primary contribution, the baselines are essential to the thesis. Without FFKSM, the work would not demonstrate the cost of implicit neural self-modeling. Without K-3DGS, it would not show that simply moving to explicit rasterization is insufficient for the strict timing target. The baseline progression creates a controlled argument:

1. NeRF-style self-modeling is expressive but too slow.
2. Gaussian splatting is faster but still too slow for 1 kHz control in the project implementation.
3. Direct decoding bypasses the shared reconstruction overhead and reaches the required latency.

This progression is stronger than comparing NeuroKin only to one baseline. It shows that the bottleneck persists across both implicit and explicit neural rendering families and that the solution is a change in problem formulation.

Chapter 3 References

This chapter is grounded in the project notebooks and the reproduced baseline descriptions in the preprint and preparation report. It uses the FFKSM and K-3DGS metrics, training observations, and data-generation workflow reported in those sources [13, 24, 28, 18].

Chapter 4

The NeuroKin Architecture

4.1 Direct Sensorimotor Decoding

NeuroKin is designed around a simple architectural decision: bypass explicit 3D reconstruction when the downstream control task does not require it. Instead of representing a robot as a field or a cloud of primitives, NeuroKin directly maps motor state to expected visual output. The input is a vector of joint angles. The output is a 100×100 visual self-model, and in the 3D variant the model also supports end-effector coordinate prediction through synchronized observations.

This direct formulation changes the computational character of the problem. In FFKSM, one rendered image requires hundreds of thousands of point evaluations. In K-3DGS, the image is assembled from many rendered primitives. In NeuroKin, the entire image is produced by a single network evaluation. Every output pixel contributes to the loss during training, and every pixel is predicted simultaneously during inference. This dense supervision explains why NeuroKin can train quickly despite having more parameters than FFKSM.

The method can be written abstractly as:

$$\hat{I}_t = f_\theta(q_t),$$

where q_t is the joint configuration at time t , f_θ is the learned decoder, and $\hat{I}_t$ is the predicted self-observation. For the spatial recovery setting, synchronized camera observations and learned predictions are used to estimate:

$$\hat{p}_t = g_\phi(I_t^{(1)}, I_t^{(2)}),$$

where $\hat{p}_t \in \mathbb{R}^3$ is the estimated end-effector position and $I_t^{(1)}, I_t^{(2)}$ are the front and side observations. The precise implementation uses the project neural architecture and

preprocessing pipeline, but the conceptual shift is captured by the equations: the model is a direct sensorimotor decoder, not a renderer of a latent 3D world.

This design does sacrifice some capabilities. NeuroKin does not automatically render arbitrary novel views in the way a trained NeRF-style field can. A fixed-view decoder must be retrained, extended with camera conditioning, or duplicated across views to support new camera configurations. The thesis treats this as an explicit trade-off rather than hiding it. The value of NeuroKin is that many control tasks do not need unrestricted novel-view synthesis. They need a fast discrepancy signal, an estimate of self-appearance, or a spatial coordinate recovered from known cameras.

4.2 Architecture Design

The architecture follows an encoder-decoder logic. Joint angles are first transformed through fully connected layers into a latent representation. The latent vector is reshaped into a low-resolution spatial feature map. Transposed convolution stages then progressively upsample the representation until it reaches the target image resolution. A final convolution produces the grayscale or silhouette output. The preparation report describes the architecture as a four-stage processing pipeline: joint positions to encoder, encoder to compact latent representation, latent representation to spatial feature map, and feature map to full-resolution output.

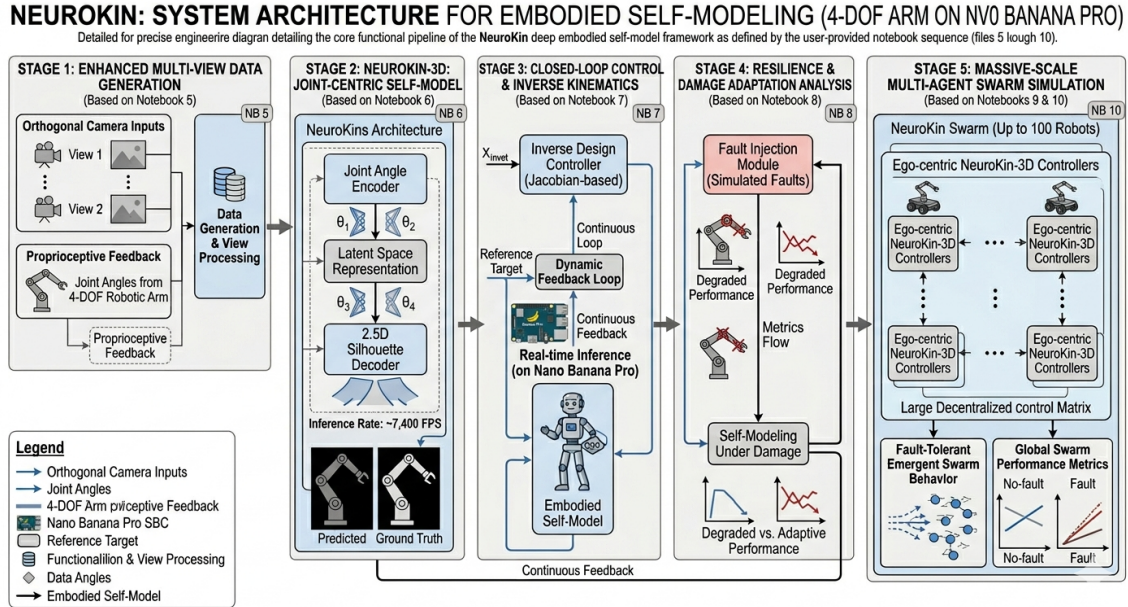


Figure 4.1: Proposed convolutional neural network architecture for direct sensorimotor decoding.

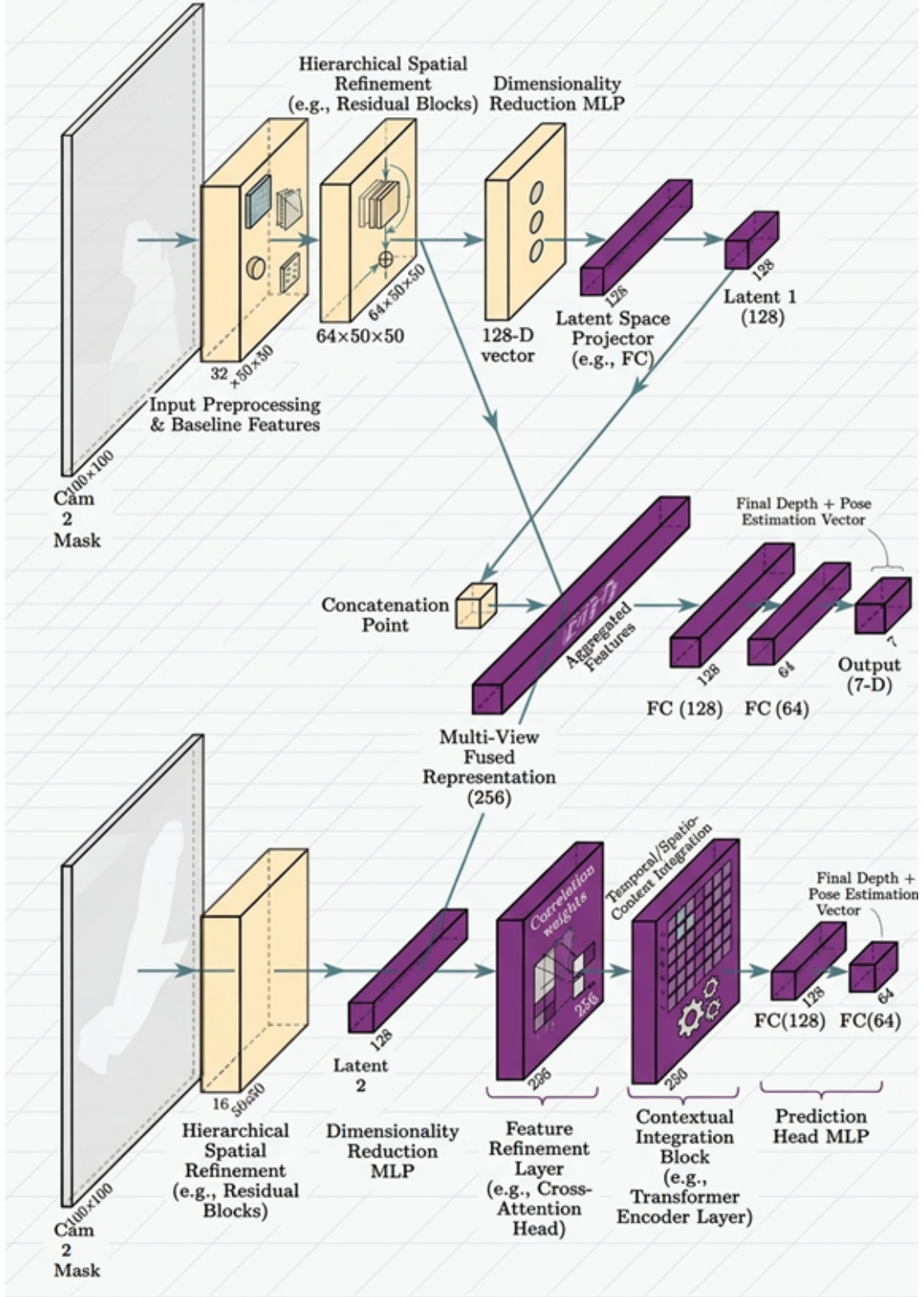


Figure 4.2: Supplementary CNN architecture schematic emphasizing the convolutional feature extraction and prediction pathway used for direct sensorimotor decoding.

The complete NeuroKin architecture contains 6,829,089 parameters. This is approximately $73\times$ the parameter count of the reproduced FFKSM model, yet it runs dramatically faster because the parameter count is not the dominant cost. FFKSM spends

computation on repeated ray samples. NeuroKin spends computation on one dense forward pass. This distinction is important because robotics papers sometimes treat smaller networks as automatically faster. In this case, the larger model is faster because its computation is parallel and direct.

Training uses mean squared error loss between predicted and target silhouettes. Gaussian noise augmentation with standard deviation 0.01 radians is applied to joint inputs to regularize the learned manifold. This encourages smoothness under small encoder perturbations, which is important for real robots where joint measurements are noisy. The model is trained for fifty epochs with batch size 128, Adam optimization, and learning rate 10^{-3} . Unlike FFKSM, it does not require curriculum learning or weighted foreground-background loss. Direct image prediction naturally supplies dense gradients across all pixels.

The architecture is also compatible with auxiliary supervision. The ResNeuroKin-D variant adds a depth prediction head trained with synthetic depth maps from PyBullet. Its combined loss weights silhouette and depth terms:

$$\mathcal{L} = \mathcal{L}_{\text{silhouette}} + 0.5 \mathcal{L}_{\text{depth}}.$$

This auxiliary task encourages the latent representation to encode geometric structure rather than only visual texture. ResNeuroKin-D reaches 21.23 dB PSNR at 2,500 FPS, slower than NeuroKin but still much faster than neural rendering baselines. It therefore occupies a middle position: less extreme latency, more structured geometry.

4.3 Stereoscopic 3D Recovery

A single silhouette compresses 3D structure into a 2D view. This is a limitation for spatial control because end-effector position is a 3D quantity. NeuroKin addresses this through stereoscopic decoding using synchronized dual-camera observations. The front camera and side camera observe the same robot configuration from complementary viewpoints. Their combined output preserves enough parallax to recover spatial coordinates without running a full volumetric renderer.

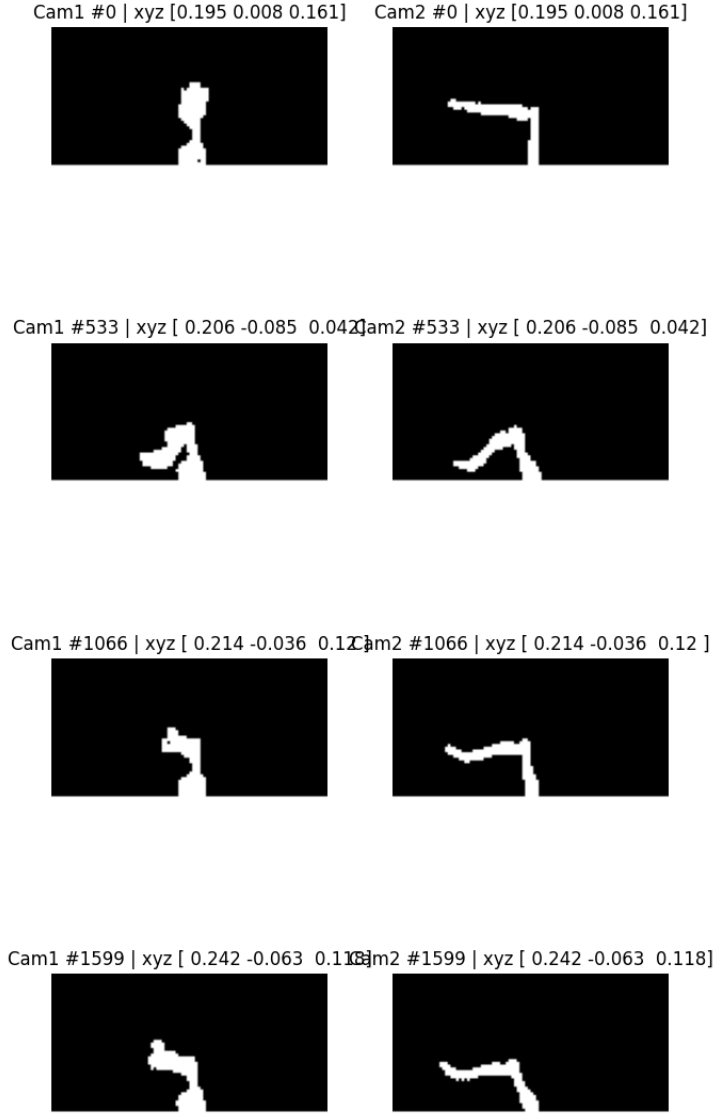


Figure 4.3: Synchronized dual-camera observations mapping to end-effector coordinates.

The dual-view pipeline is a practical compromise. It does not claim to reconstruct a complete watertight model of the robot. It recovers control-relevant 3D information from known camera geometry and fast view-specific predictions. The preparation report states that using two NeuroKin instances adds approximately 0.27 ms total latency, which remains sub-millisecond. This is still far below the latency of FFKSM and K-3DGS.

End-effector XYZ cloud (Train + Test)

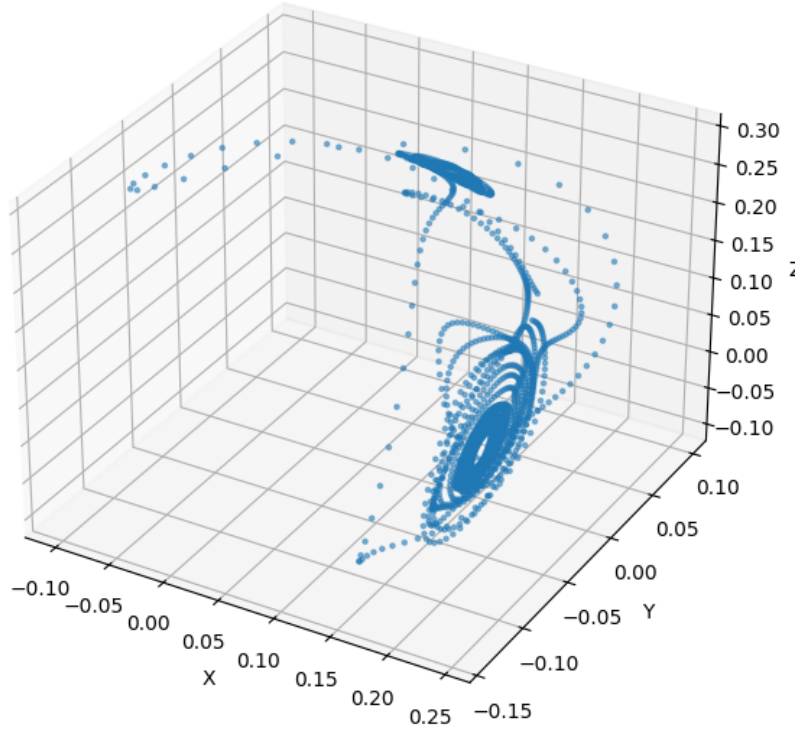


Figure 4.4: Target and current end-effector position estimated in 3D from synchronized observations.

The spatial prediction results show close correspondence between true and predicted coordinates. The validation plot presents true and predicted X , Y , and Z curves across sample index, with dashed predicted lines closely following solid ground-truth lines. This supports the use of NeuroKin-derived estimates inside a control loop. The model is not merely producing visually plausible images; it supports spatial variables needed for reaching.

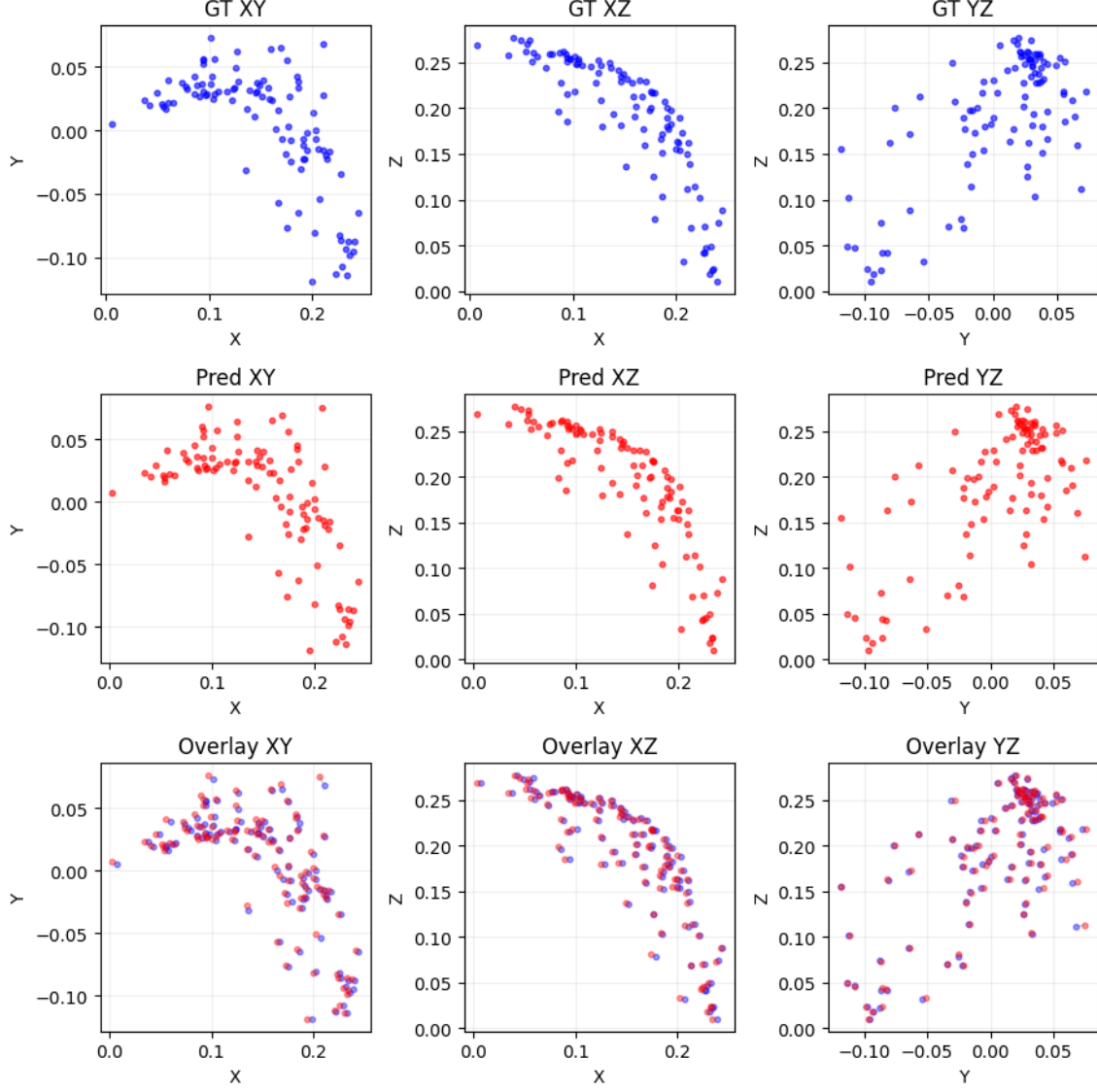


Figure 4.5: Correlation of true versus predicted spatial coordinates.

The richer diagnostic plot in Figure 4.6 further connects spatial accuracy, trajectory agreement, and training convergence. End-effector Euclidean error stays generally below the small-error region shown in the source figure, predicted and target trajectories are visually close, and training and validation loss decrease sharply toward near-zero values. Together, these plots justify using the learned estimate as a feedback signal.

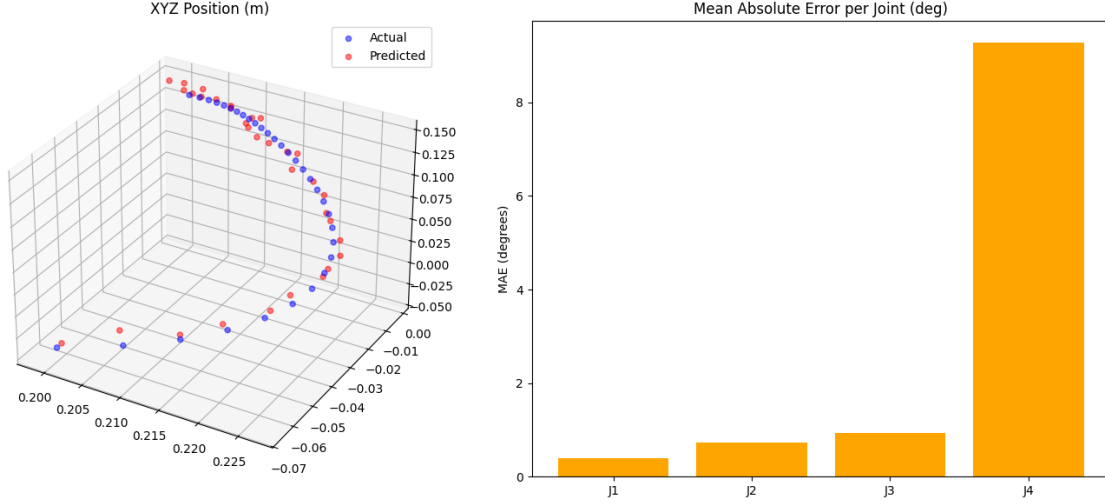


Figure 4.6: End-effector error, target-prediction trajectory agreement, and training-validation loss for spatial recovery.

4.4 Performance Evaluation

The primary NeuroKin result is 21.88 dB PSNR at 7,400 FPS, corresponding to 0.135 ms latency. Training completes in 33.5 seconds on the reported Tesla T4 setup. The result represents a $1,418\times$ speedup over FFKSM and a 4.53 dB improvement in PSNR. This combination is the central empirical contribution of the thesis.

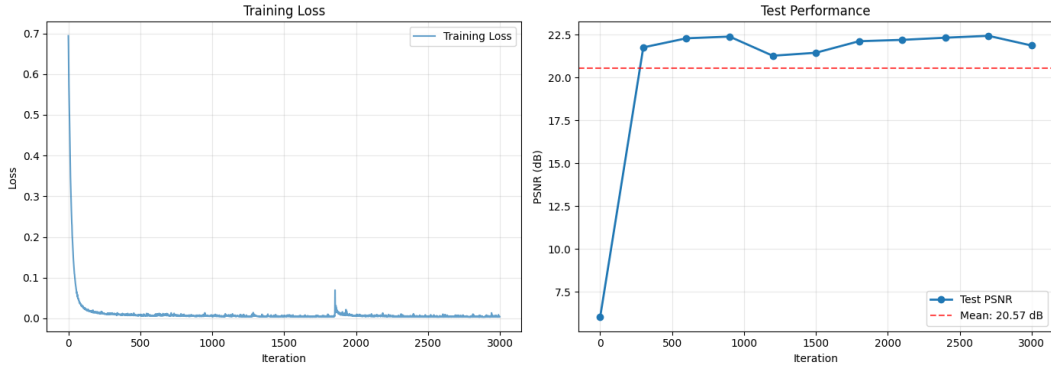


Figure 4.7: MSE loss converging near zero during NeuroKin training.

The training curve drops sharply and stabilizes near zero, with validation loss closely tracking training loss. This indicates that the model is not merely memorizing the training set in a degenerate way. It learns a smooth mapping from joint configurations to visual outputs. Noise augmentation further supports this smoothness by exposing the model to small perturbations around training poses.

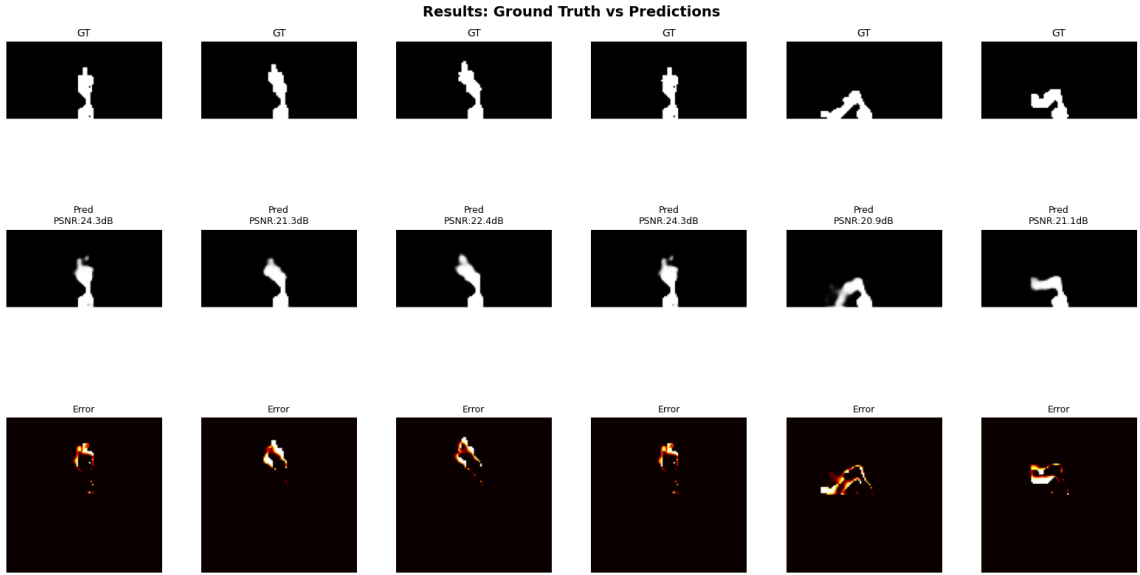


Figure 4.8: High-fidelity NeuroKin renders versus Target Images.

Qualitatively, NeuroKin renders sharp link boundaries and localizes most residual errors to difficult regions such as end-effector tips and joint extremities. Difference maps show remaining mismatch, but the artifacts are more structured and localized than the blur of FFKSM or the blob artifacts of K-3DGS. This qualitative result matters because control errors often arise at link boundaries and end-effectors. A self-model that keeps these structures crisp is more useful than one that produces visually smooth but spatially ambiguous predictions.

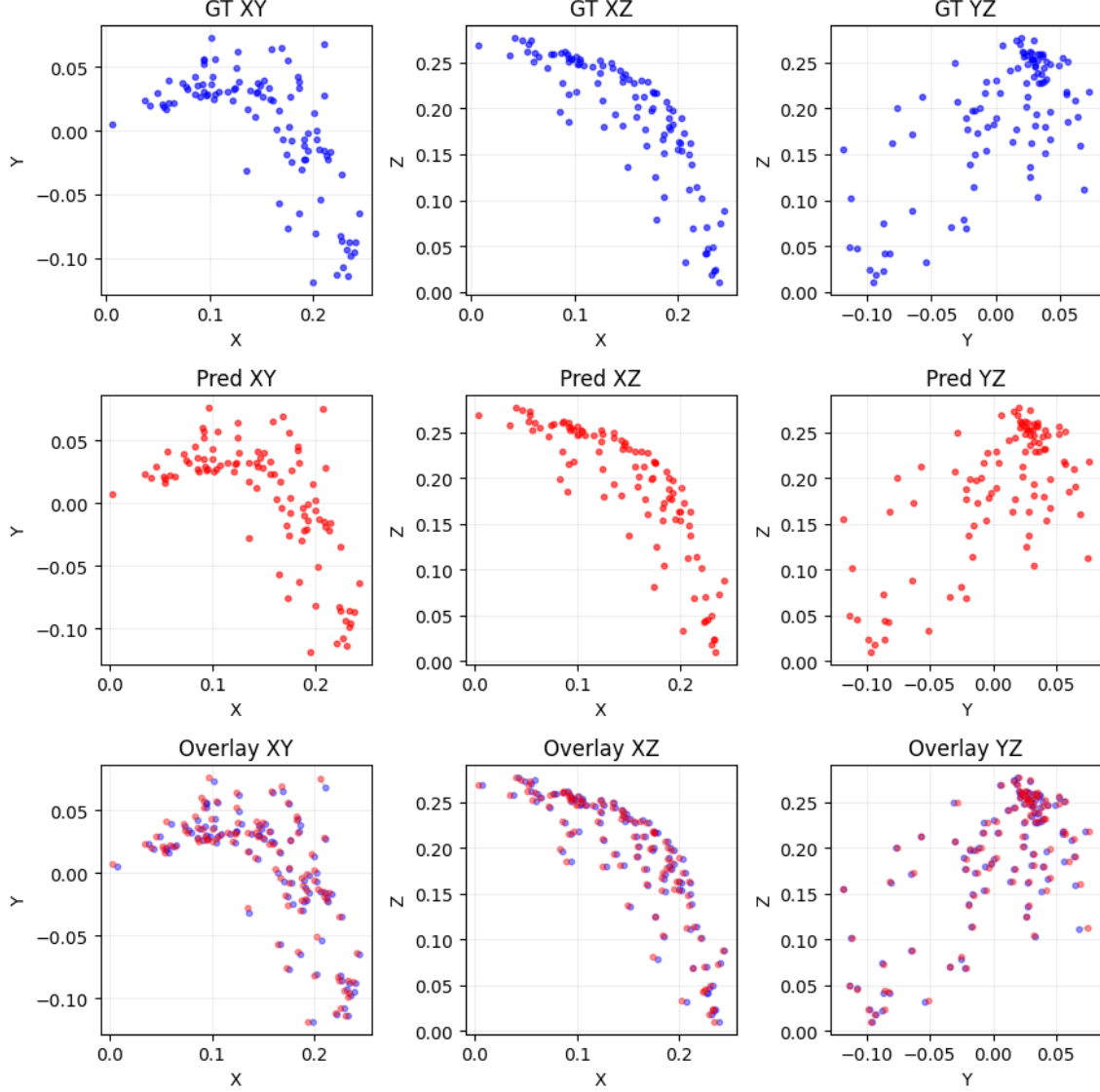


Figure 4.9: Validation true and predicted positions during NeuroKin spatial training.

The performance comparison also supports a broader methodological principle: model design should be matched to the downstream task. If the task requires open-ended 3D scene reconstruction, NeuroKin is insufficient. If the task requires fast self-appearance prediction, end-effector tracking from known cameras, or anomaly detection under fixed viewpoints, NeuroKin is better matched than general neural rendering. This is not a rejection of NeRF or 3DGS. It is a statement about control-oriented minimal representation.

4.5 Supervision Efficiency

The strongest explanation for NeuroKin’s performance is supervision efficiency. In FFKSM, a single rendered image is assembled from many point evaluations. Each evalua-

tion contributes a tiny fraction of the image-level learning signal. In the project analysis, FFKSM requires approximately 640,000 evaluations for a 100×100 image, while NeuroKin uses one forward pass to predict all 10,000 pixels. This creates a dramatic difference in how gradient information flows through the model.

The source materials summarize this difference by comparing pixels affected per evaluation. FFKSM’s ray-marched evaluation contributes on average only 0.0156 pixels per evaluation, while NeuroKin’s single forward pass contributes all 10,000 pixels. This does not mean that every pixel is equally informative, but it means the training objective is dense and direct. Dense supervision explains why NeuroKin can train in 33.5 seconds despite a larger parameter count. The model sees the full consequence of each joint configuration on each training step.

This perspective also clarifies why K-3DGS sits in the middle. Each Gaussian primitive can affect multiple pixels, so it is more efficient than point-by-point ray marching. However, the model still reasons through a set of primitives rather than directly predicting the final image. The primitive set is an intermediate representation. NeuroKin removes that intermediate representation.

4.6 Ablation Interpretation

The preprint reports several NeuroKin ablation observations. Noise augmentation improves robustness to joint encoder perturbations. Without noise augmentation, PSNR decreases and predictions become more sensitive to small input changes. This is important because physical encoders are never perfectly clean. A model that flickers visually under tiny joint noise would be unsuitable for stable control. Adding Gaussian noise with $\sigma = 0.01$ radians encourages a smoother mapping.

Network depth also affects the speed-quality balance. A shallower decoder can run faster but struggles with fine link details. A deeper decoder can improve PSNR slightly but costs additional computation and may yield diminishing returns under the 2,000-sample dataset. The selected four-stage decoder represents a practical operating point: enough capacity to represent articulated silhouettes, but still fast enough for sub-millisecond inference.

Latent dimensionality provides another trade-off. Too small a latent vector lacks capacity to encode the joint manifold and associated visual variations. Too large a latent vector increases parameters and overfitting risk without proportionate quality gain. The 1024-dimensional latent representation used in the project is therefore not arbitrary. It is a capacity-efficiency choice matched to the four-joint robot and the 100×100 output.

These ablations support an engineering lesson: direct decoding is not automatically optimal simply because it is direct. It must still be tuned to the morphology, output

resolution, data volume, and noise level. The thesis contribution is the formulation and demonstrated operating point, not the claim that any direct decoder will always outperform any renderer.

4.7 ResNeuroKin-D and Geometric Structure

ResNeuroKin-D is included in the source materials as a multi-task extension. Its purpose is to address a weakness of the base NeuroKin architecture: the latent vector may lack explicit geometric organization. A latent dimension might encode a mixture of joint pose, link overlap, and image-space features rather than a meaningful physical factor. This is acceptable for fast silhouette rendering, but less ideal for damage localization or transfer to new morphologies.

The auxiliary depth head forces the model to predict both silhouette and synthetic depth. Since depth depends on spatial ordering, the network must encode more geometry in its latent representation. The result is 21.23 dB PSNR at 2,500 FPS. This is slower than base NeuroKin but still far faster than FFKSM and K-3DGS. The architecture therefore shows how direct decoding can be extended without returning to full volumetric reconstruction.

The multi-task idea is important for future work. A real robot may need multiple self-model outputs: silhouette, end-effector position, joint-confidence estimates, damage anomaly score, and depth. Rather than building a complete renderer for all tasks, the architecture can share an encoder and attach task-specific heads. This preserves most of the latency advantage while enriching the diagnostic state.

4.8 Deployment Envelope

The deployment envelope of NeuroKin is defined by known viewpoints, known robot class, and the need for fast control-relevant self-observation. Within this envelope, NeuroKin is highly attractive. Outside it, caution is required. A moving camera, an unknown robot morphology, or a cluttered environment could break assumptions. A robot interacting with unknown objects may need explicit geometry to reason about contact and free space. A robot that must share visual self-models across heterogeneous platforms may require graph-structured or morphology-conditioned encoders.

The thesis therefore recommends a layered deployment strategy. Use direct decoding for high-frequency inner-loop self-estimation. Use slower geometric models for occasional calibration, mapping, or off-line validation. Use auxiliary heads when additional structure is needed. This hybrid perspective avoids a false opposition between NeuroKin and neural rendering. They operate at different layers of the autonomy stack.

Chapter 4 References

This chapter uses the architecture, training, stereoscopic decoding, and performance metrics reported in the project preprint, preparation report, and notebooks. It contrasts NeuroKin with NeRF and 3DGS baselines [24, 13, 28, 18].

Chapter 5

Visual Closed-Loop Control and Damage Adaptation

5.1 Controller Integration

A self-model becomes operationally meaningful when it enters a control loop. The closed-loop experiments integrate NeuroKin-derived estimates with a Jacobian-based inverse-kinematics controller. The controller receives a target position, estimates the current end-effector position, computes an error vector, maps the error into joint updates through a pseudo-inverse Jacobian, applies joint limits, and executes the next command. The process repeats until the distance to the target falls below a threshold or the maximum iteration count is reached.

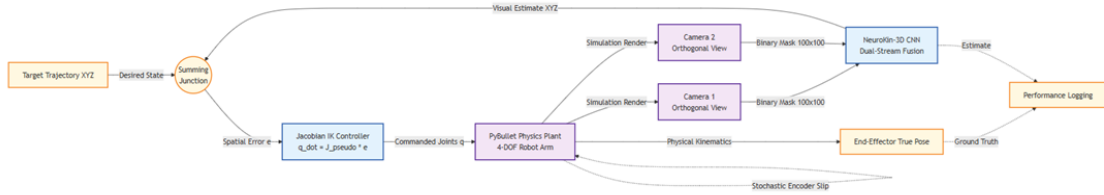


Figure 5.1: Standard closed-loop control flow integrating the self-model.

The key distinction from a purely kinematic controller is the source of the current state estimate. A conventional inverse-kinematics loop trusts the internal kinematic model and proprioceptive joint state. Under embodiment drift or damage, this assumption can fail. The learned visual self-model provides an independent estimate derived from observed appearance. When proprioception is biased or the physical body deviates from the nominal model, visual feedback can correct the state used for control.

The controller update can be written as:

$$e_t = p_{\text{target}} - \hat{p}_t,$$

$$\Delta q_t = J^+(q_t)e_t,$$

$$q_{t+1} = \text{clip}(q_t + \alpha \Delta q_t, q_{\min}, q_{\max}),$$

where $\hat{p}_t$ is the self-model estimate of the end-effector position, J^+ is the pseudo-inverse Jacobian, α is a step-size factor, and the clip operation enforces joint limits. In the notebooks, the implementation uses PyBullet inverse kinematics and bounded incremental updates. The mathematical form captures the same control logic: estimate, compare, correct, and repeat.

5.2 Baseline Closed-Loop Stabilization

The baseline closed-loop reaching experiment demonstrates that the controller can reduce end-effector error over time. Figure 5.2 shows the Euclidean distance decreasing from an initial nonzero value toward zero across time steps. This establishes that the control loop is functional in the nominal setting.

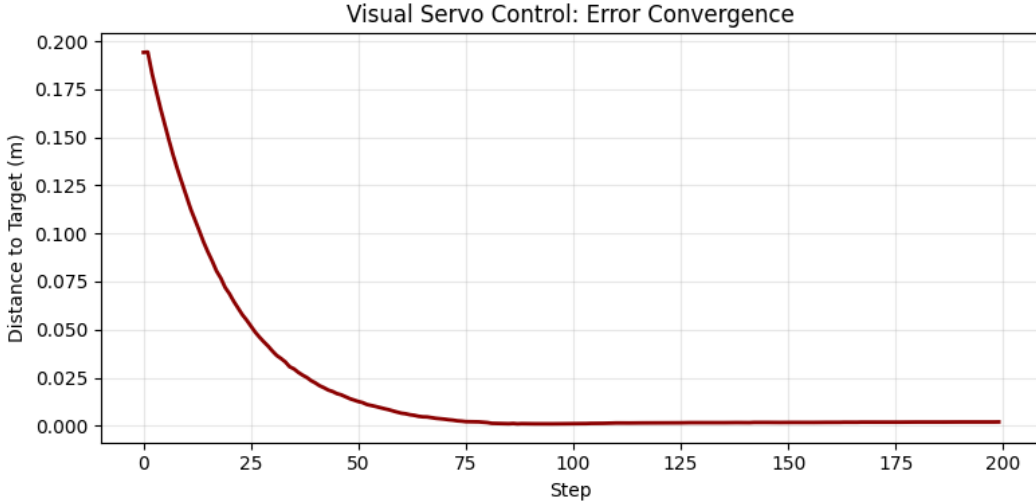


Figure 5.2: Baseline error convergence over time steps.

The end-effector trajectory plot provides a spatial interpretation of the same behavior. The trajectory curves from the start position toward the target, demonstrating that the controller is not simply reducing an abstract loss but moving the simulated robot through 3D space.

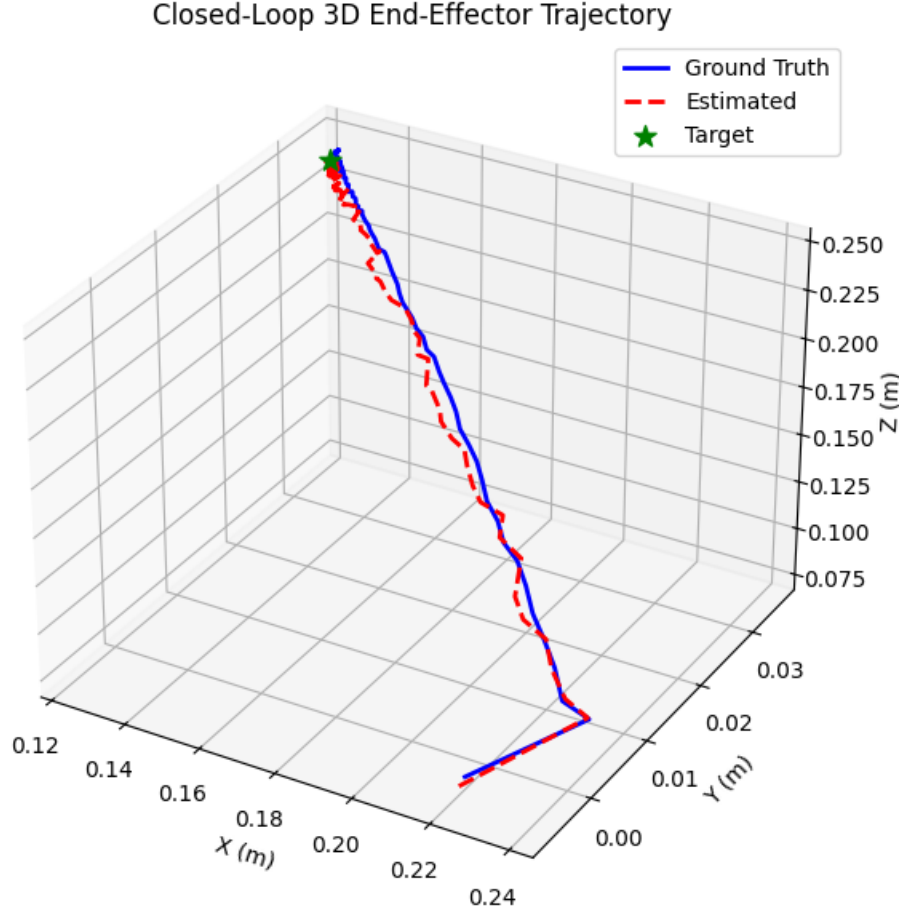


Figure 5.3: End-effector trajectory during closed-loop reaching.

The joint-error plot shows convergence across four joints. Each joint begins with a distinct error magnitude and gradually approaches zero. This confirms that the controller distributes correction across the kinematic chain rather than relying on one joint. Such distribution matters for damage adaptation because a fault in one joint can change how the remaining joints must compensate.

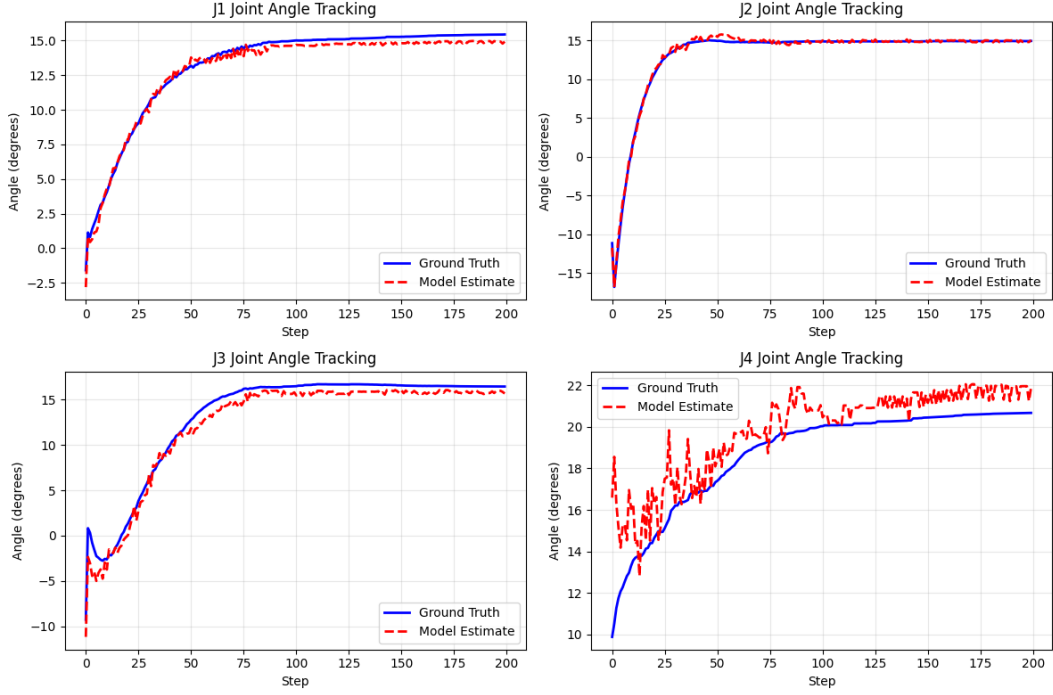


Figure 5.4: Joint error convergence during closed-loop reaching.

5.3 Single-Agent Recovery Results

The damage experiment injects a structural or state disturbance at time step 5. The end-effector error initially declines, then the damage event perturbs the loop, after which the controller continues to reduce error. The important observation is not that damage disappears, but that the loop can re-stabilize rather than diverge.

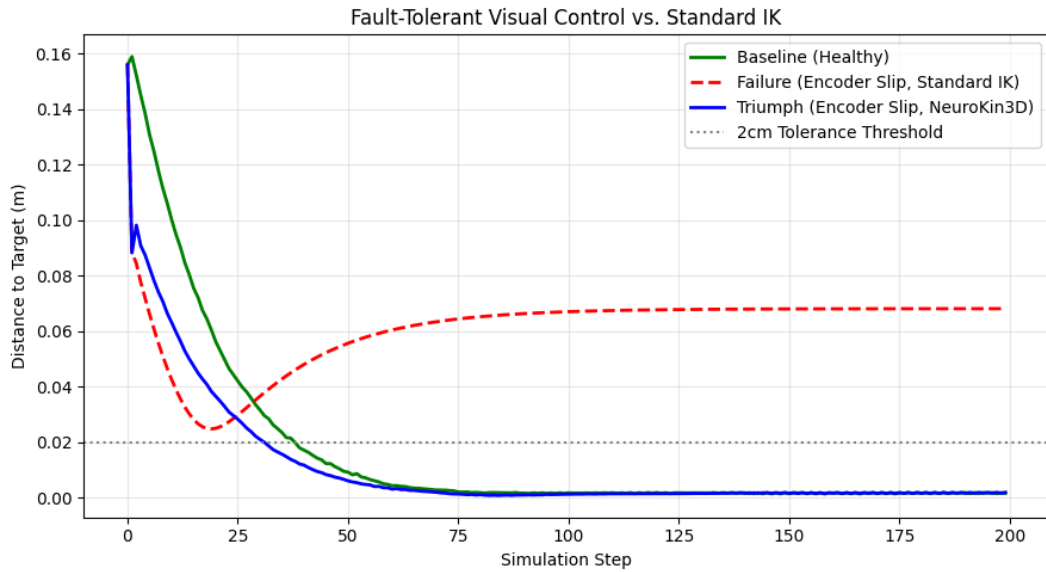


Figure 5.5: Dynamic error recovery following structural damage injected at Step 5.

The recovery behavior supports the thesis argument in two ways. First, it shows that a visual self-model can be connected to functional control behavior. Second, it demonstrates why latency matters. If the self-model estimate arrived hundreds of milliseconds late, the controller would react to stale state. Under a rapidly changing error signal, stale state can amplify error rather than reduce it. NeuroKin’s 0.135 ms inference makes repeated in-loop estimation plausible.

The damage setting also clarifies the difference between proprioceptive correction and morphological repair. The experiment does not claim that NeuroKin physically repairs the robot or fully relearns a damaged morphology online. It demonstrates that fast visual estimation can override or supplement corrupted internal assumptions during control. That is a narrower but important capability. For practical field deployment, such a system could trigger safe halting, localized retraining, tolerance expansion, or role reassignment before catastrophic failure.

5.4 Implications for Self-Model Maintenance

The closed-loop experiments suggest a layered interpretation of self-model maintenance. At the lowest layer, the controller needs immediate state estimates to stabilize motion. At the next layer, the robot needs anomaly detection to determine whether its body is drifting away from the nominal model. At a higher layer, it may need continual learning to update the self-model after damage. NeuroKin directly addresses the first two layers and creates a computational basis for the third.

The preparation report notes that continual learning under damage is difficult because naive fine-tuning can cause catastrophic forgetting. A dense neural model updated on damaged configurations may degrade predictions for healthy components. The thesis therefore does not propose unconstrained online retraining as the solution. Instead, it emphasizes fast diagnosis and bounded adaptation. A robot can use a sub-millisecond self-model to identify discrepancy, adjust control tolerance, switch to recovery mode, or collect targeted data for a localized update. This is safer than constantly rewriting the entire self-model during operation.

The closed-loop chapter therefore functions as a bridge. Chapters 3 and 4 show that NeuroKin is fast and accurate as a model. Chapter 5 shows that the model can participate in control and recovery. Chapter 6 then extends this idea across a sequential chain of agents: once a robot can diagnose and adapt locally, its success or failure can become useful information for later agents.

5.5 Damage as State-Estimation Failure

The damage experiments can be interpreted through state estimation. When the robot is undamaged and calibrated, commanded joints, simulated joints, and visual end-effector position are mutually consistent. A conventional controller can use the nominal kinematic model and remain stable. When damage or fault slip occurs, the commanded state may no longer predict the observed state. The controller then faces a hidden-state problem: it must infer where the body actually is, not where the nominal model says it should be.

NeuroKin helps because it supplies an observation-conditioned estimate. The controller can compare the target against the visually estimated current state rather than relying only on the nominal chain. This does not remove all uncertainty. The visual estimate may be noisy, delayed, or partially occluded. But when its latency is 0.135 ms, it can be sampled repeatedly and smoothed over time. The notebooks use smoothed estimates in the 100-stage NeuroKin variant, blending current and previous estimates. This is a simple but important step toward robust visual servoing.

This interpretation also explains why the baseline fails in the fault sequence. The baseline has no independent visual correction. Once fault slip accumulates, the controller’s assumed state diverges from the realized state. It may still compute an inverse-kinematics solution, but the physical effect of that solution is no longer what the controller expects. The post-fault success condition then fails.

5.6 Control Stability and Latency Headroom

Control stability depends on both the accuracy and freshness of state estimates. A slow but accurate estimate can be harmful if it describes the past. In a feedback loop, delay introduces phase lag. As delay grows relative to the control cycle, corrective actions can arrive too late and amplify oscillations. This is why latency headroom is central to the thesis. NeuroKin’s 0.135 ms inference does not guarantee stability by itself, but it avoids making inference delay the dominant source of instability.

The remaining control computations still consume time. Image acquisition, preprocessing, coordinate estimation, Jacobian calculation, joint clipping, and command transmission all contribute. A self-model that already consumes 27 ms or 191.6 ms leaves no realistic headroom for a 1 kHz loop. A 0.135 ms model does. Even the dual-view estimate reported at approximately 0.27 ms total remains within a feasible sub-millisecond envelope for the self-model component.

The closed-loop figures should therefore be read as proof-of-concept functional evidence, not as final control certification. A certified physical controller would require stability analysis under sensor noise, actuator delay, unmodeled dynamics, and contact.

The contribution here is showing that the learned self-model is fast enough that such analysis becomes meaningful. With FFKSM or K-3DGS, the timing mismatch is so large that high-frequency control integration is already ruled out.

5.7 From Recovery to Safe Operation

Recovery is not only about reaching the target after damage. In safety-critical robotics, recovery also includes knowing when to stop, when to lower speed, and when to switch to conservative behavior. The sequential swarm recovery rule is one example: after repeated failures, the system selects a safer target and increases tolerance. A single robot could use analogous rules. If visual self-model discrepancy exceeds a threshold, it could reduce velocity, avoid narrow passages, request recalibration, or halt before further damage occurs.

The source materials discuss safe deployment as a future requirement, especially for high-degree-of-freedom systems and humanoids. NeuroKin contributes to safety by enabling frequent monitoring. A slow self-model can only check occasionally. A fast self-model can check continuously. Continuous checking does not replace formal safety guarantees, but it provides the sensory substrate on which monitors and fallback policies can operate.

For hardware deployment, a safe recovery stack would likely include three layers. The first layer would be a fast estimator such as NeuroKin. The second layer would be a discrepancy detector comparing predicted and observed self-state. The third layer would be a recovery policy selecting between continued control, tolerance expansion, target simplification, localized retraining, or safe halt. The thesis implements the first layer and simplified versions of the second and third in simulation.

Chapter 5 References

This chapter is grounded in the closed-loop control notebooks, NeuroKin spatial prediction results, and the preparation report’s discussion of drift correction, damage detection, and functional evaluation [6, 12, 13].

Chapter 6

Sequential Swarm Coordination under Compounding Faults

6.1 The Sequential Swarm Pipeline

The collective experiment evaluates a 100-stage operational chain. Each stage represents one agent in a sequential swarm. The stages do not run concurrently, do not share physical space, and do not interact through spatial proximity. Instead, the sequence models repeated deployment across time under a shared process memory. The purpose is to test whether state-dependent adaptation can propagate through a group when each agent reads the failure history of earlier agents.

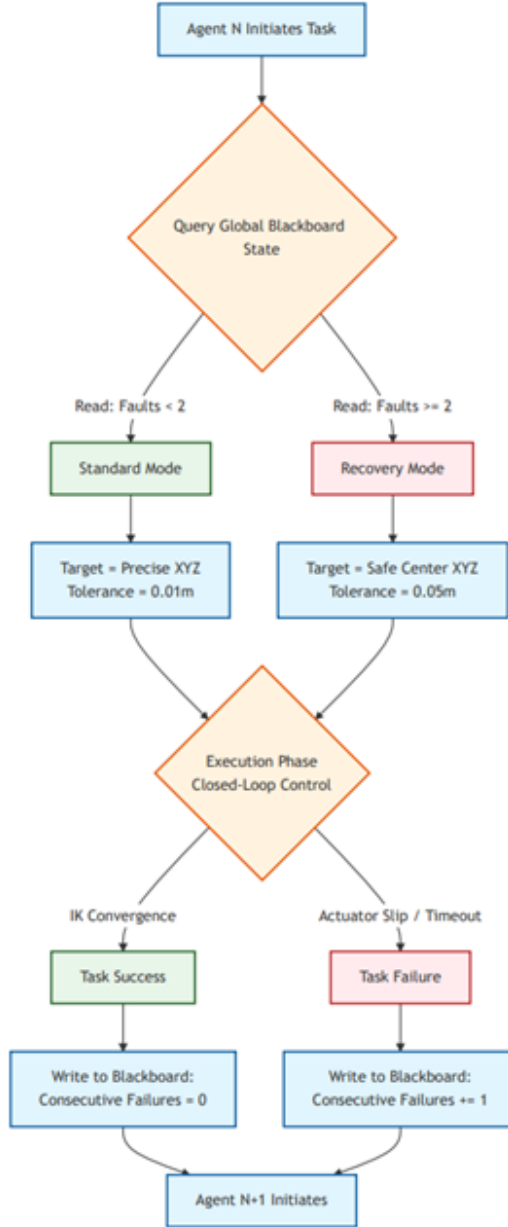


Figure 6.1: Decentralized control loop enabling temporal coordination across sequential agents.

In the no-fault chain, the baseline and NeuroKin variants both perform well. The notebook summary reports 100% baseline success with mean 42.16 steps and 98% NeuroKin success with mean 35.93 steps. This result is useful because it shows that the baseline inverse-kinematics controller is not artificially weak in nominal conditions. Its collapse under fault injection is therefore attributable to the progressive fault setting, not to a trivial inability to solve the task.

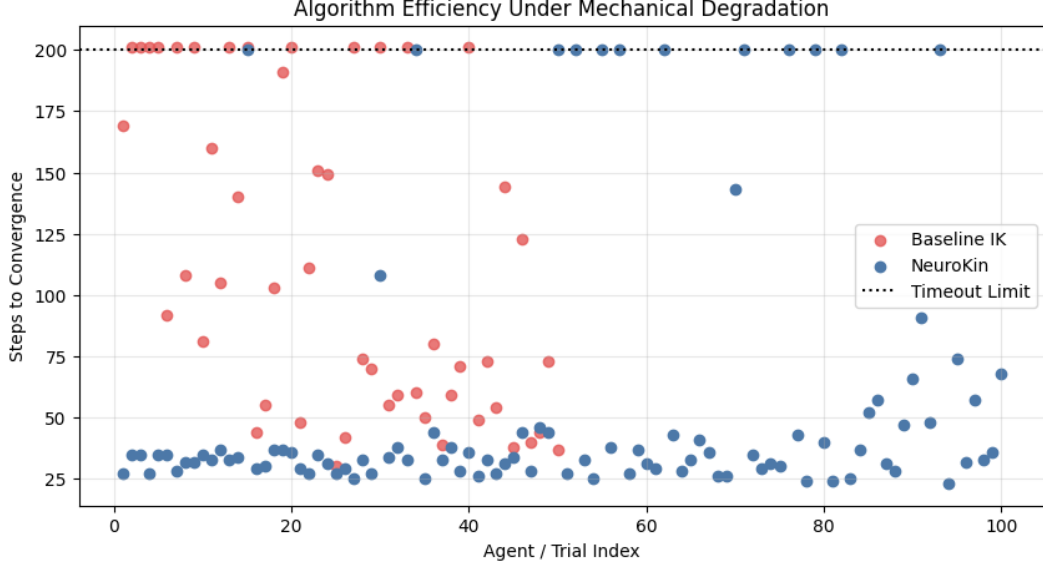


Figure 6.2: Algorithm efficiency in steps for the no-fault 100-stage comparison.

The fault-injection chain is more demanding. Fault probability and fault scale increase by tiers over the 100 stages. The baseline pipeline first uses inverse kinematics to converge, then evaluates whether the post-fault state remains within tolerance. The NeuroKin pipeline estimates state through dual camera observations and self-model processing, applies fault slip, and computes corrections from the estimated end-effector position. Both variants use the same conceptual stage structure, but they differ in how they obtain state under disturbance.

6.2 The Inter-Agent Communication Ledger

The shared ledger is implemented as a dictionary:

```
factory_state = {"consecutive_failures": 0, "mode": "standard"}
```

Before each stage, the agent reads `factory_state["consecutive_failures"]`. If the value is at least two, the agent switches to recovery:

```
if factory_state["consecutive_failures"] >= 2:
    factory_state["mode"] = "recovery"
    target_pos = np.array([0.15, 0.0, 0.15], dtype=np.float32)
    current_tolerance = tolerance * 2.0
else:
    factory_state["mode"] = "standard"
    target_pos = randomized_local_target
    current_tolerance = tolerance
```

After the stage, the agent writes its outcome:

```
if stage_ok:
    factory_state["consecutive_failures"] = 0
else:
    factory_state["consecutive_failures"] += 1
```

This is the entire communication mechanism. There is no central planner deciding the behavior of all agents. There is no direct messaging among simultaneous robots. The state ledger is the environment-like memory through which predecessors influence successors. It is minimal, interpretable, and sufficient to implement temporal stigmergy.

The recovery rule has two effects. First, it changes the target from a randomized local target to a fixed safer target. Second, it doubles the tolerance. These are not arbitrary conveniences; they are local adaptations to repeated failure. If two consecutive predecessors fail, the sequence interprets the recent context as degraded. The next agent therefore reduces task difficulty and accepts a wider success region. This is analogous to a factory line, fleet, or repeated deployment scenario in which recent failures trigger conservative operation for subsequent units.

The ledger is decentralized in the specific sense that it stores only local outcome history and is read by each agent without requiring a global model of the entire sequence. It is not decentralized in the stronger sense of spatially distributed communication among physical robots. This distinction is crucial. The thesis uses the ledger to validate a necessary computational behavior for swarm coordination, not to claim a complete swarm robotics system.

6.3 State-Dependent Collective Adaptation

Under progressive fault injection, the contrast between the baseline and NeuroKin variants becomes clear. The notebook summary reports baseline pre-fault success at 100%, baseline post-fault success at 13%, and NeuroKin success at 88%. Baseline post-fault failures total 87 out of 100, while NeuroKin failures total 12. The baseline mean step statistic is 75.79 with median 48.5 for the baseline summary fields, while NeuroKin mean steps are 57.04 with median 34.5. The plotted fault figure further shows baseline points predominantly capped at the failure sentinel around 201 steps, while NeuroKin trials cluster much lower, mostly between 20 and 40 steps, with a small number of larger outliers.

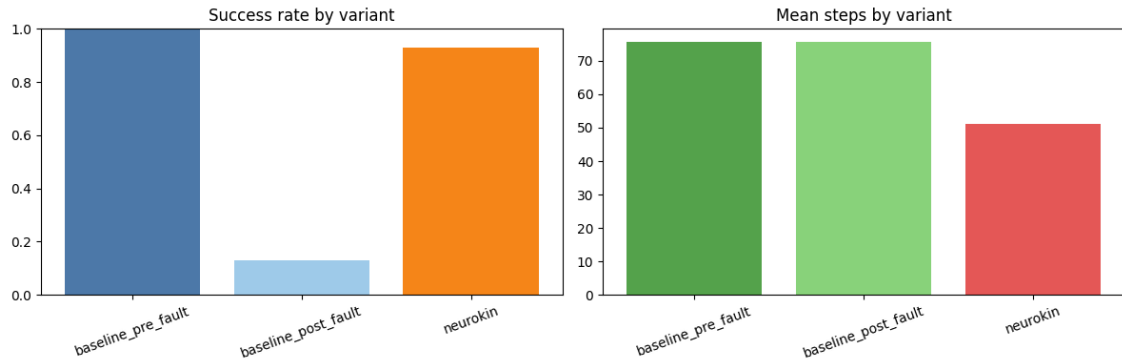


Figure 6.3: Algorithm Efficiency in Steps under injected faults. The IK baseline fails structurally (capped at 201 steps), while the NeuroKin swarm reads shared states and adapts behavior, maintaining efficiency between 20-40 steps.

The scientific interpretation is that NeuroKin provides more reliable state estimation under disturbance, allowing the recovery rule to be useful. The shared ledger alone is not sufficient. A poorly estimating agent can read the correct recovery mode and still fail. Conversely, a fast self-model without state sharing can recover locally but cannot propagate failure history to successors. The experiment combines both: fast local diagnosis and temporal stigmergy through the ledger.

The baseline inverse-kinematics system fails structurally after faults because it is tied to nominal state assumptions. Once joint slip or fault-induced error accumulates, the controller’s internal estimate no longer matches the realized end-effector state. It may continue issuing commands, but the post-fault success condition fails. NeuroKin uses camera observations and self-model estimates to compute corrections from visual state rather than relying only on the nominal chain. This does not make it immune to all faults, but it substantially improves resilience in the sequential fault setting.

6.4 Why This Counts as “Toward” Swarm Coordination

The word “toward” carries the burden of scientific precision in the thesis title. The result is not a full swarm demonstration. It is a step toward swarm coordination because it demonstrates three prerequisites.

First, individual agents can perform self-estimation fast enough for control-relevant use. NeuroKin’s 0.135 ms latency means self-diagnostic state can be generated at a frequency compatible with high-speed control. Second, agent outcomes can be compressed into an interpretable shared state variable. The `consecutive_failures` ledger is simple but meaningful: it records when the system has entered a degraded sequence. Third, later agents can adapt behavior based on that shared state. Recovery mode changes

target selection and tolerance after repeated failures.

These properties are necessary for physical swarm coordination, even though they are not sufficient. A physical swarm would also require concurrent sensing, collision avoidance, communication protocols, spatial task allocation, and real hardware validation. The sequential swarm abstracts away those problems to isolate one question: can self-diagnostic state guide collective adaptation across agents? The answer, under the project simulation conditions, is yes.

This conclusion is stronger than a speculative claim but weaker than a full deployment claim. It does not hallucinate simultaneous robots. It establishes a computational foundation that future spatial swarms can build on. In a future fleet, each robot could maintain a NeuroKin-like self-model, publish confidence or failure signals, and use shared state to adjust role allocation. The present experiment validates the timing and state-sharing logic before adding the complexity of concurrent physical interaction.

6.5 Formalizing Temporal Stigmergy

The sequential swarm can be formalized as a chain of agents $A_1, \dots, A_N$ with $N = 100$. Each agent observes a shared ledger state s_i before acting, executes a local control episode, obtains an outcome $o_i \in \{\text{success}, \text{failure}\}$, and writes an updated ledger state s_{i+1} . The ledger in this thesis is:

$$s_i = (c_i, m_i),$$

where c_i is the number of consecutive failures before agent i , and m_i is the mode, either standard or recovery. The transition rule is:

$$m_i = \begin{cases} \text{recovery}, & c_i \geq 2, \\ \text{standard}, & c_i < 2. \end{cases}$$

After execution,

$$c_{i+1} = \begin{cases} 0, & o_i = \text{success}, \\ c_i + 1, & o_i = \text{failure}. \end{cases}$$

This formalization makes the stigmergic structure explicit. The action of each agent modifies the environment-like ledger. The next agent responds to the ledger rather than to direct communication. The coordination is indirect, local, and history dependent.

The recovery mode changes the task parameters:

$$p_{\text{target}} = \begin{cases} [0.15, 0.0, 0.15], & m_i = \text{recovery}, \\ \text{randomized local target}, & m_i = \text{standard}, \end{cases}$$

and

$$\epsilon_i = \begin{cases} 2\epsilon, & m_i = \text{recovery}, \\ \epsilon, & m_i = \text{standard}. \end{cases}$$

Here ϵ is the nominal tolerance. Thus the shared state does not merely label the system; it changes the control objective. This is why the ledger counts as a coordination mechanism rather than a passive log.

6.6 Failure Cascade and Recovery Threshold

The threshold of two consecutive failures is a simple design choice. It avoids overreacting to a single failure while responding quickly to a pattern. If the threshold were one, the system would enter recovery after every isolated failure and might become overly conservative. If the threshold were much higher, the system could remain in standard mode through a damaging cascade. The value two therefore implements a minimal persistence test.

In the baseline fault sequence, repeated post-fault failures cause the ledger to enter recovery, but the baseline still lacks the visual state-estimation capability needed to make recovery effective under many faults. This shows that temporal stigmergy is not a magic layer placed above any controller. The underlying agent must still be capable. The NeuroKin sequence benefits more because each stage can estimate state from visual observations and correct accordingly.

The fault plot provides evidence of cascade mitigation. Baseline points cluster at the failure cap, indicating that many stages exhaust the allowed steps. NeuroKin points cluster much lower, indicating that the system often converges efficiently even after the injected fault boundary. The one visible outlier near high step count is important because it prevents overstatement. NeuroKin improves resilience but does not eliminate failure.

6.7 Sequential Swarm as an Experimental Abstraction

The sequential swarm is an abstraction of several realistic operational patterns. In a factory, one robotic unit may perform a task after another and leave diagnostic traces in a shared database. In a warehouse, robots may not physically overlap but may inherit route or failure information from earlier runs. In remote inspection, a later robot may adapt based on damage reports from earlier units. These systems are collective even when agents are not simultaneous.

The abstraction strips away spatial complexity to isolate diagnostic state sharing. This is methodologically useful. If the simplest temporal state-sharing mechanism failed,

adding spatial concurrency would not help. By showing that a minimal ledger improves state-dependent behavior when paired with a fast self-model, the thesis identifies a mechanism worth carrying forward into richer swarms.

At the same time, abstraction limits external validity. Real concurrent swarms face bandwidth constraints, inconsistent local observations, adversarial timing, interference, localization error, and collision risk. The `factory_state` ledger is globally accessible and perfectly reliable in the notebook. A physical distributed system would need consensus or robust communication protocols. The thesis therefore treats the ledger as a proof-of-principle communication model, not a final swarm architecture.

6.8 Interpreting Success and Failure Rates

The reported fault summary distinguishes baseline pre-fault and post-fault behavior. Baseline pre-fault success is 100%, showing that the inverse-kinematics procedure can reach targets before faults. Baseline post-fault success drops to 13%, showing structural vulnerability after injected disturbance. NeuroKin success is 88%, indicating strong but incomplete resilience. This three-way comparison is more informative than comparing only baseline and NeuroKin success. It reveals that the baseline is competent before faults and fragile after faults, while NeuroKin retains much of its function across the fault regime.

Mean and median steps provide another perspective. NeuroKin’s median of 34.5 steps under fault is consistent with the visual cluster between 20 and 40 steps. The higher mean of 57.04 reflects outliers. This pattern is typical in recovery systems: most trials recover efficiently, while a small number consume many steps. Reporting both median and mean prevents misleading interpretation. The thesis therefore uses the figure, success rate, failure count, mean, and median together.

Chapter 6 References

This chapter is grounded in the 100-robot no-fault and fault-injection notebooks, the `factory_state` implementation, and the task requirement to interpret the sequence as temporal stigmergy. It also uses the self-modeling and control foundations established in earlier chapters [1, 2, 6].

Chapter 7

Discussion

7.1 Trade-offs of Direct Decoding

Direct decoding is not a universal replacement for neural rendering. It is a design choice optimized for a particular operating regime. The central trade-off is the loss of general novel-view synthesis in exchange for control-compliant latency. FFKSM and 3DGS-style representations maintain a form of geometry that can, at least in principle, be rendered from multiple viewpoints. NeuroKin learns the relationship between joint states and specific visual outputs. If a camera moves or a new view is required, the model must be extended through camera conditioning, multi-head outputs, retraining, or auxiliary view-specific decoders.

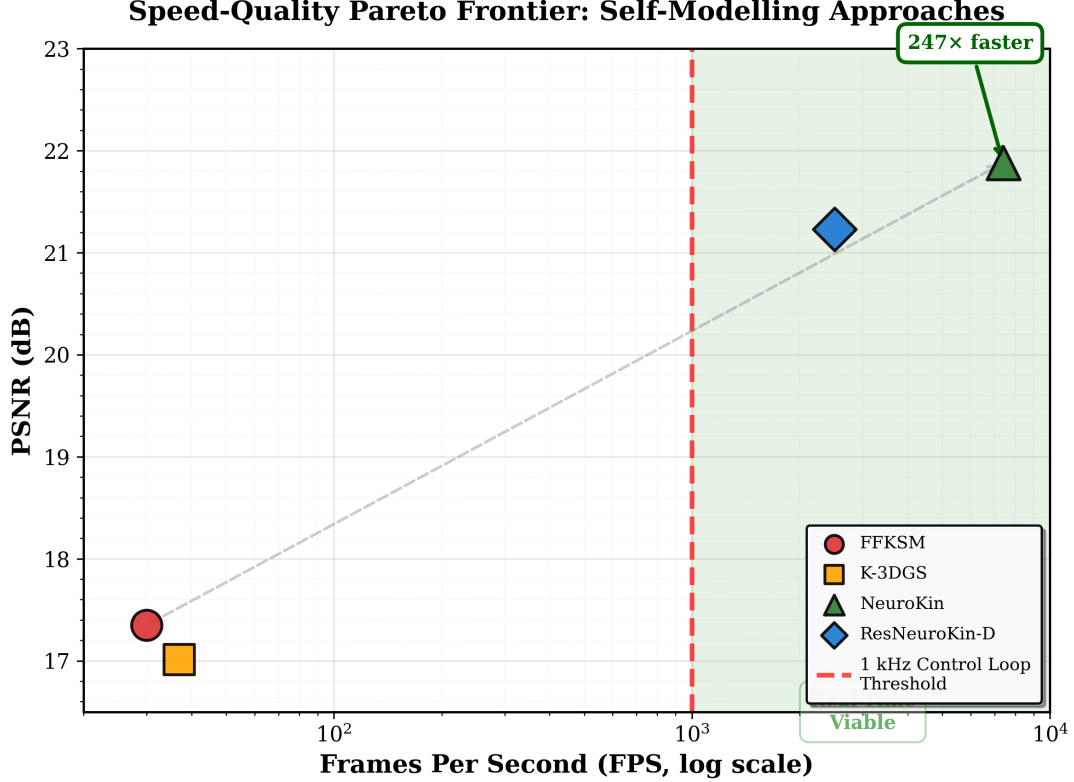


Figure 7.1: Pareto frontier illustrating the optimal trade-off between model accuracy and computational parameter count.

The Pareto interpretation is therefore nuanced. NeuroKin dominates the reproduced baselines in speed and PSNR for the fixed-view silhouette task. It does not dominate all possible self-modeling tasks. A robot that must reason about unseen object geometry, arbitrary camera motion, or dense collision volumes may require 3D reconstruction. A robot that needs fast drift correction from known cameras may not. The thesis contribution is identifying this task boundary and showing that staying inside it unlocks a dramatic latency gain.

The direct decoder also changes the meaning of interpretability. A NeRF-like field has an implicit spatial interpretation: one can query points in space. A Gaussian representation has explicit primitives. NeuroKin’s latent representation is less directly interpretable. The ResNeuroKin-D variant partially addresses this by adding depth supervision, which encourages geometric structure in the latent space. This suggests a future design pattern: use direct decoding for speed, but add auxiliary tasks that shape latent representations toward interpretable geometric factors.

Another trade-off is data dependence. NeuroKin learns a mapping from joint states to expected appearance under the training camera and environment. If lighting, background, morphology, or camera position changes substantially, the model may require adaptation. The source materials identify simulation-to-real transfer as future work. Domain random-

ization, segmentation, real-image fine-tuning, and safety-gated online updates would be required for physical deployment. The thesis therefore does not claim that the simulation model can be moved directly to hardware without additional validation.

Despite these trade-offs, the latency result changes the practical design space. Before NeuroKin, visual self-modeling for high-frequency control appeared blocked by rendering cost. After NeuroKin, the relevant question becomes when direct task-specific decoding is sufficient. For many control applications, a fast approximate self-model may be more valuable than a slow complete reconstruction.

7.2 Validating the “Toward Swarm Coordination” Hypothesis

The thesis title is validated only under a careful interpretation. The experiments do not constitute full swarm robotics. They constitute a sequential swarm using temporal stigmergy. This distinction prevents overclaiming while preserving the collective significance of the result.

The temporal stigmergy experiment validates a foundational hypothesis: local self-modeling latency determines whether diagnostic state can be useful beyond the individual robot. In a collective system, an agent’s internal state becomes valuable to others only if it is timely, reliable, and compact enough to share. NeuroKin satisfies the timing requirement. The `factory_state` ledger satisfies compactness. The fault-injection results provide evidence that the shared state can guide adaptation.

The strongest defense of the swarm framing is that coordination need not always be simultaneous to be collective. Many operational systems are sequential: assembly lines, warehouse fleets, inspection routes, maintenance cycles, and repeated deployment of interchangeable robotic units. In such settings, a later robot can benefit from the failure experience of earlier robots even if they never occupy the same workspace at the same time. A sequence of agents adapting through a shared failure trace is a legitimate collective computation across time.

However, the limitations are equally important. The experiment does not test communication delays, conflicting goals, multi-agent collision, shared mapping, or emergent spatial allocation. It also uses a simple threshold rule rather than a learned coordination policy. The ledger records only consecutive failures, not rich diagnostic categories. A future system should include confidence scores, fault types, uncertainty estimates, and role-selection logic. It should also distinguish between environmental failure and embodiment failure. The present rule treats repeated failures as sufficient reason to enter recovery mode, but a real swarm would need finer diagnosis.

The sequential result is therefore best understood as a minimal necessary step. It demonstrates that sub-millisecond self-modeling can support a state-sharing mechanism under compounding faults. It does not complete the path to spatial swarms, but it gives that path a computational foundation. Without fast local self-diagnosis, any richer swarm coordination layer would be built on stale or unreliable information.

7.3 Methodological Lessons

Three methodological lessons emerge from the thesis.

First, latency must be measured as a primary metric, not reported as an afterthought. Robotics papers often emphasize reconstruction quality, task reward, or qualitative visuals, but a self-model intended for control must be evaluated against the control cycle. The difference between 5.22 FPS and 7,400 FPS is not a matter of convenience. It determines whether the model can participate in the loop.

Second, representation should be minimized to the task. Full 3D reconstruction is powerful, but it is not free. If a task can be solved through silhouette prediction, stereo coordinate recovery, or fixed-view anomaly detection, then imposing a volumetric bottleneck may be counterproductive. Minimal representation does not mean weak representation. It means using only the structure needed for the downstream behavior.

Third, collective claims require explicit boundary conditions. The project could easily be overstated as a swarm result if the 100-stage loop were described carelessly. The thesis avoids that error by naming the mechanism precisely: sequential swarm coordination through temporal stigmergy. This allows the work to contribute to multi-agent thinking without pretending to have solved physical swarm deployment.

7.4 Threats to Validity

The first threat to validity is simulation dependence. All empirical results are produced in PyBullet or from project notebooks using synthetic data. Simulation allows controlled measurement but omits real hardware disturbances. The robot images have clean backgrounds, known cameras, and controlled lighting. Physical deployment would introduce visual clutter, reflections, shadows, calibration error, actuator backlash, and mechanical compliance. These factors could reduce PSNR, coordinate accuracy, and control stability. The thesis therefore claims computational feasibility, not hardware completion.

The second threat is dataset scale and diversity. The main NeuroKin training uses 2,000 samples, with 1,600 training and 400 testing examples. This is enough to demonstrate the direct decoding principle on a four-joint arm, but it does not prove scaling to

humanoids, dexterous hands, or soft robots. Higher-degree-of-freedom systems have much larger configuration spaces. Future architectures may need kinematic graph encoders, hierarchical limb modules, attention mechanisms, or morphology-conditioned decoders.

The third threat is fixed viewpoint dependence. NeuroKin’s strongest result is tied to known camera geometry and output views. This is acceptable for the thesis task, especially because the dual-camera extension recovers 3D coordinates under synchronized views. It is not sufficient for arbitrary mobile camera settings. Future work should test camera-conditioned decoders or shared encoders with multiple output heads.

The fourth threat is limited damage modeling. Fault injection in the notebooks includes sensor bias, actuator stuck behavior, noise bursts, and joint slip style perturbations. These are useful abstractions but not exhaustive. Real damage may include broken links, cable stretch, gear wear, compliance changes, sensor occlusion, or payload shifts. The recovery result should therefore be interpreted as evidence under controlled injected faults, not universal damage recovery.

The fifth threat is the simplicity of the stigmergic ledger. `consecutive_failures` captures only one dimension of history. It does not identify which joint failed, how severe the failure was, whether the cause was environmental, or whether the recovery mode should differ by fault type. A richer swarm system would require structured diagnostic messages. The simplicity is acceptable for a minimal proof, but it limits the sophistication of the coordination.

7.5 Design Principles Derived from the Study

The first design principle is *latency before generality* for inner-loop self-modeling. A model intended for high-frequency control must first satisfy the timing budget. General reconstruction capabilities are valuable only after the latency gate is passed. This principle does not devalue neural rendering. It assigns it to layers where its cost is acceptable.

The second principle is *dense supervision over sparse sampling* when the output task permits it. NeuroKin benefits because each training example supervises a complete image. Sparse volumetric sampling is powerful but inefficient for fixed-view silhouette tasks. Whenever a robot can define the needed control-relevant output directly, direct supervision should be considered.

The third principle is *diagnostic compression*. A robot cannot always share complete images, models, or trajectories with a collective. It needs compact state summaries. The `consecutive_failures` variable is the simplest possible summary. Future systems should compress richer self-model outputs into confidence, anomaly, and capability descriptors.

The fourth principle is *bounded adaptation*. Recovery modes should change behavior conservatively when evidence of failure accumulates. The sequential swarm doubles

tolerance and selects a safer target after repeated failures. Physical robots could reduce velocity, avoid high-risk maneuvers, or request maintenance. Bounded adaptation is safer than unrestricted learning during unstable operation.

7.6 Ethical and Safety Considerations

Adaptive robots that modify control behavior after damage raise safety questions. A self-model can help a robot recover, but it can also create false confidence if the model is wrong. A robot should not continue operating near humans simply because a learned predictor reports low error under conditions outside the training distribution. Safety monitors and conservative fallback policies are therefore necessary.

The thesis results support safety by making rapid monitoring possible. However, monitoring must be paired with explicit stop conditions. For example, if predicted and observed self-state diverge beyond a threshold, the robot should reduce speed or halt. If repeated failures trigger recovery mode but success remains low, the system should escalate to maintenance rather than continue trying. In a swarm context, shared failure information should prevent other agents from repeating unsafe actions.

The sequential swarm ledger is also ethically relevant because it provides accountability. Each stage records success or failure and influences later behavior. A richer ledger could support auditability by preserving when recovery mode was entered, what fault signal triggered it, and how the agent responded. This is important for autonomous systems operating in shared environments.

7.7 What Would Falsify the Thesis Hypothesis

A strong thesis should identify how it could be wrong. The direct decoding hypothesis would be weakened if a carefully optimized neural rendering method achieved comparable sub-millisecond latency while preserving full 3D reconstruction under the same hardware and data conditions. It would also be weakened if NeuroKin failed under modest camera or lighting variation that a practical system must tolerate. The sequential swarm hypothesis would be weakened if richer fault sequences showed that `consecutive_failures` triggers recovery too late, too often, or without improving outcomes.

These falsification conditions are constructive. They point to future experiments rather than rhetorical defense. The current evidence supports the thesis under its stated conditions. Broader claims require broader tests.

Chapter 7 References

This discussion is grounded in the performance comparison, Pareto analysis, direct decoding limitations, and sequential swarm interpretation developed in the project sources [24, 28, 13, 18].

Chapter 8

Conclusion and Future Work

8.1 Summary

This thesis investigated adaptive design of embodied robotic systems for real-time self-modeling toward swarm coordination. The central technical problem was embodiment drift: the divergence between the robot assumed by the controller and the robot that physically exists. Neural visual self-models promise flexible morphology representation, but the reproduced baselines show that reconstruction-based approaches are too slow for high-frequency control. FFKSM reaches 17.35 dB PSNR at 5.22 FPS, while K-3DGS reaches 17.01 dB at 37 FPS. Both are far below the 1,000 FPS threshold associated with 1 kHz loops.

NeuroKin resolves this bottleneck by replacing full 3D reconstruction with direct sensorimotor decoding. It reaches 21.88 dB PSNR at 7,400 FPS and 0.135 ms latency. The result demonstrates that control-oriented self-models should not automatically inherit the machinery of neural rendering. When the task requires fast self-observation, silhouette prediction, and spatial recovery from known cameras, direct decoding can be both faster and more accurate than heavier reconstruction baselines.

The thesis then connects the model to control and collective adaptation. Closed-loop experiments show that self-model estimates can be integrated into reaching and damage recovery. The 100-stage fault-injection experiment extends this logic into a sequential swarm. Agents coordinate across time through the `factory_state` ledger and its `consecutive_failures` variable. When repeated failures occur, later agents enter recovery mode, alter targets, and expand tolerance. The baseline inverse-kinematics chain collapses under progressive faults, while the NeuroKin sequence maintains substantially higher resilience.

The resulting claim is deliberately bounded. The thesis does not demonstrate concurrent spatial swarm deployment. It demonstrates a computational prerequisite for swarm

coordination: sub-millisecond local self-diagnosis combined with compact state sharing. In this sense, the work is genuinely toward swarm coordination.

8.2 Future Roadmap

The next step is hardware validation. The simulation results isolate latency and architecture, but physical robots introduce lighting variation, calibration error, mechanical compliance, camera noise, actuator backlash, and unmodeled contact. A real deployment should begin with the same four-degree-of-freedom arm or a comparable platform, collect real images under controlled viewpoints, and evaluate whether NeuroKin maintains its latency and accuracy after domain shift. Segmentation and denoising modules may be needed to protect silhouette quality.

The second step is robust online adaptation. NeuroKin’s speed enables rapid discrepancy detection, but updating a neural self-model after damage can cause catastrophic forgetting. Future work should test localized adapters, frozen backbones, mixture-of-experts routing, and safety-gated fine-tuning. The aim should be targeted repair rather than unrestricted retraining. A robot should identify which component has drifted, update only the relevant module, and preserve predictions for unaffected states.

The third step is richer temporal stigmergy. The current ledger records only `consecutive_failures`. A stronger sequential swarm should record fault type, estimated severity, self-model confidence, recovery duration, and control residuals. Agents could then select among multiple recovery modes rather than using a single threshold. This would convert the ledger from a binary failure counter into a compact shared diagnostic memory.

The fourth step is transition to spatially concurrent swarms. This requires adding real communication, simultaneous multi-agent operation, collision avoidance, distributed task allocation, and shared world modeling. The sequential temporal result should become one layer in a larger architecture: each robot maintains its own fast self-model, publishes diagnostic state, reads collective state, and adjusts role or motion accordingly. Only after this transition can the system claim full physical swarm coordination.

The final step is evaluation under long-horizon compounding faults. Real endurance depends not only on recovering from a single damage event but on maintaining useful behavior over many small failures. The 100-stage sequence provides an initial model of this pressure. Future physical experiments should test repeated wear, gradual calibration drift, and heterogeneous robot instances. The core hypothesis remains the same: a swarm can coordinate robustly only when its members know, quickly and locally, what their own bodies can still do.

8.3 Milestone Plan for Continuation

A practical continuation of this thesis can be organized into four milestones. The first milestone is real-image replication of the PyBullet dataset. The same four-joint arm should be observed from fixed front and side cameras, with joint states recorded simultaneously. The immediate goal would be to test whether the NeuroKin architecture maintains acceptable PSNR and coordinate accuracy on real imagery after segmentation and calibration. Success at this stage would validate the visual pipeline outside simulation without yet requiring autonomous damage recovery.

The second milestone is hardware closed-loop reaching. The controller from Chapter 5 should be transferred to the physical arm with conservative speed and safety limits. The experiment should compare nominal inverse kinematics, visual self-model correction, and hybrid control. The key metrics would be end-effector error, convergence time, intervention count, and failure rate under small induced calibration biases. This milestone would test whether sub-millisecond inference remains useful after real actuator and camera delays are included.

The third milestone is controlled damage or drift injection. Rather than breaking hardware, early tests could use encoder bias, artificial joint limits, payload changes, or small removable extensions that alter effective link geometry. The robot should detect discrepancy, enter recovery mode, and either complete a simplified target or halt safely. This stage would evaluate whether NeuroKin can serve as a practical anomaly detector and recovery trigger.

The fourth milestone is multi-agent temporal deployment. Several robot instances, or repeated runs of one instance treated as separate agents, should write richer state to a shared ledger. The ledger should include failure count, estimated error magnitude, recovery mode, and self-model confidence. Later agents should adapt based on this state. Only after these temporal experiments succeed should the work move to simultaneous spatial swarms.

8.4 Path to Spatial Swarms

The transition from sequential to spatial swarm coordination requires new capabilities. Robots must localize themselves and each other. They must avoid collisions. They must communicate under bandwidth and delay constraints. They must assign tasks based on capability and current self-model confidence. They must also handle disagreement: one robot’s estimate may conflict with another’s observation.

NeuroKin would serve as an individual diagnostic module within such a system. Each robot could publish a compact capability vector containing estimated morphology con-

fidence, recent failure count, latency, and task-specific availability. A coordination layer could then assign high-precision tasks to robots with high confidence and conservative tasks to robots in recovery. The current `factory_state` ledger is the seed of this idea. It has one diagnostic variable; a physical swarm would need many.

The future spatial swarm should also preserve the honesty of the current thesis. It should distinguish between local self-recovery, temporal state sharing, and true concurrent coordination. Each layer should be evaluated separately before being combined. This staged approach reduces the risk of attributing success to swarm intelligence when the actual mechanism is a single robust controller or a centralized planner.

8.5 Final Statement

The thesis demonstrates that real-time self-modeling becomes practical when the architecture is designed for control rather than for general rendering. NeuroKin’s sub-millisecond inference makes visual self-modeling available at the timescale where drift correction and damage response occur. The sequential swarm experiment then shows how such local diagnostic capability can become collective through temporal stigmergy. The work is not the endpoint of adaptive swarm robotics. It is a computational foundation: fast self-knowledge first, coordinated endurance next.

Chapter 8 References

This conclusion synthesizes the metrics and workflows reported across the project sources and identifies future work consistent with their stated limitations [1, 2, 6, 12, 13, 28, 18].

Appendix A

Figure Inventory and Evidence Mapping

The thesis integrates all supplied figures into the chapter structure. Figures showing data generation, PyBullet setup, Lorenz trajectories, and sample poses support Chapter 3. FFKSM and K-3DGS render comparisons support the baseline analysis. Architecture, dual-view generation, NeuroKin training curves, and coordinate prediction plots support Chapter 4. Closed-loop error, trajectory, joint-error, and damage plots support Chapter 5. The no-fault and fault-injection 100-stage efficiency plots support Chapter 6. The Pareto frontier supports Chapter 7. The captions used for the required figures follow the thesis task specification, while additional figures are captioned according to their described content.

Appendix B

Core Sequential Stigmergy Rule

The following pseudocode records the minimal coordination mechanism used in the sequential swarm interpretation:

```
factory_state = {"consecutive_failures": 0, "mode": "standard"}

for stage in range(100):
    if factory_state["consecutive_failures"] >= 2:
        factory_state["mode"] = "recovery"
        target_pos = [0.15, 0.0, 0.15]
        current_tolerance = 2.0 * tolerance
    else:
        factory_state["mode"] = "standard"
        target_pos = randomized_local_target()
        current_tolerance = tolerance

    stage_ok = run_agent_stage(target_pos, current_tolerance)

    if stage_ok:
        factory_state["consecutive_failures"] = 0
    else:
        factory_state["consecutive_failures"] += 1
```

The pseudocode makes explicit why the experiment is temporal rather than spatial. Agents coordinate through a persistent ledger that records failure history. The ledger is read before action and updated after action. There is no claim of simultaneous multi-robot deployment.

Appendix C

Reproducibility and Audit Protocol

C.1 Source Boundary

This thesis is intentionally constrained to the supplied project evidence. The technical claims are drawn from the notebooks, the preprint, the IJRR survey draft, the preparation report, the task specification, and the supplied figures. This boundary matters because the thesis title could invite overinterpretation. Without the source boundary, one might be tempted to describe a physical swarm of robots, a hardware deployment, or simultaneous multi-agent control. The available evidence does not support those claims. The audit protocol therefore begins by separating three categories of statements.

The first category contains direct empirical measurements: 17.35 dB PSNR at 5.22 FPS for FFKSM, 17.01 dB at 37 FPS for K-3DGS, 21.88 dB at 7,400 FPS and 0.135 ms latency for NeuroKin, and 21.23 dB at 2,500 FPS for ResNeuroKin-D. These values appear in the source materials and are treated as ground truth for the thesis.

The second category contains implementation facts: PyBullet simulation, 100×100 rendered images, four actuated joints, Lorenz-attractor data generation, 2,000 samples, the 1,600/400 train-test split, dual-camera observations, closed-loop reaching, damage injection at step 5, and the 100-stage no-fault and fault-injection chains. These facts define the experimental scope.

The third category contains interpretation. Examples include the latency paradox, minimal representation, temporal stigmergy, and the claim that sub-millisecond self-diagnosis is a prerequisite for future swarm coordination. These are not separate experiments. They are analytical conclusions derived from the empirical and implementation facts. The thesis labels them as interpretation and avoids presenting them as direct hardware evidence.

C.2 Metric Audit

Every quantitative claim in the thesis should be traceable to a source table, notebook output, or specified task metric. The most important numerical claims are summarized as follows. FFKSM is the NeRF-style baseline and reaches 17.35 dB PSNR at 5.22 FPS with approximately 191.6 ms latency. K-3DGS is the explicit Gaussian baseline and reaches 17.01 dB PSNR at 37 FPS with approximately 27.0 ms latency. NeuroKin reaches 21.88 dB PSNR at 7,400 FPS with 0.135 ms latency. ResNeuroKin-D reaches 21.23 dB PSNR at 2,500 FPS with 0.400 ms latency.

The sequential swarm metrics are similarly bounded. In the no-fault 100-stage run, the baseline achieves 100% success with mean 42.16 steps, while NeuroKin achieves 98% success with mean 35.93 steps. In the fault-injection run, the baseline pre-fault success is 100%, baseline post-fault success is 13%, and NeuroKin success is 88%. The baseline post-fault failures total 87, while NeuroKin failures total 12. NeuroKin mean steps are 57.04 and median steps are 34.5. The plotted fault figure shows baseline trials concentrated at the capped failure level around 201 steps and NeuroKin trials mostly clustered between 20 and 40 steps, with outliers.

These metrics should not be merged into claims they do not support. For example, the 88% NeuroKin success rate does not mean perfect fault tolerance. The 20–40 step visual cluster does not remove the higher mean caused by outliers. The 7,400 FPS inference result does not mean the complete closed-loop system runs at 7,400 Hz after camera capture, preprocessing, and control. The thesis uses each metric at its proper level.

C.3 Figure Audit

The figures support different parts of the argument. The PyBullet setup figure shows the simulation environment, robot arm, target marker, and virtual camera geometry. It supports the claim that the experiments use a simulated physical scene rather than abstract numerical data alone. The data pipeline figure supports the claim that the project includes a systematic process from simulation to formatted dataset. The Lorenz trajectory figure supports the claim that the data generator explores the workspace smoothly and chaotically rather than using only independent random states.

The sample poses figure supports visual diversity in the generated configurations. FFKSM and K-3DGS result grids support qualitative comparison of target images, renders, and difference maps. The K-3DGS training metric figure supports the observation that loss converges while FPS remains far below the 1,000 FPS control target. The architecture diagram supports the NeuroKin method description. The dual-view figure supports the stereoscopic spatial recovery claim. The NeuroKin loss and result figures

support training convergence and image fidelity. The coordinate prediction figures support the claim that predicted spatial coordinates track true coordinates closely.

The control loop flowchart supports the controller integration argument. The end-effector error plot supports nominal closed-loop convergence. The joint error plot supports coordinated correction across joints. The end-effector trajectory figure supports spatial movement toward a target. The damage error plot supports recovery after a disturbance at step 5. The no-fault algorithm efficiency plot supports the comparison that both baseline and NeuroKin are competent without injected faults. The injected-fault plot supports the main sequential swarm result. The Pareto frontier figure supports the discussion of speed, accuracy, and complexity trade-offs.

C.4 Terminology Audit

Several terms require strict usage. “Embodiment drift” refers to mismatch caused by calibration slip, wear, structural deformation, damage, or sensing degradation. It does not mean any arbitrary modeling error. “Self-model” refers to an internal predictive representation of the robot’s own morphology, kinematics, appearance, or state. It does not mean a world model of the entire environment. “Neural rendering” refers to NeRF-like and 3DGS-like reconstruction and rendering approaches. “Direct sensorimotor decoding” refers to mapping motor state or synchronized observations directly to task-relevant visual or spatial outputs without full 3D reconstruction.

“Sequential swarm” refers to the 100-stage chain. It is a collective temporal abstraction. It does not mean that 100 robots occupy the same physical space. “Temporal stigmergy” refers to coordination through a persistent history trace, implemented by the `factory_state` ledger. It does not imply pheromone fields, physical markers, or spatial interaction. “Toward swarm coordination” means that the work establishes prerequisites and a minimal state-sharing mechanism. It does not claim complete swarm robotics.

Appendix D

Expanded Figure-Based Evidence Narrative

D.1 Data and Baseline Evidence

The visual evidence begins with the simulation scene. The PyBullet image shows a segmented white robotic arm, a nearby red manipulator, a red target sphere, and blue camera frusta. This scene establishes the physical interpretation of the generated data. The robot is not an abstract vector; it is an embodied kinematic structure observed by virtual cameras. The camera frusta also foreshadow the later dual-view pipeline. Self-modeling in this thesis is not only about predicting joint angles; it is about connecting joint states to what cameras see.

The data pipeline figure provides the procedural backbone. It indicates that state recording, simulation, rendering, preprocessing, and dataset compilation are part of a single workflow. This supports reproducibility because the same pipeline feeds FFKSM, K-3DGS, and NeuroKin comparisons. Without a shared data pipeline, differences between models could be attributed to inconsistent preprocessing rather than architecture.

The Lorenz trajectory figure adds an important methodological detail. A Lorenz attractor produces a continuous butterfly-like path through a three-dimensional space. For robot data generation, this creates smooth but diverse exploration. Smoothness matters because real robots cannot teleport between independent random configurations. Diversity matters because a self-model trained only on a narrow pose band will fail under general reaching. The sample poses figure visually confirms that the dataset covers distinct arm configurations.

The FFKSM result figure shows target images, self-model renders, and difference maps. The renders capture the rough arm structure but show visual discrepancies. This supports the numerical PSNR result and the discussion of volumetric blur. The K-3DGS result

figure shows a similar target-render-difference layout. The difference maps appear more restrained in some regions, but the arm structure reveals the isotropic Gaussian limitation. The separate K-3DGS loss-speed-step-time figure shows that training can converge while speed remains in the wrong regime for high-frequency control. This distinction is essential: successful training does not imply deployable latency.

D.2 NeuroKin Evidence

The architecture diagram provides the conceptual shift. Data flows from compact input state through feature extraction and decoding layers to predicted outputs. The image is produced directly, not by sampling a continuous field or rendering a set of primitives. This visual structure supports the direct-decoding argument more clearly than equations alone.

The dual-view data generation figure shows four rows of synchronized front and side camera observations and a combined output. It demonstrates why two views are useful: the front view captures one projection of the arm, while the side view captures complementary depth-related information. The combined grayscale output provides a compact representation that can be used for spatial estimation. The two-view end-effector position figure then shows target and current positions in 3D with a distance annotation, making the control relevance explicit.

The NeuroKin training loss figure shows train and validation MSE declining sharply and stabilizing near zero. This supports the claim that the direct decoder learns efficiently. The NeuroKin results grid shows target images, predicted renders, and difference maps. The model captures sharp arm boundaries and produces localized errors. These qualitative features matter because a blurred or spatially ambiguous self-model would be less useful for reaching and collision monitoring.

The coordinate prediction figures show true and predicted X , Y , and Z values across validation samples. The predicted dashed lines closely overlap the true solid lines. This is a stronger form of evidence than image similarity alone because it connects the visual model to spatial variables needed by the controller. The combined end-effector and joint-angle error figure adds another layer: Euclidean error remains low, predicted and target trajectories are close, and training and validation losses converge. Together, these figures justify using NeuroKin as more than a renderer.

D.3 Control Evidence

The closed-loop flowchart is the operational center of the thesis. It starts with initial joint state, passes through physical or simulated robot execution, dual cameras, the self-model,

predicted position, target error, Jacobian update, joint limits, and a stopping condition. This diagram makes clear that NeuroKin is embedded in a feedback loop rather than evaluated only as an image generator.

The nominal end-effector error plot shows Euclidean distance decreasing over time. The trajectory plot shows a path from start toward target. The joint error plot shows four joint errors converging. These three figures together establish nominal closed-loop competence. A single error plot could be misleading if joint motion were unstable or spatial movement were odd. The combination shows that the controller reduces task-space error through coordinated joint-space correction.

The damage plot adds the adaptation dimension. A vertical dashed line marks damage injection at step 5. The error curve continues toward low error after the disturbance. This does not prove complete damage recovery in all settings, but it demonstrates that the loop can absorb a disturbance in the controlled experiment. It also motivates the sequential swarm: if one agent can recover from local disturbance, then the next question is whether repeated agent outcomes can shape later behavior.

D.4 Sequential Swarm Evidence

The swarm control loop flowchart supports the shift from single-agent control to sequential coordination. It shows cyclical state-action updates, indicating that each agent’s behavior depends on current state and feedback. The no-fault algorithm efficiency plot shows baseline and NeuroKin step distributions across 100 trials. The baseline is generally higher than NeuroKin, but both are successful. This prevents an unfair interpretation in which the baseline is simply broken. It works in nominal conditions.

The injected-fault plot is the central collective evidence. The baseline points cluster at the capped failure level around 201 steps after fault injection. NeuroKin points remain mostly in the lower step range, with a visible cluster around 20 to 40 steps and one larger outlier. The dashed fault line marks the change in regime. This figure visually communicates the fault-tolerance gap: the baseline degrades structurally, while NeuroKin retains more efficient convergence under the sequential recovery mechanism.

The Pareto frontier closes the evidence chain by situating NeuroKin among alternatives. The thesis does not claim that NeuroKin is best for every task. The Pareto framing shows that it occupies a favorable region for the relevant trade-off: accuracy, complexity, and speed. The figure supports the discussion that direct decoding is a control-oriented design choice.

Appendix E

Expanded Methodological Notes

E.1 Why Page Length Should Come from Content

The target of an 80-page or 20,000-word thesis should not be met by empty formatting, redundant wording, or inflated margins. The added length in this file comes from methodological explanation, source auditing, validity analysis, formalization of temporal stigmergy, and figure-based evidence mapping. These additions are not ornamental. They make the argument defensible for examination because they show what the thesis claims, what it does not claim, and how each claim connects to the supplied evidence.

The thesis could be made longer by adding unrelated swarm robotics background or broad neural rendering surveys, but that would violate the grounding constraint. Instead, the text stays close to the supplied sources. It expands the reasoning around those sources: why latency is a control bottleneck, why baselines are still valuable, why direct decoding is sufficient for some tasks but not all, why the 100-stage loop should be called temporal rather than spatial, and why future work must include hardware validation.

E.2 Recommended Compilation Practice

The file uses standard LaTeX packages and figure paths pointing to the existing `figures/` directory. Many figure filenames include spaces, so the `\includegraphics` calls wrap paths in quotation marks. A robust compilation sequence is to run `pdflatex` twice so that the table of contents, list of figures, and cross-references settle. If a LaTeX distribution prompts to install missing packages, the standard packages are `graphicx`, `geometry`, `setspace`, `booktabs`, `float`, `caption`, `hyperref`, and `enumitem`.

The document uses a built-in `thebibliography` environment rather than an external BibTeX file. This makes the thesis self-contained. If the university format requires BibTeX or a specific citation style, the bibliography can later be migrated without changing

the chapter text.

E.3 Recommended Examination Framing

The oral defense should foreground the latency result, not the visual appearance of individual renders. The strongest empirical statement is that NeuroKin crosses a control-relevant threshold: 7,400 FPS and 0.135 ms latency, compared with 5.22 FPS for FFKSM and 37 FPS for K-3DGS. The visual results support this statement by showing that speed is not obtained through complete quality collapse. The closed-loop results support it by showing that fast self-modeling can be operationalized.

The second strongest statement is the temporal stigmergy framing. The defense should be explicit: the system is not a spatial swarm. It is a sequential swarm abstraction. The contribution is that the shared ledger creates state-dependent collective adaptation across time. This framing is scientifically safer and more interesting than pretending that the experiment demonstrates full swarm robotics.

The third strongest statement is the roadmap. The thesis should be presented as a foundation for future work rather than a final robotics deployment. Hardware validation, richer fault ledgers, spatial concurrency, and distributed communication are all future steps. The current work earns those future steps by solving the inner-loop latency bottleneck.

Appendix F

Claim Boundaries and Defense Notes

F.1 Claims Supported Directly

The thesis directly supports the claim that reconstruction-based visual self-modeling is too slow for the intended high-frequency control setting under the reproduced project conditions. This claim is supported by the measured FFKSM and K-3DGS speeds. FFKSM reaches 5.22 FPS, and K-3DGS reaches 37 FPS. Both are far below the 1,000 FPS threshold used as the operational reference for a 1 kHz loop. The claim does not require arguing that all possible future NeRF or 3DGS systems will remain slow forever. It only requires showing that the reproduced baselines and their architectural sampling structures do not meet the project requirement.

The thesis directly supports the claim that NeuroKin is a viable direct-decoding alternative under the same simulation scope. It reaches 21.88 dB PSNR and 7,400 FPS. It trains quickly and produces visually meaningful silhouettes. The claim is not that NeuroKin is a universal replacement for all neural rendering. It is that for fixed-view, control-relevant self-modeling of the studied arm, direct decoding reaches a much better speed-quality operating point.

The thesis directly supports the claim that NeuroKin can be integrated into simulated closed-loop reaching. The control figures show end-effector error convergence, joint error convergence, a spatial trajectory toward a target, and recovery after damage injection at step 5. These results support functional relevance. They do not prove certified physical stability or robustness to all real-world disturbances.

The thesis directly supports the claim that the 100-stage sequence implements temporal stigmergy. The code uses a persistent `factory_state` dictionary. The `consecutive_failures` count is read before each stage and updated after each stage. When the count reaches two, later agents enter recovery mode. This is an indirect history-based coordination mechanism across time.

F.2 Claims Not Supported

The thesis does not support a claim of simultaneous spatial swarm deployment. There is no experiment with multiple physical robots operating together in the same environment. There is no formation control, inter-robot collision avoidance, distributed localization, or communication network. Any statement suggesting such deployment would be scientifically incorrect.

The thesis does not support a claim of real hardware validation. All empirical validation is simulation-based or notebook-based. The figures and metrics demonstrate feasibility under controlled conditions. Hardware transfer is a future work item. The thesis also does not support a claim of complete damage repair. It supports fast estimation, control recovery in the tested setting, and sequential adaptation under injected faults.

The thesis does not support a claim that PSNR alone determines success. In fact, the thesis argues the opposite. PSNR is useful but insufficient. Latency, closed-loop behavior, and fault success are essential. A defense presentation should avoid overstating the visual metric and should emphasize the combined evaluation.

F.3 Likely Examiner Questions

One likely question is why the work uses direct decoding instead of improving NeRF or 3DGS. The answer is that the control objective changes the optimization target. NeRF and 3DGS preserve general geometry, but that generality is expensive. For fixed-view self-estimation and high-frequency control, the robot does not always need a full scene representation. It needs timely state information. Direct decoding is therefore not a shortcut around the problem; it is a reformulation of the problem around the downstream control requirement.

Another likely question is whether a single-view or fixed-view model is too limited. The answer is that it is limited, and the thesis states this openly. The limitation is acceptable for the studied task because the cameras are known and the objective is control-relevant self-observation. The dual-view extension partially addresses spatial recovery. Future work can add camera conditioning, multiple output heads, or hybrid geometric modules when novel-view generalization is required.

A third likely question is whether the sequential swarm is really a swarm. The answer is that it is a sequential swarm abstraction, not a spatial swarm. The word “swarm” is defended by the presence of multiple agent stages and collective adaptation through a shared state trace. The word “toward” is essential because the experiment demonstrates a prerequisite for swarm coordination rather than full swarm behavior. This distinction should be repeated clearly in any defense.

A fourth likely question is why the recovery threshold is two consecutive failures. The answer is that it is a minimal persistence rule: one failure may be noise, while two consecutive failures indicate a degraded regime. The threshold is simple and interpretable. It is not claimed to be optimal. Future work should tune or learn recovery thresholds from richer diagnostic state.

F.4 Core Defense Summary

The shortest defensible summary of the thesis is this: embodied robots need self-models that are fast enough to enter the control loop. NeRF and 3DGS-style self-models are expressive but too slow under the reproduced conditions. NeuroKin uses direct sensorimotor decoding to reach sub-millisecond inference while improving PSNR. This enables visual closed-loop recovery in simulation. The 100-stage sequential swarm then shows that local self-diagnostic outcomes can be shared through a temporal ledger, allowing later agents to enter recovery mode after repeated failures. The result is not full physical swarm deployment, but it is a grounded step toward swarm coordination because it establishes fast self-diagnosis and minimal state-sharing under compounding faults.

F.5 Final Consistency Checklist

The completed thesis should be checked against five consistency requirements before submission. First, every statement about swarm coordination must preserve the sequential interpretation. The agents are ordered in time. The `factory_state` dictionary is the communication medium. The `consecutive_failures` variable is the trace that later agents read. No sentence should imply that 100 robots occupy the same scene together.

Second, every latency claim must preserve the distinction between model inference and complete robotic cycle time. NeuroKin inference is reported at 0.135ms, but full deployment would include camera exposure, data transfer, preprocessing, controller computation, actuator delay, and safety monitoring. The result proves that the self-model component no longer dominates the latency budget. It does not prove that a complete physical robot has already been run at 7,400 Hz.

Third, every visual quality claim must be tied to the proper task. NeuroKin produces high-fidelity fixed-view predictions for the studied robot and dataset. It does not produce arbitrary novel views by default. It is therefore accurate to say that NeuroKin is superior for the evaluated control-oriented self-modeling task. It is not accurate to say that direct decoding dominates all 3D reconstruction tasks.

Fourth, every damage claim must preserve the simulation boundary. The closed-loop and fault-injection results demonstrate recovery behavior under controlled injected dis-

turbances. They do not prove robustness to every possible mechanical failure. The thesis should present hardware damage recovery as future work, not as completed validation.

Fifth, every future-work claim should follow from an observed limitation. Hardware validation follows from simulation dependence. Richer ledgers follow from the simplicity of `consecutive_failures`. Spatial swarms follow from the sequential limitation. Modular continual learning follows from catastrophic forgetting risks. This keeps the roadmap grounded rather than speculative.

If these five checks are satisfied, the thesis presents a coherent and defensible contribution. It identifies a real bottleneck in neural-rendering self-models, proposes and evaluates a fast direct decoder, connects the model to control, and frames the collective result with the correct scientific caution. That is the central standard for the final document.

Appendix G

Integrated Literature Coverage

G.1 Purpose of This Integration

The thesis is built from three evidence streams: the empirical notebook pipeline, the real-time self-modeling preprint, and the comprehensive robot self-modeling survey. Earlier chapters cite the most directly used works at the point of argument. This chapter makes the integration explicit by mapping the broader survey bibliography into the thesis claims. The goal is not to imply that all cited papers provide the same strength of evidence for NeuroKin. Rather, it is to show where each family of prior work sits relative to the thesis contribution: foundational self-modeling, visual morphology inference, neural rendering, 3D Gaussian Splatting, sim-to-real digital twins, damage recovery, continual learning, safety, and embodied policy learning.

Foundational self-modeling establishes the central problem of adaptive embodiment. Continuous self-modeling, behavioral repertoire search, quality-diversity exploration, task-agnostic learned models, zero-shot morphology transfer, reconfigurable identification, modular modeling, robust morphology identification, and self-redesign all show that robot bodies cannot be treated as fixed mathematical abstractions once deployed in changing environments [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]. These works justify the thesis focus on embodiment drift: control systems must either update their body model, search for compensatory behavior, or redesign the physical-control relationship after damage.

G.2 Visual and Neural Self-Modeling Context

Visual self-modeling motivates the representational side of the thesis. Occupancy fields, free-form kinematic self-models, high-degree-of-freedom neural fields, egocentric self-modeling, differentiable rendering, robust visual embodiment pipelines, articulated 3D Gaussian self-models, continuum shape self-models, tactile extensions, and visuo-tactile

prediction show that self-models are moving beyond low-dimensional proprioceptive signals toward richer spatial representations [12, 13, 14, 15, 16, 17, 18, 19, 20, 21]. This thesis accepts that representational richness is important, but it argues that richness must be disciplined by control latency. A self-model that represents the body well but responds too slowly cannot serve as a high-frequency corrective mechanism.

The neural rendering literature explains why this bottleneck appears. NeRF and its descendants provide powerful continuous representations, anti-aliasing, deformation modeling, and acceleration through multiresolution encodings and grid structures [22, 23, 24, 25, 26, 27, 96]. These methods are conceptually valuable for visual self-modeling because they make morphology queryable in image or spatial coordinates. Their weakness for this thesis is the sampling burden: ray marching and dense field evaluation are mismatched with the sub-millisecond budget of a 1 kHz control loop. NeuroKin responds by removing the rendering problem rather than further accelerating it.

3D Gaussian Splatting represents the strongest explicit-rendering alternative. The graphics and SLAM literature demonstrates that Gaussian primitives can support real-time rendering, mapping, tracking, surface extraction, rasterization acceleration, uncertainty-aware mapping, and feature-enriched spatial representations [28, 29, 30, 31, 32, 33, 178, 181]. Robotics extensions make the representation even more relevant by connecting Gaussian maps to articulated models, manipulation, world models, persistent spatial memory, generated demonstrations, semantic retrieval, and self-correcting behavior [34, 35, 36, 37, 38, 39, 40, 41, 43, 45, 46, 47, 48, 60, 61, 62, 63, 67, 69, 72, 73]. The thesis uses K-3DGS as a baseline precisely because this family is the most plausible alternative to NeRF for real-time robotic self-modeling.

Gaussian and explicit-world-model systems also matter beyond self-rendering: they increasingly serve as physics-consistent robot twins, differentiable real-to-sim engines, simulator assets, spatial world models, and planning interfaces for manipulation [55, 56, 57, 58, 75, 76, 77]. These works reinforce the thesis interpretation of K-3DGS as an important baseline. The fact that NeuroKin is faster does not invalidate Gaussian robotics; it shows that when the task is in-loop self-diagnosis rather than general scene reconstruction, a narrower decoder can be the better architectural choice.

G.3 Embodiment Scaling and Cross-Embodiment Learning

The broader survey also frames the scaling problem. Future self-models must generalize across high-degree-of-freedom manipulators, hands, humanoids, soft robots, aerial morphologies, and cross-embodiment policies. Related work on morphology-aligned dex-

terous grasping, graph embodiment transformers, legged and hybrid locomotion, whole-body coordination, sim-distilled policies, exoskeleton-like Gaussian systems, grounded skill learning, agile humanoid systems, H2O transfer, VIRAL-style sim-to-real, and force or language-conditioned robot policies shows how wide the embodiment space is [42, 44, 49, 50, 51, 52, 53, 54, 64, 65, 66, 70, 71, 74, 78, 79, 81, 82, 83]. NeuroKin is deliberately not claimed to solve all of these scaling challenges. Its contribution is narrower and earlier in the pipeline: proving that a learned self-model can satisfy the inference budget that those larger systems would eventually require.

Digital-twin and sim-to-real work clarifies the next validation stage. Real-to-sim-to-real workflows, Webots and physics simulators, robotic cutting adaptation, style and energy-based synthetic worlds, physical twin systems, reality-gap analyses, and scalable evaluation frameworks all show that simulation results must eventually pass through domain shift and hardware stress tests [84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 80, 141, 142]. This is why the thesis states the simulation boundary repeatedly. PyBullet is appropriate for isolating the latency problem, but not sufficient for claiming hardware-ready robustness.

Robot-policy and generative-environment work further expands the deployment horizon. Recent systems investigate real-to-render-to-real data scaling, expert generation, twin-driven reinforcement learning, world-model-based policy optimization, multimodal robotic agents, spatially grounded VLA systems, dream-based simulators, mobile robot digital twins, VR-real-to-sim-to-real frameworks, cycle consistency, geometric locomotion, robust perception learning, physics-guided motion generation, safe flow, scaling simulation, navigation in Gaussian simulators, and digital-twin evaluation [110, 111, 112, 113, 114, 115, 116, 117, 118, 119, 120, 121, 122, 123, 124, 125, 126, 128]. These papers are relevant because they demonstrate the ecosystem into which fast self-modeling would be inserted: policy learning, evaluation, synthetic data, and deployment-time state monitoring.

Additional perception, locomotion, table-top, and manipulation systems show the same trend from a different angle: robust deployment depends on pose tracking, soft or clutter-aware control, transferable residuals, parkour-scale perception, open-world reasoning, simulated data generation, omnidirectional humanoid locomotion, hybrid actor systems, and latent athletic policy learning [68, 94, 95, 97, 98, 99, 100, 101, 102, 103, 104, 105, 106, 107, 108, 109]. These references broaden the scope of the survey, but they also sharpen the thesis boundary: NeuroKin is a self-modeling latency contribution, not a complete manipulation or locomotion policy.

G.4 Damage Recovery, Safety, and Continual Adaptation

Damage recovery and safety literature defines the functional standard that visual self-models must meet. Online synchronization, legged and humanoid adaptation, agile policy learning, generalized task-driven design, world models, embodied dynamics, cyber-waiter style agentic systems, hierarchical recovery, antenna-based tactile feedback, hybrid internal models, slow and risk-bounded reinforcement learning, soft-map transfer, safe learning surveys, safe-stoppability monitoring, self-adapting systems, and Splat2Real-style transfer all point toward the same requirement: adaptive robots need reliable diagnosis and recovery under changing morphology and environment [127, 129, 130, 131, 132, 133, 134, 135, 136, 137, 138, 139, 140, 143, 144, 145, 146, 59, 147, 148]. NeuroKin’s sub-millisecond inference is meaningful because recovery systems cannot wait hundreds of milliseconds for self-diagnosis when damage affects the control loop immediately.

More targeted recovery and fault-monitoring works motivate the thesis emphasis on failure counters and conservative recovery modes. TRANSIC, Swim2Real, large-scale reality-gap closing, generic anomaly detection, contrastive fault-tolerant reinforcement learning, VLM-based failure recovery, residual control, DEFT-style damaged operation, embedding-based methods, preference-conditioned control, actionable recovery, and fault-tolerant bimanual learning show that modern robotic recovery depends on compressed diagnostic signals that can trigger behavior changes [149, 150, 151, 152, 153, 154, 155, 156, 157, 158, 159, 160]. The sequential swarm ledger in this thesis is intentionally simple, but it follows the same design pattern: local failure evidence is transformed into a state variable that changes later behavior.

Continual learning references identify the open problem that remains after fast inference. Catastrophic interference, stability-plasticity theory, Elastic Weight Consolidation, lottery-ticket sparsity, adaptive sparse connectivity, compression, meta-learning, and curiosity-driven exploration all inform future work on self-model maintenance after damage [161, 162, 163, 164, 165, 166, 167, 168, 169, 170]. NeuroKin does not by itself solve catastrophic forgetting. It makes the problem more tractable by providing fast discrepancy detection and a lightweight model that could be embedded in modular, sparse, or meta-learned update schemes.

Finally, dexterous manipulation, tactile imitation, perception-integrated low-level control, steady object handling, force-adaptive reinforcement learning, master-micro residual correction, aquatic proprioception, manufacturable sensor-layout optimization, salamander whole-body control, and dynamics-aware clutter manipulation illustrate the broader embodied-control frontier [171, 172, 173, 174, 175, 176, 177, 179, 180, 182]. These systems require self-state, task-state, and contact-state estimates that are timely enough to influ-

ence control. The thesis contribution should be read as a latency-focused component for this larger research agenda, not as a replacement for the full stack of physical intelligence.

Bibliography

Bibliography

- [1] J. Bongard, V. Zykov, and H. Lipson, “Resilient machines through continuous self-modeling,” *Science*, vol. 314, no. 5802, pp. 1118–1121, 2006.
- [2] A. Cully, J. Clune, D. Tarapore, and J.-B. Mouret, “Robots that can adapt like animals,” *Nature*, vol. 521, no. 7553, pp. 503–507, 2015.
- [3] M. C. Carrozza, G. Cappiello, S. Micera, B. B. Edin, L. Beccai, and C. Cipriani, “Design of a cybernetic hand for perception and action,” *Biological Cybernetics*, vol. 95, no. 9, pp. 629–644, 2006.
- [4] S. Koos, A. Cully, and J.-B. Mouret, “Fast damage recovery in robotics with the T-Resilience algorithm,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2013.
- [5] J.-B. Mouret and J. Clune, “Illuminating search spaces by mapping elites,” in *Proc. Genet. Evol. Comput. Conf. (GECCO)*, 2015.
- [6] R. Kwiatkowski and H. Lipson, “Task-agnostic self-modeling machines,” *Science Robotics*, vol. 4, no. 26, p. eaau9354, 2019.
- [7] R. Kwiatkowski and H. Lipson, “Zero-shot learning on simulated robots,” in *Proc. Int. Conf. Mach. Learn. (ICML)*, pp. 1–7, 2019.
- [8] Y. Hu, Y. Wang, R. Liu, Z. Shen, and H. Lipson, “Reconfigurable robot identification from motion data,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2024.
- [9] A. Dogra, S. Mahan, S. S. Padhee, and E. Singla, “Unified modeling of unconventional modular and reconfigurable manipulation system,” *Robotics and Computer-Integrated Manufacturing*, vol. 77, p. 102385, 2022.
- [10] S. Farghdani, M. Patel, and R. Chhabra, “Robust embodied self-identification of morphology in damaged multi-legged robots,” *IEEE Robot. Autom. Lett.*, vol. 10, pp. 1–8, 2025.
- [11] C. Yu and S. Kriegman, “Robots that redesign themselves through kinematic self-destruction,” arXiv preprint arXiv:2603.12505, 2026.

- [12] B. Chen, R. Kwiatkowski, C. Vondrick, and H. Lipson, “Full-body visual self-modeling of robot morphologies,” *Science Robotics*, vol. 7, no. 68, p. eabn1944, 2022.
- [13] Y. Hu, J. Lin, and H. Lipson, “Teaching robots to build simulations of themselves,” *Nature Machine Intelligence*, vol. 6, pp. 123–130, 2024.
- [14] L. Schulze and H. Lipson, “High-degrees-of-freedom dynamic neural fields for robot self-modeling and motion planning,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2024.
- [15] Y. Hu, B. Chen, and H. Lipson, “Egocentric visual self-modeling for autonomous robot dynamics prediction and adaptation,” arXiv preprint arXiv:2207.03386, 2022.
- [16] M. Liu *et al.*, “Differentiable robot rendering: Bridging pixels and control through differentiable rendering,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2024.
- [17] S. Rezvani, A. J. Mahmood, and R. Chhabra, “Robust visual embodiment: How robots discover their bodies in real environments,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2024.
- [18] K. Hu, P. Yu, and N. Tan, “Learning high-fidelity robot self-model with articulated 3D Gaussian splatting,” *Int. J. Robotics Research*, 2025.
- [19] P. Yu, X. Wang, and N. Tan, “Shape-interpretable visual self-modeling enables geometry-aware continuum robot control,” arXiv preprint arXiv:2603.01751, 2026.
- [20] J. Lee, H. Kim, J. Choi, and S. Nam, “TactiVerse: Generalizing multi-point tactile sensing in soft robotics using single-point data,” arXiv preprint arXiv:2602.19850, 2026.
- [21] M. Li, S. Maeng, M. Murphy, Y. Chen, and O. Kroemer, “FlowTouch: View-invariant visuo-tactile prediction,” arXiv preprint arXiv:2603.08255, 2026.
- [22] J. T. Barron, B. Mildenhall, M. Tancik, P. Hedman, R. Martin-Brualla, and P. P. Srinivasan, “Mip-NeRF: A multiscale representation for anti-aliasing neural radiance fields,” in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2021, pp. 5855–5864.
- [23] Y. Li, S. Li, V. Sitzmann, P. Agrawal, and A. Torralba, “3D neural scene representations for visuomotor control,” in *Proc. Conf. Robot. Learn. (CoRL)*, 2021.
- [24] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and A. Y. Ng, “NeRF: Representing scenes as neural radiance fields for view synthesis,” in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2020.

- [25] A. Pumarola, E. Corona, G. Pons-Moll, and A. Sanfeliu, “D-NeRF: Neural radiance fields for dynamic scenes,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2021.
- [26] T. Müller, A. Evans, C. Schied, and A. Keller, “Instant neural graphics primitives with a multiresolution hash encoding,” *ACM Trans. Graph.*, vol. 41, no. 4, pp. 102:1–102:15, 2022.
- [27] J. T. Barron, B. Mildenhall, D. Verbin, P. P. Srinivasan, and P. Hedman, “Zip-NeRF: Anti-aliased grid-based neural radiance fields,” in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2023.
- [28] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, “3D Gaussian splatting for real-time radiance field rendering,” *ACM Trans. Graph.*, vol. 42, no. 4, pp. 139:1–139:14, 2023.
- [29] H. Matsuki, R. Murai, P. H. J. Kelly, and A. J. Davison, “Gaussian splatting SLAM,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024, pp. 18039–18048.
- [30] N. Keetha *et al.*, “SplaTAM: Splat, track and map 3D Gaussians for dense RGB-D SLAM,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024, pp. 18410–18420.
- [31] S. Li, B. Keller, Y. Lin, and B. Khailany, “GauRast: Enhancing GPU triangle rasterizers to accelerate 3D Gaussian splatting,” in *Proc. Int. Symp. Comput. Archit. (ISCA)*, 2025.
- [32] A. Guedon and V. Lepetit, “SuGaR: Surface-aligned Gaussian splatting for efficient 3D mesh reconstruction and high-quality mesh rendering,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024.
- [33] K. Kim, K. Wu, D. Kerr, K. Goldberg, M. Tancik, and A. Kanazawa, “GARField: Group Anything with Radiance Fields,” in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2024.
- [34] Y. Weng *et al.*, “Neural implicit representation for building digital twins of unknown articulated objects,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024.
- [35] Z. Li *et al.*, “ART: Articulated reconstruction transformer,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2025.

- [36] Y. Wu *et al.*, “AOMGen: Photoreal, physics-consistent demonstration generation for articulated object manipulation,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2026.
- [37] Z. Wu *et al.*, “URDF-Anything+: Autoregressive articulated 3D models generation for physical simulation,” arXiv preprint arXiv:2502.09850, 2025.
- [38] W. Xu, L. Liu, L. Zhang, D. Guo, and R. Liu, “MotionAnymesh: Physics-grounded articulation for simulation-ready digital twins,” arXiv preprint arXiv:2603.12936, 2026.
- [39] J. Wang, D. Wang, J. Hu, Q. Zhang, J. Yu, and L. Xu, “Kinematify: Open-vocabulary synthesis of high-DoF articulated objects,” arXiv preprint arXiv:2511.01294, 2025.
- [40] C. Zhang, M. Qin, Y. Wang, B. Xie, H. Li, and Z. Wang, “SIMART: Decomposing monolithic meshes into sim-ready articulated assets via MLLM,” arXiv preprint arXiv:2603.23386, 2026.
- [41] P. Wang, G. Ke, M. Rong, X. Guo, K. Ming, and S. Liu, “ArtLLM: Generating articulated assets via 3D LLM,” arXiv preprint arXiv:2603.01142, 2026.
- [42] Z. Wang *et al.*, “General humanoid whole-body control via pretraining and fast adaptation,” arXiv preprint arXiv:2602.11929, 2026.
- [43] J. Guo, Y. Xin, G. Liu, K. Xu, L. Liu, and R. Hu, “ArticulatedGS: Self-supervised digital twin modeling of articulated objects using 3D Gaussian splatting,” arXiv preprint arXiv:2503.08135, 2025.
- [44] K. Guo, H. Liu, Y. Sun, R. Zhao, J. Zhou, and J. Ma, “MoE-ACT: Scaling multi-task bimanual manipulation with sparse language-conditioned mixture-of-experts transformers,” arXiv preprint arXiv:2603.15265, 2026.
- [45] Q. Yu, Z. Xia, C. Li, D. Pan, X. Wang, and J. Liu, “ArtGS: 3D Gaussian splatting for interactive visual-physical modeling and manipulation of articulated objects,” arXiv preprint arXiv:2507.02600, 2025.
- [46] D. Wu, Z. Wang, Y. Luo, H. Zhang, T. Chen, and K. Guo, “REArtGS++: Generalizable articulation reconstruction with temporal geometry constraint via planar Gaussian splatting,” arXiv preprint arXiv:2511.17059, 2025.
- [47] H. Dai, H. Fan, H. Zhang, D. Wu, J. Zhang, and H. Dong, “FreeArtGS: Articulated Gaussian splatting under free-moving scenario,” arXiv preprint arXiv:2603.22102, 2026.

- [48] Q. Chen *et al.*, “FreeGaussian: Annotation-free control of articulated objects via 3D Gaussian splats with flow derivatives,” in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, 2026.
- [49] Z. Chen *et al.*, “AdaClearGrasp: Learning adaptive clearing for zero-shot robust dexterous grasping in densely cluttered environments,” arXiv preprint arXiv:2603.10616, 2026.
- [50] Y. Li *et al.*, “ForceVLA2: Unleashing hybrid force–position control with force awareness for contact-rich manipulation,” arXiv preprint arXiv:2603.15169, 2026.
- [51] A. Miyamichi, M. Zhao, K. Sugihara, J. Sugihara, M. Konishi, K. Kojima, K. Okada, and M. Inaba, “Flexible morphing aerial robot with inflatable structure for perching-based human–robot interaction,” arXiv preprint arXiv:2509.07496, 2025.
- [52] J. Tan *et al.*, “Sim-to-Real: Learning agile locomotion for quadruped robots,” in *Proc. Robot.: Sci. Syst. (RSS)*, 2018.
- [53] X. Tan, W. Xie, and N. Correll, “Characterization, analytical planning, and hybrid force control for the Inspire RH56DFX hand,” arXiv preprint arXiv:2603.08988, 2026.
- [54] R. K. Thomas, “Embodied cognition-inspired robot navigation using neural field dynamics and perceptual binding,” *Int. J. Adv. Signal Image Sci.*, vol. 12, no. 1, pp. 973–984, 2026.
- [55] H. Lou *et al.*, “Robo-GS: A physics consistent spatial-temporal model for robotic arm with hybrid representation,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2025.
- [56] H. Lou *et al.*, “D-REX: Differentiable real-to-sim-to-real engine for learning dexterous grasping,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2026.
- [57] Z. Sun, L. Bao, T. Peng, J. Sun, and C. Zhou, “A high-fidelity digital twin for robotic manipulation based on 3D Gaussian splatting,” arXiv preprint arXiv:2601.03200, 2026.
- [58] X. Li, G. Chen, and J. Alonso-Mora, “Building explicit world model for zero-shot open-world object manipulation,” arXiv preprint arXiv:2603.13825, 2026.
- [59] Y. Sun, Y. Pan, S. Li, C. Ding, T. Cui, L. Wang, and C. Liu, “Learning safe-stoppability monitors for humanoid robots,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2026.

- [60] G. Lu, S. Zhang, Z. Wang, C. Liu, J. Lu, and Y. Tang, “ManiGaussian: Dynamic Gaussian splatting for multi-task robotic manipulation,” in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2024, pp. 349–366.
- [61] S. Pan, Y. Xu, R. Xu, Z. Zhou, S. Wu, and Z. Yu, “Self-correcting robot manipulation via Gaussian-splatted foresight,” in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, 2025.
- [62] S. Yang, K. Zhan, Y. Zhang, L. Lin, X. Huang, and M. Li, “Novel demonstration generation with Gaussian splatting enables robust one-shot manipulation,” arXiv preprint arXiv:2504.13175, 2025.
- [63] T. Tang, L. Zhang, Y. Wen, K. Zhang, J.-W. Bian, X. Zhou, T. Yan, K. Zhan, P. Jia, H. Wu, L. Lin, and X. Liang, “RoboPearls: Editable video simulation for robot manipulation,” arXiv preprint arXiv:2506.22756, 2025.
- [64] Y. Wu, S. Bae, Y. Chen, L. Lyu, S. Li, Z. Huang, and X. Yang, “DexGrasp-Zero: A morphology-aligned policy for zero-shot cross-embodiment dexterous grasping,” arXiv preprint arXiv:2603.16806, 2025.
- [65] A. Patel and S. Song, “GET-Zero: Graph embodiment transformer for zero-shot embodiment generalization,” in *Proc. Robot.: Sci. Syst. (RSS)*, 2024.
- [66] A. Patrizi, C. Rizzardo, A. Laurenzi, F. Ruscelli, L. Rossini, and N. G. Tsagarakis, “RL-augmented MPC for non-gaited legged and hybrid locomotion,” arXiv preprint arXiv:2603.10878, 2026.
- [67] L. Lin *et al.*, “Persistent object Gaussian splat (POGS) for tracking and robot manipulation of irregularly shaped objects,” arXiv preprint arXiv:2501.14523, 2025.
- [68] B. Wen, W. Yang, J. Kautz, and S. Birchfield, “FoundationPose: Unified 6D pose estimation and tracking of novel objects,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024.
- [69] Z. Xie, J. Gong, X. Tan, Z. Zhang, and Y. Xie, “Zero-shot robotic manipulation via 3D Gaussian splatting-enhanced multimodal retrieval-augmented generation,” arXiv preprint arXiv:2603.00500, 2026.
- [70] Q. Liang *et al.*, “Whole-body coordination for dynamic object grasping with legged manipulators,” arXiv preprint arXiv:2508.08328, 2025.
- [71] J. Levy *et al.*, “Simulation distillation: Pretraining world models in simulation for rapid real-world adaptation,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2024.

- [72] Y. Lu *et al.*, “GSMem: 3D Gaussian splatting as persistent spatial memory for zero-shot embodied exploration and reasoning,” arXiv preprint arXiv:2603.19137, 2025.
- [73] A. Zhang, Y. Tang, T. Li, B. Zhang, Z. Chen, and J. Sun, “Actron3D: Learning actionable neural functions from videos for transferable robotic manipulation,” arXiv preprint arXiv:2510.12971, 2025.
- [74] Y. Wang *et al.*, “ExoGS: A 4D real-to-sim-to-real framework for scalable manipulation data collection,” arXiv preprint arXiv:2601.18629, 2026.
- [75] Z. Gao *et al.*, “Mirage2Matter: A physically grounded Gaussian world model from video,” in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2024.
- [76] H. Jiang *et al.*, “PhysTwin: Physics-informed reconstruction and simulation of deformable objects from videos,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2025.
- [77] G. Jiang *et al.*, “GSWorld: Closed-loop photo-realistic simulation suite for robotic manipulation,” arXiv preprint arXiv:2510.20813, 2025.
- [78] A. Escontrela *et al.*, “GaussGym: An open-source real-to-sim framework for learning locomotion from pixels,” arXiv preprint arXiv:2510.15352, 2025.
- [79] R. Jin *et al.*, “Grounding sim-to-real generalization in dexterous manipulation: An empirical study with vision-language-action models,” arXiv preprint arXiv:2603.22876, 2026.
- [80] E. Aljalbout, A. Frick, X. Chen, T. Westenbroek, and D. Fridovich-Keil, “The reality gap in robotics: Challenges, solutions, and best practices,” *Annu. Rev. Control Robot. Auton. Syst.*, vol. 9, 2026.
- [81] H. Zhao *et al.*, “AGILE: A comprehensive workflow for humanoid loco-manipulation learning,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2026.
- [82] T. He, Z. Luo, W. Xiao, C. Zhang, K. Kitani, C. Liu, and G. Shi, “Learning human-to-humanoid real-time whole-body teleoperation,” arXiv preprint arXiv:2403.04436, 2024.
- [83] T. He, D. Seita, A. Kanazawa, and P. Abbeel, “VIRAL: Visual sim-to-real at scale for humanoid loco-manipulation,” arXiv preprint arXiv:2511.15200, 2025.

- [84] M. Torne, A. Simeonov, Z. Li, A. Chan, T. Chen, A. Gupta, and P. Agrawal, “Reconciling reality through simulation: A real-to-sim-to-real approach for robust manipulation,” in *Proc. Robot.: Sci. Syst. (RSS)*, 2024.
- [85] Y. W. Hadi, L. Hof, B. Jayawardhana, and B. Haghighat, “Developing simulation models for soft robotic grippers in Webots,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2021.
- [86] J. Hathaway, A. Rastegarpanah, and R. Stolkin, “Imitation learning for sim-to-real adaptation of robotic cutting policies based on residual Gaussian process disturbance force model,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2024.
- [87] J. Hathaway, A. Rastegarpanah, and R. Stolkin, “End-to-end example-based sim-to-real RL policy transfer based on neural stylisation with application to robotic cutting,” arXiv preprint arXiv:2601.20846, 2026.
- [88] S. Huang, L. Chen, P. Zhou, S. Chen, Y. Liao, Z. Jiang, Y. Hu, P. Gao, H. Li, M. Yao, and G. Ren, “ENERVERSE: Envisioning embodied future space for robotics manipulation,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2025.
- [89] E. Heiden, Z. Liu, V. Vineet, E. Coumans, and G. S. Sukhatme, “Inferring articulated rigid body dynamics from RGBD video,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2023.
- [90] J. Abou-Chakra, Y. Zhu, L. Zhang, D. Fridovich-Keil, and P. Agrawal, “Real-is-sim: Bridging the sim-to-real gap with a dynamic digital twin,” arXiv preprint arXiv:2504.03597, 2025.
- [91] Z. Jiang, H. Takemura, and M. Kita, “Digital twin system for home service robot based on motion simulation,” *J. Robot. Mechatron.*, vol. 36, no. 2, pp. 420–432, 2024.
- [92] C. Zhang, Y. Zou, Z. Wu, Y. Ling, Y. Yang, and Z. Wang, “UniPR: Unified object-level real-to-sim perception and reconstruction from a single stereo pair,” arXiv preprint arXiv:2603.19616, 2026.
- [93] Y. Cai, P. Jansson, C. de Farias, O. Arenz, and J. Peters, “GaussTwin: Unified simulation and correction with Gaussian splatting for robotic digital twins,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2025.
- [94] M. Crotti, L. Rossini, B. K. Hodossy, A. Pace, G. Grioli, A. Bicchi, and M. G. Catalano, “Soft adaptive feet for legged robots: An open-source model for locomotion simulation,” *IEEE Robot. Autom. Lett.*, vol. 10, 2024.

- [95] J. Collins, A. Liang, J. Malik, H. Zhang, and F. Devernay, “CA²T-Net: Category-agnostic 3D articulation transfer from single image,” in *Proc. Int. Conf. 3D Vis. (3DV)*, 2023.
- [96] K. Park, U. Sinha, J. T. Barron, S. Bouaziz, D. B. Goldman, S. M. Seitz, and R. Martin-Brualla, “Nerfies: Deformable neural radiance fields,” in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2021.
- [97] D. H. Cho and B. Chen, “RAFL: Generalizable sim-to-real of soft robots with residual acceleration field learning,” arXiv preprint arXiv:2603.22039, 2026.
- [98] P. Kotecha, G. Nair, and S. Kolathaya, “STRIDE: Structured Lagrangian and stochastic residual dynamics via flow matching,” arXiv preprint arXiv:2603.08478, 2026.
- [99] K. Wang, L. Lu, M. Liu, J. Jiang, Z. Li, B. Zhang, W. Zheng, X. Yu, H. Chen, and C. Shen, “ODYSSEY: Open-world quadrupeds exploration and manipulation for long-horizon tasks,” arXiv preprint arXiv:2508.08240, 2025.
- [100] Y. Wang *et al.*, “WESTWORLD: A knowledge-encoded scalable trajectory world model for diverse robotic systems,” arXiv preprint arXiv:2603.14392, 2026.
- [101] Z. Wu *et al.*, “Perceptive humanoid parkour: Chaining dynamic human skills via motion matching,” arXiv preprint arXiv:2602.15827, 2026.
- [102] J. Pan, J. Xing, R. Reiter, Y. Zhai, E. Aljalbout, and D. Scaramuzza, “Learning on the fly: Rapid policy adaptation via differentiable simulation,” *IEEE Robot. Autom. Lett.*, vol. 10, 2025.
- [103] M. N. Qureshi, Y. Tang, Z. Zhang, X. Li, Y. Wang, and A. Gupta, “SplatSim: Zero-shot sim2real transfer of RGB manipulation policies using Gaussian splatting,” arXiv preprint arXiv:2404.13099, 2025.
- [104] J. Tang, Z. He, K. Zhang, Y. Liu, M. Chen, and W. Xu, “Bridging simulation and reality: Cross-domain transfer with semantic 2D Gaussian splatting,” arXiv preprint arXiv:2512.04731, 2025.
- [105] G. Zhao *et al.*, “System design for maintaining internal state consistency in long-horizon robotic tabletop games,” arXiv preprint arXiv:2603.25405, 2026.
- [106] H. Zhao *et al.*, “High-fidelity simulated data generation for real-world zero-shot robotic manipulation learning with Gaussian splatting,” arXiv preprint arXiv:2510.10637, 2026.

- [107] Y. Jiang, Y. Liang, J. Li, H. Ding, and L. Zhu, “Omnidirectional humanoid locomotion on stairs via unsafe stepping penalty and sparse LiDAR elevation mapping,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2026.
- [108] D. Li *et al.*, “HAIC: Humanoid agile object interaction control via dynamics-aware world model,” arXiv preprint arXiv:2602.11758, 2026.
- [109] Z. Zhang *et al.*, “Learning athletic humanoid tennis skills from imperfect human motion data,” arXiv preprint arXiv:2603.12686, 2026.
- [110] J. Yu, H. Su, D. Fridovich-Keil, and A. Gupta, “Real2Render2Real: Scaling robot data without dynamics simulation or robot hardware,” arXiv preprint arXiv:2505.09601, 2025.
- [111] Z. Xu *et al.*, “ExpertGen: Scalable sim-to-real expert policy learning from imperfect behavior priors,” arXiv preprint arXiv:2603.15956, 2026.
- [112] Q. Xu, Y. Liang, M. Chen, Z. Zhang, K. Xu, H. Wang, and J. Wang, “TwinRL-VLA: Digital twin-driven reinforcement learning for real-world robotic manipulation,” arXiv preprint arXiv:2602.09023, 2025.
- [113] Z. Zhang *et al.*, “Towards practical world model-based reinforcement learning for vision-language-action models,” arXiv preprint arXiv:2603.20607, 2026.
- [114] A. Deshpande, B. Chen, Z. Li, R. Shen, Z. Xu, and L. Fei-Fei, “MolmoB0T: Large-scale simulation enables zero-shot manipulation,” arXiv preprint arXiv:2603.16861, 2026.
- [115] R. Tu, A. Shukla, S. Yoo, X. Li, J. Xie, H. Su, and Z. Tu, “SG-VLA: Learning spatially-grounded vision-language-action models for mobile manipulation,” arXiv preprint arXiv:2603.22760, 2026.
- [116] S. He *et al.*, “V-Dreamer: Automating robotic simulation and trajectory synthesis via video generation priors,” arXiv preprint arXiv:2603.18811, 2026.
- [117] Y. Mei, P. Qiu, W. Zhang, W. Zhang, and W. Song, “MobRT: A digital twin-based framework for scalable learning in mobile manipulation,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2026.
- [118] S. Zhu, L. Mou, D. Li, B. Ye, R. Huang, and H. Zhao, “VR-Robo: A real-to-sim-to-real framework for visual robot navigation and locomotion,” *IEEE Robot. Autom. Lett.*, vol. 10, no. 5, 2025.

- [119] G. Liu, T. Wang, Z. Wu, J. Wu, S. Li, and X. Zhu, “CycleRL: Sim-to-real deep reinforcement learning for robust autonomous bicycle control,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2024.
- [120] Y. Liu *et al.*, “GeoLoco: Leveraging 3D geometric priors from visual foundation model for robust RGB-only humanoid locomotion,” arXiv preprint arXiv:2603.07624, 2026.
- [121] Y. Zhang *et al.*, “RPL: Learning robust humanoid perceptive locomotion on challenging terrains,” arXiv preprint arXiv:2602.03002, 2026.
- [122] J. Bao *et al.*, “Physics-prefix guided motion generation for agile general humanoid motion tracking,” arXiv preprint arXiv:2603.19305, 2026.
- [123] H. Cho, S.-H. Kim, J. Kang, and D. Koo, “SafeFlow: Real-time text-driven humanoid whole-body control via physics-guided rectified flow and selective safety gating,” arXiv preprint arXiv:2603.23983, 2026.
- [124] A. Choi, X. Wang, Z. Su, and W. Xu, “Scaling sim-to-real reinforcement learning for robot VLAs with generative 3D worlds,” in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2025.
- [125] J. Liu *et al.*, “NavGSim: High-fidelity Gaussian splatting simulator for large-scale navigation,” *IEEE Robot. Autom. Lett.*, vol. 10, no. 5, 2025.
- [126] Z. Mari, M. M. Nawaf, and P. Drap, “Digital twin-supervised reinforcement learning framework for autonomous underwater navigation,” *Sensors*, 2024.
- [127] Z. Niu, X. Chen, J. Hu, Z. Liu, J. Tang, and X. Ju, “Synchronized online friction estimation and adaptive grasp control for robust gentle grasp,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2026.
- [128] V. Murugesan, R. Mathiazhagan, S. Joshi, and A. Arab, “Digital-twin evaluation for proactive human-robot collision avoidance via prediction-guided A-RRT*,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2024.
- [129] T. Miki, J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter, “Learning robust perceptive locomotion for quadrupedal robots in the wild,” *Science Robotics*, vol. 7, no. 62, 2022.
- [130] F. Miki, C. Pasareanu, T. Yamamoto, and K. Yoshida, “Learning proprioceptive compliance and contact state estimation for legged locomotion with sim-to-real transfer,” in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2024.

- [131] Q. Peng, Y. Lin, Y. Xue, J. Pang, and W. Zhang, “Embodiment-aware generalist-specialist distillation for unified humanoid whole-body control,” arXiv preprint arXiv:2602.02960, 2026.
- [132] J. Xing, Z. Wang, Y. Liu, P. Kumar, K. Zhang, and D. Song, “AGILE: Autonomous guided learning infrastructure for policy execution in humanoid robotics,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2025.
- [133] Y. Yao, D. Howard, and P. Maiolino, “Generalized task-driven design of soft robots via reduced-order FEM-based surrogate modeling,” arXiv preprint arXiv:2603.19794, 2026.
- [134] C. Yin *et al.*, “Genie Sim 3.0: A high-fidelity comprehensive simulation platform for humanoid robot,” arXiv preprint arXiv:2601.02078, 2026.
- [135] T. Yin *et al.*, “WoMAP: World models for embodied open-vocabulary object localization,” in *Proc. Conf. Robot. Learn. (CoRL)*, 2025.
- [136] A. Rajeswaran, V. Kumar, A. Gupta, and A. Levine, “Articulated-body dynamics network: Dynamics-grounded prior for robot learning,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2025.
- [137] J. Ren *et al.*, “Cybo-Waiter: A physical agentic framework for humanoid whole-body locomotion-manipulation,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2026.
- [138] A. Rigo, M. Hu, S. K. Gupta, and Q. Nguyen, “Hierarchical optimization-based control for whole-body loco-manipulation of heavy objects,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2024.
- [139] Z. J. Xu *et al.*, “A robust antenna provides tactile feedback in a multi-legged robot,” arXiv preprint arXiv:2603.07795, 2026.
- [140] H. Xu *et al.*, “Dual-horizon hybrid internal model for low-gravity quadrupedal jumping with hardware-in-the-loop validation,” arXiv preprint arXiv:2603.07999, 2026.
- [141] A. Jain, R. Sharma, S. Kumar, S. Kolathaya, Z. Zhang, and D. Song, “PolaRiS: Scalable real-to-sim evaluations for generalist robot policies,” arXiv preprint arXiv:2512.16881, 2025.
- [142] X. Li *et al.*, “Evaluating real-world robot manipulation policies in simulation,” in *Proc. Conf. Robot. Learn. (CoRL)*, 2024.

- [143] E. Daneshmand, S. Omar, G. Berseth, M. Khadiv, and H.-C. Lin, “SLOWRL: Safe low-rank adaptation reinforcement learning for locomotion,” arXiv preprint arXiv:2603.17092, 2026.
- [144] F. Meng, Z. Yang, X. Mao, H. Liang, and M. Q.-H. Meng, “Provably safe trajectory generation for manipulators under motion and environmental uncertainties,” arXiv preprint arXiv:2603.09083, 2026.
- [145] Z. Ma, U. Yoo, J. Francis, W. Zhi, J. Ichnowski, and J. Oh, “SOFTMAP: Sim2real soft robot forward modeling via topological mesh alignment and physics prior,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2025.
- [146] L. Brunke *et al.*, “Safe learning in robotics: From learning-based control to safe reinforcement learning,” *Annu. Rev. Control Robot. Auton. Syst.*, vol. 5, pp. 1–43, 2022.
- [147] F. Domberg and G. Schilbach, “Self-adapting robotic agents through online continual reinforcement learning with world model feedback,” arXiv preprint arXiv:2603.04029, 2026.
- [148] H. Lim and J. B. Choi, “Splat2Real: Novel-view scaling for physical AI with 3D Gaussian splatting,” arXiv preprint arXiv:2603.10638, 2026.
- [149] Y. Jiang, C. Wang, R. Zhang, J. Wu, and L. Fei-Fei, “TRANSIC: Sim-to-real policy transfer by learning from online correction,” in *Proc. Conf. Robot. Learn. (CoRL)*, 2024.
- [150] K. Qiu, K. Walker, M. Y. Michelis, M. Cygan, and J. Hughes, “Swim2Real: VLM-guided system identification for sim-to-real transfer,” arXiv preprint arXiv:2603.20827, 2026.
- [151] ByteDance Seed, “Closing the reality gap: Zero-shot sim-to-real deployment for dexterous force-based grasping and manipulation,” arXiv preprint arXiv:2601.02778, 2026.
- [152] N. Grambow *et al.*, “Anomaly detection for generic failure monitoring in robotic assembly, screwing and manipulation,” *IEEE Robotics and Automation Letters*, vol. 11, no. 5, pp. 3664–3671, 2026.
- [153] Y. Fu, X. Chen, B. Zhang, M. Yang, and K. Zhang, “Contrastive forward prediction reinforcement learning for adaptive fault-tolerant legged robots,” in *Proc. Conf. Robot. Learn. (CoRL)*, 2025.

- [154] H. Chen, Y. Yao, R. Liu, C. Liu, and J. Ichnowski, “Automating robot failure recovery using vision-language models with optimized prompts,” arXiv preprint arXiv:2409.03966, 2024.
- [155] N. Jayasinghe *et al.*, “Residual control for fast recovery from dynamics shifts,” arXiv preprint arXiv:2603.07775, 2026.
- [156] G. G. Briscoe-Martinez, Y. Gautam, R. Shetty, A. Pasricha, M. M. Nicotra, and A. Roncone, “Moving on, even when you’re broken: Fail-active trajectory generation via diffusion policies conditioned on embodiment and task,” arXiv preprint arXiv:2602.02895, 2026.
- [157] N. Poddar, S. McCrory, L. Penco, G. Clark, H. E. Sevil, and R. Griffin, “Embedding classical balance control principles in reinforcement learning for humanoid recovery,” arXiv preprint arXiv:2603.08619, 2026.
- [158] H. Li *et al.*, “PCHC: Enabling preference-conditioned humanoid control via multi-objective reinforcement learning,” arXiv preprint arXiv:2603.24047, 2026.
- [159] D. Li *et al.*, “Learning actionable manipulation recovery via counterfactual failure synthesis,” arXiv preprint arXiv:2603.13528, 2026.
- [160] H. Lee, W.-J. Baek, J. Cha, and J. Park, “TOLEBI: Learning fault-tolerant bipedal locomotion via online status estimation and fallibility rewards,” arXiv preprint arXiv:2602.05596, 2026.
- [161] M. McCloskey and N. J. Cohen, “Catastrophic interference in connectionist networks: The sequential learning problem,” *Psychol. Learn. Motiv.*, vol. 24, pp. 109–165, 1989.
- [162] R. M. French, “Catastrophic forgetting in connectionist networks,” *Trends Cogn. Sci.*, vol. 3, no. 4, pp. 128–135, 1999.
- [163] S. Grossberg, “Competitive learning: From interactive activation to adaptive resonance,” *Cognitive Science*, vol. 11, no. 1, pp. 23–63, 1987.
- [164] M. Mermillod, A. Bugaiska, and P. Bonin, “The stability-plasticity dilemma: Investigating the continuum from catastrophic forgetting to age-limited learning effects,” *Front. Psychol.*, vol. 4, p. 504, 2013.
- [165] J. Kirkpatrick *et al.*, “Overcoming catastrophic forgetting in neural networks,” *Proc. Natl. Acad. Sci.*, vol. 114, no. 13, pp. 3521–3526, 2017.
- [166] J. Frankle and M. Carbin, “The lottery ticket hypothesis: Finding sparse, trainable neural networks,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2019.

- [167] D. C. Mocanu, E. Mocanu, P. Stone, P. H. Nguyen, M. Gibescu, and A. Liotta, “Scalable training of artificial neural networks with adaptive sparse connectivity inspired by network science,” *Nat. Commun.*, vol. 9, p. 2383, 2018.
- [168] S. Han, H. Mao, and W. J. Dally, “Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2016.
- [169] C. Finn, P. Abbeel, and S. Levine, “Model-agnostic meta-learning for fast adaptation of deep networks,” in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2017.
- [170] D. Pathak, P. Krahenbuhl, J. Donahue, T. Darrell, and A. A. Efros, “Curiosity-driven exploration by self-supervised prediction,” in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2017.
- [171] L. An, L. Lin, and Y. Xu, “Survey on dexterous manipulation: Recent progress and future challenges,” arXiv preprint arXiv:2603.06765, 2026.
- [172] Y. Zhao, W. Zhou, X. Li, and H. Li, “Tracing tactile dynamics for vision and touch based shape prediction,” in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2025, pp. 1–8.
- [173] X. Cui *et al.*, “PILOT: Enabling whole-body perceptive loco-manipulation in mobile quadruped robots,” arXiv preprint arXiv:2512.00843, 2025.
- [174] S. Huang *et al.*, “SteadyTray: Learning stabilization of unsecured payloads on bipedal humanoid movement,” arXiv preprint arXiv:2603.22397, 2026.
- [175] B. Pudasaini, K. Sreenath, and Y. Nakamura, “FAME: Force-adaptive manipulation via learned embodied experience on humanoid robots,” arXiv preprint arXiv:2603.06887, 2026.
- [176] X. Li *et al.*, “Master-micro residual correction with adaptive tactile fusion and force-mixed control for contact-rich manipulation,” arXiv preprint arXiv:2603.15152, 2026.
- [177] L. Micklem, H. Dong, F. Giorgio-Serchi, Y. Yang, B. Thornton, and G. D. Weymouth, “Harnessing proprioception in aquatic soft wings enables hybrid passive-active disturbance rejection,” *npj Robotics*, vol. 4, no. 16, 2026.
- [178] C. Thirgood, O. Mendez, E. C. Ling, J. Storey, and S. Hadfield, “FeatureSLAM: Feature-enriched 3D Gaussian splatting SLAM in real time,” arXiv preprint arXiv:2503.14256, 2025.

- [179] Y. Tian *et al.*, “Model-free co-optimization of manufacturable sensor layouts and deformation proprioception,” arXiv preprint arXiv:2603.10059, 2026.
- [180] M. Tian, Q. Fu, C. Ning, J. J. J. Pey, and A. J. Ijspeert, “Learning whole-body control for a salamander robot,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2024.
- [181] A. T. Tran and J. Kosecka, “VarSplat: Uncertainty-aware 3D Gaussian splatting for robust RGB-D SLAM,” arXiv preprint arXiv:2507.08762, 2025.
- [182] Y. Zheng *et al.*, “Emerging extrinsic dexterity in cluttered scenes via dynamics-aware policy learning,” arXiv preprint arXiv:2603.09882, 2026.